

BỘ GIÁO DỤC ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM VÀ KỸ THUẬT TP. HCM
KHOA CHẤT LƯỢNG CAO



HCMUTE

BÁO CÁO CUỐI KỲ
Môn học: Đồ án Công nghệ thông tin

Đề tài:

TÌM HIỂU VỀ APACHE SPARK VÀ
ỨNG DỤNG XÂY DỰNG MÔ HÌNH
DỰ ĐOÁN CẢM XÚC

Mã môn học: PROJ215879

Giảng viên hướng dẫn: ThS. LÊ THỊ MINH CHÂU

Sinh viên thực hiện: Cao Thị Ngọc Phụng 21110276

Nguyễn Phú Thành 21110299

Trần Văn Tiến 20110319

Nguyễn Văn Hào 21110175

Tp. Hồ Chí Minh, tháng 5 năm 2024

DANH SÁCH SINH VIÊN THAM GIA

Mã học phần: BDAN333977_24_1_02CLC

Nhóm: 02

Tên đề tài: *Tìm hiểu về Apache Spark và ứng dụng xây dựng mô hình dự đoán cảm xúc*

STT	HỌ VÀ TÊN THÀNH VIÊN	MÃ SỐ SINH VIÊN	TỶ LỆ THAM GIA
1	Nguyễn Phú Thành	21110299	100%
2	Cao Thị Ngọc Phụng	21110276	100%
3	Trần Văn Tiến	21110319	100%
4	Nguyễn Văn Hào	21110175	100%

Ghi chú: Tỷ lệ %: Mức độ phần trăm hoàn thành của từng sinh viên tham gia.

Trưởng nhóm: Nguyễn Phú Thành

Điểm số:

Nhận xét của giáo viên

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Tp. Hồ Chí Minh - Tháng 5 năm 2024

BẢNG KẾ HOẠCH LÀM VIỆC VÀ PHÂN CÔNG NHIỆM VỤ

STT	Nội dung công việc	Thời gian	Thành viên thực hiện	
			Họ tên	MSSV
1	- Cài đặt Apache Spark và thiết lập Spark Cluster Master-Slave - Làm bài tập Spark 30 + 31 + 32 + 34 + 43 - Bài toán xây dựng mô hình dự đoán cảm xúc: Xử lý và làm sạch dữ liệu	26/4-7/5	Nguyễn Phú Thành	21110299
2	- Cài đặt Apache Spark và thiết lập Spark Cluster Master-Slave - Làm bài tập Spark 35+ 36 + 37 + 38 + 44 - Bài toán xây dựng mô hình dự đoán cảm xúc: Spark Streaming	26/4-7/5	Cao Thị Ngọc Phụng	21110276
3	- Cài đặt Apache Spark và thiết lập Spark Cluster Master-Slave - Làm bài tập Spark 33 + 40 +42 +46 +50 - Bài toán xây dựng mô hình dự đoán cảm xúc: Đánh giá mô hình	26/4-7/5	Trần Văn Tiến	21110319
4	- Cài đặt Apache Spark và thiết lập Spark Cluster Master-Slave - Làm bài tập Spark 39 + 39 bis + 41 + 45 +49 - Bài toán xây dựng mô hình dự đoán cảm xúc: Xây dựng Pineline cho mô hình học máy		Nguyễn Văn Hào	21110175

LỜI MỞ ĐẦU

Lời đầu tiên, nhóm xin phép được gửi lời cảm ơn chân thành và sâu sắc nhất đến Khoa Công Nghệ Thông Tin – Trường Đại Học Sư Phạm Kỹ Thuật Thành Phố Hồ Chí Minh đã tạo điều kiện cho nhóm chúng em được học tập, phát triển nền tảng kiến thức cơ bản để thực hiện đề tài này.

Sự thành công của một đề tài không thể thiếu sự quan tâm, góp ý của giảng viên hướng dẫn, do đó nhóm chúng em xin gửi đến cô Lê Thị Minh Châu lời cảm ơn sâu sắc nhất. Trải qua một quá trình dài học tập và thực hiện đề tài trong thời gian qua, thầy đã tận tâm chỉ bảo nhóm chúng em từ lúc bắt đầu cũng như kết thúc đề tài này.

Tập thể các thầy cô Khoa Công Nghệ Thông Tin và đặc biệt cô Lê Thị Minh Châu đã giúp cho chúng em tiếp thu một khối lượng kiến thức và kinh nghiệm lớn về chuyên ngành và công việc trong tương lai. Đặc biệt điều này đã giúp và thôi thúc chúng em hoàn thành được đề tài. Chúng em thật sự cảm ơn và khắc ghi những kiến thức đó. Đây sẽ là hành trang vô cùng lớn của chúng em trước khi bước ra một môi trường mới.

Tuy nhiên, lượng kiến thức là vô tận và với khả năng hạn hẹp chúng em đã rất cố gắng để hoàn thành một cách tốt nhất. Tuy vậy việc xảy ra những thiếu sót là điều khó có thể tránh khỏi. Chúng em hi vọng nhận được sự góp ý tận tình của quý thầy (cô). Qua đó, chúng em có thể rút ra được bài học kinh nghiệm và hoàn thiện và cải thiện sản phẩm của mình một cách tốt nhất.

Cuối cùng một lần nữa chúng em xin gửi lời cảm ơn sâu sắc nhất đến với cô Lê Thị Minh Châu và tập thể quý thầy (cô) Khoa Công Nghệ Thông Tin – Trường Đại Học Sư Phạm Kỹ Thuật Thành Phố Hồ Chí Minh. Xin kính chúc các thầy cô có một sức khỏe thật tốt.

Chúng em xin chân thành cảm ơn!

MỤC LỤC

PHẦN MỞ ĐẦU	1
1. TỔNG QUAN VỀ ĐỀ TÀI	1
3. PHẠM VI NGHIÊN CỨU	1
PHẦN NỘI DUNG	2
CHƯƠNG 1: GIỚI THIỆU APACHE SPARK	2
1.1 LỊCH SỬ HÌNH THÀNH CỦA APACHE SPARK	2
1.2 CÁC THÀNH PHẦN CỦA SPARK	3
1.2.1 Spark Core.....	3
1.2.2 SparkSQL.....	4
1.2.3 Spark Streaming	5
1.2.4 MLlib (Machine Learning Library).....	5
1.2.5 GraphX.....	6
1.3 NỀN TẢNG XỬ LÝ DỮ LIỆU ĐƯỢC ƯA CHUỘNG BỞI CÁC CÔNG TY CÔNG NGHỆ HÀNG ĐẦU	7
1.4 TÍNH NĂNG CỦA APACHE SPARK.....	8
1.4.1 Ưu điểm	8
1.4.2 Nhược điểm.....	9
CHƯƠNG 2: CÀI ĐẶT HADOOP CLUSTER VÀ SPARK CLUSTER.....	10
2.1 CÀI ĐẶT JDK	10
2.1.1 Tải JDK 1.8	10
2.1.2 Cài đặt môi trường JDK.....	10
2.2 CÀI ĐẶT HADOOP.....	11
2.3 CẤU HÌNH CÁC FILE CỦA HADOOP.....	13
2.3.1 Cấu hình địa chỉ IP	13
2.3.2 Cấu hình core-site.xml (cấu hình trên cả master và slave).....	14
2.3.3 Cấu hình mapred-site.xml (cấu hình trên cả master và slave).....	14
2.3.4 Cấu hình hdfs-site.xml (cấu hình trên cả master và slave)	15
2.3.5 Cấu hình yarn-site.xml (cấu hình trên cả master và slave).....	17
2.3.6 Cấu hình hadoop-env.cmd (cấu hình trên cả master và slave)	18
2.3.7 Cập nhật file BIN.....	19
2.3.8 Format hệ thống	20
2.3.9 Chạy thử	20
2.4 CÀI ĐẶT SPARK VÀ SPARK CLUSTER.....	22
2.4.1 Cài đặt môi trường	22

2.4.1.1 Cài đặt Java 11	22
2.4.1.2 Cài đặt Python	22
2.4.2 Cài đặt Spark	22
2.4.2.1 Tải xuống Spark	22
2.4.2.2 Thiết lập Spark	22
2.4.3 Kiểm tra Spark	23
2.4.4 Spark Cluster	23
2.4.4.1 Cài đặt Cluster	23
2.4.4.2 Đổi tên máy chủ	25
2.4.4.3 Tắt tường lửa	26
2.4.4.4 Kiểm tra kết nối	27
2.4.4.5 Triển khai trên máy chủ (thực thi trên máy chủ)	27
2.4.4.6 Triển khai trên máy Slave (thực thi trên máy Slave)	28
2.4.4.7 Kiểm tra máy master	28
CHƯƠNG 3: BÀI TẬP APACHE SPARK.....	29
PHẦN 1	29
Bài 30	29
Bài 31	31
Bài 32	33
Bài 34	35
Bài 43 Phân tích dữ liệu trạm chia sẻ xe đạp.....	37
PHẦN 2	42
Bài 35	42
Bài 36	44
Bài 37	45
Bài 38	47
Bài 44	49
PHẦN 3	51
Bài 33	51
Bài 40	52
Bài 42	54
Bài 46	57
Bài 50	59
Câu 39	61
Câu 39 - Critical dates analysis - bis	63

<i>Câu 41- Top-k most critical sensors</i>	65
<i>Câu 45 - Profile update</i>	68
<i>Câu 49</i>	72

CHƯƠNG 4: ỨNG DỤNG SPARK ĐỂ XÂY DỰNG MÔ HÌNH DỰ ĐOÁN CẢM

XÚC	75
4.1 GIỚI THIỆU BÀI TOÁN	75
4.2 XỬ LÝ VÀ LÀM SẠCH DỮ LIỆU	77
4.2.1 <i>Xử lý dữ liệu</i>	77
4.2.2 <i>Spark Streaming</i>	86

PHẦN MỞ ĐẦU

1. Tổng quan về đề tài

Trong thời đại công nghệ phát triển vượt bậc, dữ liệu lớn (Big Data) đang trở thành một nguồn tài nguyên quan trọng, thúc đẩy nhu cầu xử lý và phân tích dữ liệu với tốc độ cao và hiệu quả. Apache Spark là một nền tảng xử lý dữ liệu lớn mã nguồn mở, được đánh giá cao nhờ hiệu suất vượt trội và khả năng mở rộng linh hoạt. Spark hỗ trợ nhiều tính năng mạnh mẽ như xử lý dữ liệu theo thời gian thực, phân tích dữ liệu lớn và tích hợp học máy (Machine Learning). Đề tài này tập trung vào việc tìm hiểu về Apache Spark và ứng dụng của nó trong xây dựng mô hình dự đoán cảm xúc. Thông qua đó, giúp người học nắm bắt được cách sử dụng Spark để xử lý, phân tích dữ liệu cảm xúc và áp dụng các kỹ thuật học máy để dự đoán trạng thái cảm xúc dựa trên dữ liệu đầu vào.

2. Mục tiêu đề tài

Tìm hiểu lý thuyết: Nắm bắt các khái niệm cơ bản về Apache Spark, bao gồm kiến trúc, cơ chế hoạt động và các thành phần chính như Spark SQL, Spark Streaming, và thư viện học máy (MLlib). **Cài đặt và cấu hình:** Xây dựng môi trường Apache Spark, bao gồm thiết lập cụm Spark Cluster theo mô hình Master-Slave. **Xây dựng mô hình dự đoán cảm xúc:** Phân tích và xử lý dữ liệu cảm xúc bằng Apache Spark, sau đó ứng dụng các thuật toán học máy trên Spark MLlib để xây dựng và đánh giá mô hình dự đoán. **Thực hành lập trình:** Viết các chương trình Spark bằng Scala hoặc Python để xử lý và phân tích dữ liệu cảm xúc, bao gồm các bài toán như tiền xử lý dữ liệu, trích xuất đặc trưng, và dự đoán cảm xúc. **Đánh giá hiệu quả:** So sánh hiệu suất và độ chính xác của mô hình dự đoán cảm xúc khi xử lý trên Spark với các nền tảng khác.

3. Phạm vi nghiên cứu

Đề tài tập trung vào các nội dung chính sau. **Nghiên cứu lý thuyết:** Tổng quan về Apache Spark và các thành phần liên quan, lý thuyết về học máy và dự đoán cảm xúc. **Cài đặt và cấu hình:** Hướng dẫn chi tiết cách cài đặt Apache Spark, thiết lập cụm Spark Cluster, và tích hợp các thành phần như Spark SQL và Spark Streaming. **Ứng dụng thực tế:** Xây dựng quy trình xử lý dữ liệu cảm xúc, từ việc thu thập và tiền xử lý dữ liệu đến áp dụng các thuật toán học máy để xây dựng mô hình dự đoán cảm xúc trên nền tảng Apache Spark. **Hạn chế:** Đề tài không tập trung vào so sánh chi tiết Spark với các nền tảng khác mà chỉ đánh giá hiệu quả của Spark trong bối cảnh cụ thể.

PHẦN NỘI DUNG

CHƯƠNG 1: GIỚI THIỆU APACHE SPARK

1.1 Lịch sử hình thành của Apache Spark

Apache Spark là một hệ thống xử lý dữ liệu lớn mã nguồn mở, được biết đến với hiệu suất cao, khả năng mở rộng và dễ sử dụng. Hành trình phát triển của Spark bắt đầu từ một dự án nghiên cứu tại Đại học California, Berkeley vào năm 2009, và trải qua nhiều giai đoạn quan trọng để trở thành công cụ phổ biến như hiện nay. Năm 2010, một nhóm 5 nhà nghiên cứu thuộc AMPLab - UC Berkeley đã cho ra đời một bài báo mang tính đột phá mang tên [“Spark: Cluster Computing with Working Sets”](#). Lúc bấy giờ, Hadoop MapReduce đang thống trị lĩnh vực lập trình song song, trở thành công cụ open-source đầu tiên để xử lý dữ liệu quy mô lớn trên các cụm máy chủ với hàng ngàn node. Nhóm AMPLab, nhận thức được tiềm năng to lớn của Cluster Computing, đã bắt tay vào nghiên cứu để phát triển một nền tảng lập trình mạnh mẽ và linh hoạt hơn MapReduce.

Trong suốt quá trình khảo sát và phân tích, họ đã xác định được hai điểm chính

1. *Tiềm năng to lớn của Cluster Computing*: Mô hình này ẩn chứa tiềm năng phát triển to lớn, cho phép xây dựng và vận hành các ứng dụng mới hoàn toàn dựa trên hệ thống và dữ liệu hiện có.
2. *Hạn chế của MapReduce*: Việc sử dụng MapReduce để xây dựng các ứng dụng lớn trở nên khó khăn và kém hiệu quả.

Nhóm AMPLab quyết định phát triển Spark, đặt mục tiêu cốt lõi là tạo ra một nền tảng lập trình dễ dàng sử dụng, có khả năng mở rộng linh hoạt để xử lý các tập dữ liệu khổng lồ. Spark được thiết kế dựa trên API lập trình hàm (Functional Programming), cho phép viết ứng dụng một cách ngắn gọn và hiệu quả. Nền tảng này còn tích hợp công cụ chia sẻ dữ liệu trong bộ nhớ hiệu quả, cho phép tái sử dụng dữ liệu qua các bước tính toán.

Spark cung cấp một hệ thống xử lý dữ liệu song song mạnh mẽ, có thể hoạt động trên hệ thống hàng ngàn máy chủ. Nền tảng này hỗ trợ đa ngôn ngữ lập trình phổ biến như Python, Java, Scala và R, cùng với các thư viện dành cho nhiều tác vụ khác nhau, từ SQL đến Streaming, Machine Learning và tính toán đồ thị song song. Năm 2013, dự án Spark được trao tặng cho Apache Software Foundation, trở thành dự án open-source thu hút sự quan tâm và đóng góp mạnh mẽ từ cộng đồng lập trình viên. Cuối cùng, vào ngày 26/5/2014, Apache chính thức phát hành Spark phiên bản 1.0, sẵn sàng cho ứng dụng trong môi trường sản xuất.

1.2 Các thành phần của Spark



1.2.1 Spark Core

Là nền tảng cho tất cả các chức năng khác của Spark. Nó cung cấp khả năng tính toán phân tán hiệu quả trên bộ nhớ RAM và cả bộ dữ liệu lưu trữ trên các hệ thống lưu trữ ngoài (như HDFS). Tất cả các chức năng được cung cấp bởi Apache Spark đều được xây dựng trên nền tảng của Spark Core. Spark Core cung cấp khả năng tính toán trong bộ nhớ (in-memory computation) để mang lại tốc độ xử lý nhanh. Nó là nền tảng của việc xử lý song song và phân tán các tập dữ liệu khổng lồ. Spark Core đóng vai trò trụ cột cho các chức năng I/O cần thiết và đóng vai trò quan trọng trong việc lập trình và giám sát cụm Spark. Spark Core bao gồm các thành phần liên quan đến việc lập lịch, phân phối và giám sát các tác vụ trên cụm, phân (phân vaii nhiệm vụ - task dispatching), và phục hồi lỗi.

Các chức năng của Spark Core bao gồm:

- Hỗ trợ nhiều mô hình lập lịch: local (chạy cục bộ trên một máy), cluster (chạy trên cụm máy tính) và YARN (dành cho các ứng dụng quy mô lớn).
- Là nơi lưu trữ API để định nghĩa RDD (Các tập dữ liệu đàn hồi phân tán).
- Bộ dữ liệu phân tán có khả năng phục hồi), Spark Context (ngữ cảnh Spark) và các primitive operations (phép toán nguyên thủy) cho RDD.

1.2.2 SparkSQL

Spark SQL được xây dựng trên nền tảng Spark Core, cung cấp một giao diện SQL để truy vấn và xử lý dữ liệu có cấu trúc và bán cấu trúc. Nó đóng vai trò như một lớp trung gian giúp truy cập và thao tác với nhiều nguồn dữ liệu khác nhau một cách dễ dàng. Spark SQL hỗ trợ đọc dữ liệu từ các định dạng phổ biến như Hive table, Parquet, JSON, và cho phép truy vấn dữ liệu bằng cả ngôn ngữ SQL và HiveQL.

Điểm mạnh của Spark SQL là cung cấp các tính năng phân tích mạnh mẽ, tương tác, hoạt động trên cả dữ liệu thời gian thực (streaming) và dữ liệu lịch sử. Spark SQL có thể được coi như một thành phần mở rộng của Spark, tích hợp xử lý dữ liệu theo quan hệ với các API lập trình của Spark.

Các chức năng chính của Spark SQL:

- Xử lý dữ liệu có cấu trúc trong Spark. Giới thiệu khái niệm SchemaRDD: cung cấp một biểu diễn trừu tượng cho dữ liệu có cấu trúc, bao gồm cả schema (sơ đồ dữ liệu).
- Đọc dữ liệu từ nhiều nguồn khác, tích hợp với các hệ thống lưu trữ dữ liệu khác nhau: Hive, HBase, Cassandra, CSV, JSON, Parquet, ORC và hơn thế nữa.
- Cho phép lập trình viên kết hợp các truy vấn SQL với thao tác dữ liệu theo chương trình được hỗ trợ bởi RDD trong Python, Scala và Java.
- Cho phép thực thi các truy vấn SQL phức tạp: JOIN, aggregation, filtering, windowing và subqueries.

1.2.3 Spark Streaming

Spark Streaming cung cấp khả năng xử lý luồng dữ liệu trực tiếp với tính mở rộng, thông lượng cao và chịu lỗi. Spark có thể truy cập dữ liệu từ các nguồn như Flume, socket TCP. Nó có thể xử lý các thuật toán khác nhau trên dữ liệu được nhận và kết quả có thể được lưu trữ ở nhiều định dạng như hệ thống tập tin, cơ sở dữ liệu hoặc hiển thị trên bảng điều khiển trực tiếp.

Spark Streaming sử dụng kỹ thuật Micro-batching để xử lý luồng dữ liệu thời gian thực. Micro-batching phân chia luồng dữ liệu thành các nhóm nhỏ (micro-batch) để xử lý hiệu quả hơn. Mỗi micro-batch được xử lý giống như một batch dữ liệu thông thường trong Spark Core, giúp tận dụng tối đa sức mạnh tính toán của Spark.

Chức năng chính của Spark Streaming:

- Cho phép xử lý luồng dữ liệu trực tiếp, ví dụ như các file log được tạo bởi các dịch vụ web.
- Cung cấp API xử lý luồng dữ liệu tương tự như API RDD của Spark Core, giúp dễ dàng học hỏi và sử dụng.
- Cung cấp tính năng xử lý dữ liệu theo thời gian thực (real-time) trên Spark.
- Lấy dữ liệu theo các mini-batch nhỏ và áp dụng các phép biến đổi RDD trên mỗi mini-batch.
- Hỗ trợ nhiều nguồn dữ liệu thời gian thực: Kafka, Flume, Twitter và hơn thế nữa.
- Cho phép thực hiện các phép tính phân tích phức tạp: cập nhật trạng thái, phân tích dòng chảy, phát hiện bất thường và hơn thế nữa

1.2.4 MLlib (Machine Learning Library)

Thư viện Học máy (Machine Learning) mở rộng trên nền Spark – MLlib, một thư viện học máy mở rộng tích hợp trên Spark, cung cấp nhiều thuật toán học máy khác nhau. Mục tiêu chính của việc xây dựng MLlib là đơn giản hóa việc triển khai các thuật toán học máy. MLlib bao gồm các công cụ cần thiết cho học máy cùng với việc triển khai nhiều thuật toán phổ biến. Ví dụ, các thuật toán thường dùng trong MLlib bao gồm: cụm (clustering), dự báo (regression), phân loại (classification) và đề xuất (collaborative filtering).

Những điểm chính cần lưu ý:

- MLlib là một thư viện mở rộng, tận dụng khả năng tính toán phân tán của Spark để tăng tốc độ đào tạo và dự đoán mô hình.
- MLlib cung cấp nhiều thuật toán học máy phổ biến, giúp người dùng dễ dàng lựa chọn và triển khai thuật toán phù hợp cho bài toán của mình.
- Mục tiêu của MLlib là đơn giản hóa việc sử dụng học máy, giúp người dùng có thể tập trung vào việc xây dựng các mô hình hiệu quả.
- Có thể tích hợp với các thư viện học máy phổ biến khác: TensorFlow, scikit-learn và hơn thế nữa.

1.2.5 GraphX

Là một framework xử lý đồ thị phân tán được xây dựng trên Spark Core. Cung cấp một API để thực hiện các phép tính đồ thị

- Phân tích mạng lưới: Lưu trữ, truy vấn và phân tích dữ liệu dạng đồ thị, bao gồm các đỉnh (vertex) và cạnh (edge) kết nối chúng.
- Hỗ trợ nhiều loại đồ thị: directed graphs (đồ thị có hướng), undirected graphs (đồ thị vô hướng), weighted graphs (đồ thị có trọng số) và hơn thế nữa. Cho phép thực hiện các thuật toán đồ thị phức tạp: PageRank, community detection (phát hiện cộng đồng) và social network analysis (phân tích mạng xã hội)
- Tối ưu hóa hiệu suất: Tối ưu cách lưu trữ và biểu diễn đồ thị, hỗ trợ xử lý dữ liệu đồ thị lớn hiệu quả.
- Hỗ trợ các phép toán: Cung cấp các phép toán cơ bản để thao tác với dữ liệu đồ thị như lấy tập con, nối đỉnh, tổng hợp tin nhắn.
- API Pregel: Cung cấp phiên bản tối ưu hóa của API Pregel để xử lý song song các thuật toán trên đồ thị.

1.3 Nền tảng xử lý dữ liệu được ưa chuộng bởi các công ty công nghệ hàng đầu

Apache Spark đang ngày càng khẳng định vị thế là nền tảng xử lý dữ liệu hàng đầu, được tin dùng bởi nhiều công ty công nghệ lớn trên thế giới. Nhờ khả năng xử lý dữ liệu lớn hiệu quả, linh hoạt và dễ sử dụng, Spark mang đến giải pháp tối ưu cho các bài toán phức tạp trong nhiều lĩnh vực.

Alibaba: Alibaba một trong những nền tảng thương mại điện tử lớn nhất thế giới, sử dụng Apache Spark để phân tích dữ liệu khổng lồ với hàng trăm petabyte dữ liệu trên nền tảng thương mại điện tử của mình, cung cấp thông tin chi tiết về hành vi khách hàng, xu hướng thị trường và hiệu quả hoạt động. Spark giúp Alibaba trích xuất tính năng từ dữ liệu hình ảnh một cách nhanh chóng và chính xác, hỗ trợ cho các ứng dụng như nhận dạng sản phẩm, phân loại ảnh và đề xuất sản phẩm phù hợp. Ngoài ra, Spark được sử dụng để xử lý các đồ thị phức tạp mô tả mối quan hệ giữa người dùng, sản phẩm và nhà bán hàng, giúp Alibaba đưa ra các quyết định kinh doanh sáng suốt hơn.

Ebay: eBay sử dụng Apache Spark để cung cấp các ưu đãi được nhắm mục tiêu đến từng khách hàng, giúp họ dễ dàng tìm kiếm sản phẩm phù hợp và gia tăng tỷ lệ chuyển đổi. Xử lý lượng lớn dữ liệu một cách nhanh chóng và hiệu quả, góp phần nâng cao hiệu suất hoạt động của toàn hệ thống. Apache Spark được tận dụng tại eBay thông qua Hadoop YARN.

Uber: Spark giúp Uber dự đoán nhu cầu đi xe một cách chính xác, hỗ trợ tối ưu hóa việc phân bổ tài xế và đáp ứng nhu cầu của khách hàng hiệu quả hơn. Spark được sử dụng để phân tích dữ liệu di chuyển của người dùng, giúp Uber hiểu rõ hơn về hành vi di chuyển và đưa ra các giải pháp cải thiện dịch vụ.

Netflix: Sử dụng Spark để phân tích sở thích của người dùng và đề xuất nội dung phù hợp với từng cá nhân, nâng cao trải nghiệm người dùng và gia tăng tỷ lệ giữ chân khách hàng. Phân tích hiệu suất của các nội dung trên Netflix, giúp công ty đánh giá hiệu quả các chiến dịch marketing và điều chỉnh nội dung phù hợp với thị hiếu người xem.

Ngoài ra, Spark còn được ứng dụng rộng rãi trong nhiều lĩnh vực khác như:

- Phân tích khoa học: Spark được sử dụng bởi các tổ chức như NASA, CERN và Broad Institute of MIT và Harvard để phân tích dữ liệu khoa học khổng lồ, hỗ trợ nghiên cứu khoa học và phát triển công nghệ.
- Xử lý ngôn ngữ tự nhiên: Spark được sử dụng để xử lý và phân tích dữ liệu ngôn ngữ tự nhiên, hỗ trợ các ứng dụng như dịch máy, tóm tắt văn bản và chatbot.
- Internet vạn vật (IoT): Spark được sử dụng để xử lý và phân tích dữ liệu từ các thiết bị IoT, giúp theo dõi và quản lý các thiết bị hiệu quả hơn.

Với những ưu điểm vượt trội, Apache Spark đang trở thành nền tảng xử lý dữ liệu không thể thiếu cho các doanh nghiệp và tổ chức trong thời đại dữ liệu bùng nổ. Khả năng xử lý dữ liệu lớn hiệu quả, linh hoạt và dễ sử dụng giúp Spark đáp ứng mọi nhu cầu xử lý dữ liệu phức tạp, đồng thời mang lại lợi thế cạnh tranh cho các doanh nghiệp trong thị trường ngày càng cạnh tranh.

1.4 Tính năng của Apache Spark



1.4.1 Ưu điểm

Apache Spark là một nền tảng xử lý dữ liệu mã nguồn mở, nổi tiếng với khả năng phân tích dữ liệu mạnh mẽ, tốc độ xử lý nhanh chóng và hỗ trợ đa ngôn ngữ. Spark được sử dụng rộng rãi trong nhiều lĩnh vực như khoa học dữ liệu, phân tích dữ liệu lớn, học máy, xử lý ngôn ngữ tự nhiên, Internet vạn vật (IoT), ... Một số điểm nổi bật của Apache Spark như:

Advanced Analytics: Spark không chỉ giới hạn trong các phép toán "Map" và "Reduce" truyền thống, mà còn cung cấp các tính năng phân tích dữ liệu tiên tiến như

hỗ trợ Spark truy vấn SQL, Streaming data, Machine learning (ML) và các thuật toán xử lý đồ thị đóng vai trò như một bộ công cụ phân tích dữ liệu cực kì mạnh mẽ.

Speed: Spark giúp chạy một ứng dụng với tốc độ xử lý vượt trội. So với Hadoop cluster, Spark có thể chạy ứng dụng nhanh hơn tới 100 lần khi xử lý dữ liệu trong bộ nhớ và 10 lần khi xử lý dữ liệu trên đĩa.. Điều này do Spark tối ưu hóa việc sử dụng bộ nhớ, hạn chế tối đa việc đọc/ghi dữ liệu vào ổ đĩa, giúp tăng tốc độ xử lý và giảm thiểu thời gian chờ.

Supports multiple languages: Spark cung cấp API tích hợp sẵn cho các ngôn ngữ lập trình phổ biến như Java, Scala, Python và R. Điều này cho phép người dùng lựa chọn ngôn ngữ phù hợp với sở thích và kỹ năng, có thể code Spark applications với nhiều lựa chọn về ngôn ngữ lập trình. Bên cạnh đó Spark còn cung cấp nhiều toán tử cấp cao (high-level operators) cho việc truy vấn dữ liệu, giúp đơn giản hóa việc viết mã và tăng năng suất lập trình.

1.4.2 Nhược điểm

Mặc dù Apache Spark là một nền tảng xử lý dữ liệu mạnh mẽ và linh hoạt, nhưng nó cũng có một số hạn chế nhất định cần được cân nhắc trước khi sử dụng:

Phụ thuộc vào hệ thống tệp: Spark không có hệ thống tệp riêng mà phụ thuộc vào các hệ thống tệp khác như HDFS (Hadoop Distributed File System) hoặc các hệ thống lưu trữ đám mây như S3, Google Cloud Storage,... Điều này có thể dẫn đến một số hạn chế về khả năng tương thích và hiệu suất. Spark yêu cầu hệ thống tệp phải hỗ trợ truy cập dữ liệu song song và hiệu quả để đảm bảo hiệu suất tối ưu.

Chi phí cao: Apache Spark được thiết kế để xử lý dữ liệu trong bộ nhớ, do đó yêu cầu lượng RAM lớn để hoạt động hiệu quả. Điều này có thể dẫn đến chi phí phần cứng cao hơn, đặc biệt khi xử lý các tập dữ liệu lớn

Khả năng xử lý luồng dữ liệu: Spark Streaming được thiết kế để xử lý luồng dữ liệu với độ trễ thấp, nhưng nó không hoàn toàn là thời gian thực. Luồng dữ liệu được chia thành các micro-batch nhỏ để xử lý, dẫn đến một số độ trễ nhất định. Để đạt được hiệu suất xử lý luồng dữ liệu cao, Spark Streaming cần được cấu hình và tối ưu hóa cẩn thận, đòi hỏi chuyên môn kỹ thuật cao.

Tối ưu hóa thủ công: Việc tối ưu hóa, tinh chỉnh để phù hợp với các bộ dữ liệu cụ thể cần có kinh nghiệm và kiến thức để điều chỉnh hiệu suất Spark phù hợp với từng trường hợp sử dụng.

CHƯƠNG 2: CÀI ĐẶT HADOOP CLUSTER VÀ SPARK CLUSTER

2.1 Cài đặt JDK

2.1.1 Tải JDK 1.8

Hadoop sử dụng JDK 1.8

Ta vào link sau: [Java Archive Downloads - Java SE 8 \(oracle.com\)](https://www.oracle.com/technetwork/java/javase-downloads-1320263.html)

Windows x64	211.58 MB	 jdk-8u202-windows-x64.exe
-------------	-----------	---

Sau khi tải về ta có file

 jdk-8u202-windows-x64	4/5/2024 8:02 AM	Application	216,653 KB
---	------------------	-------------	------------

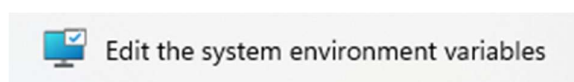
Sau đó chạy để cài đặt các gói về

 jdk-1.8	4/5/2024 8:12 AM	File folder
 jdk-22	3/27/2024 4:01 AM	File folder
 jre-1.8	4/5/2024 8:12 AM	File folder

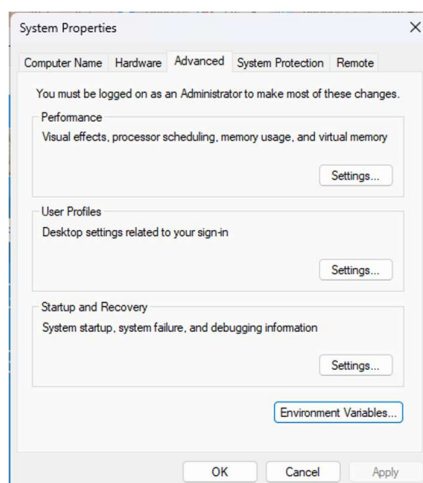
 Java 11	8/24/2023 2:15 PM	File folder
---	-------------------	-------------

2.1.2 Cài đặt môi trường JDK

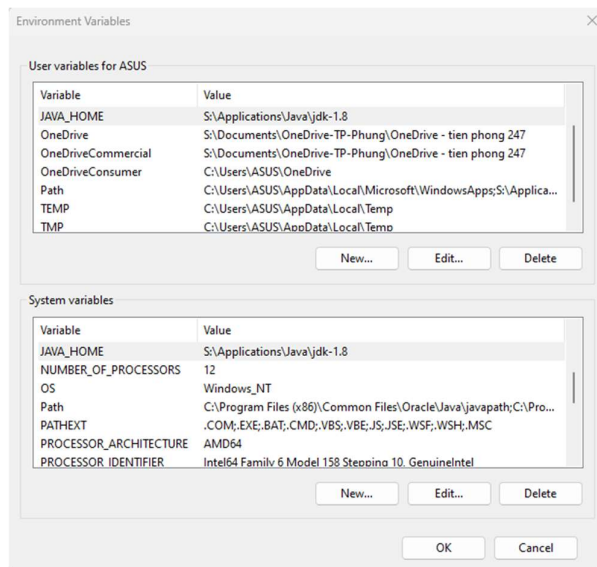
Có thể tìm ở thanh công cụ “Edit the system environment variables”



Chọn “Environment variables”



Dẫn đường dẫn phù hợp cho JAVA_HOME



Và Path



Kiểm tra xem hệ thống đã nhận jdk chưa

```
C:\Users\ASUS>java -version
java version "1.8.0_202"
Java(TM) SE Runtime Environment (build 1.8.0_202-b08)
Java HotSpot(TM) 64-Bit Server VM (build 25.202-b08, mixed mode)
```

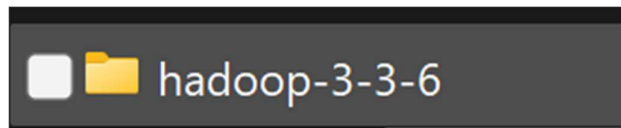
2.2 Cài đặt Hadoop

Đầu tiên chúng ta tải Hadoop

Hadoop sử dụng Hadoop-3.3.6

Ta vào link sau: [Apache Hadoop](#)

Sau khi tải về và giải nén

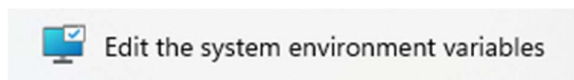


Cài đặt biến môi trường Hadoop

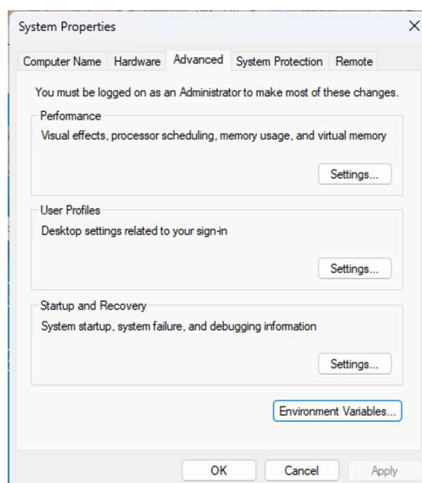
Máy master và máy slave đều phải cài đặt chung 1 đường dẫn

Ví dụ ở dưới là “C:\hadoop-3-3-6” thì cả máy master và slave đều lưu hadoop ở địa chỉ “C:\hadoop-3-3-6”

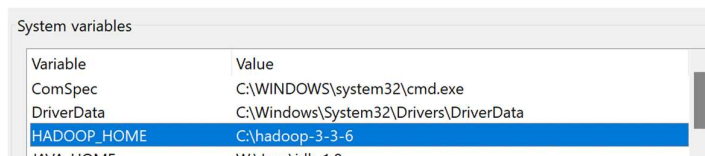
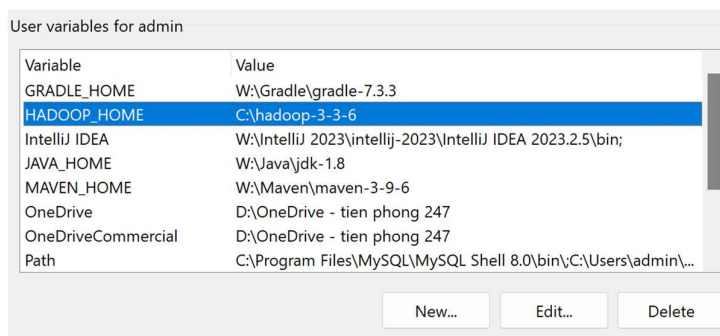
Có thể tìm ở thanh công cụ “Edit the system environment variables”



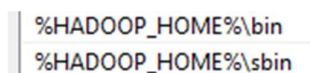
Chọn “Environment variables”



Dẫn đường dẫn phù hợp cho HADOOP



Và Path

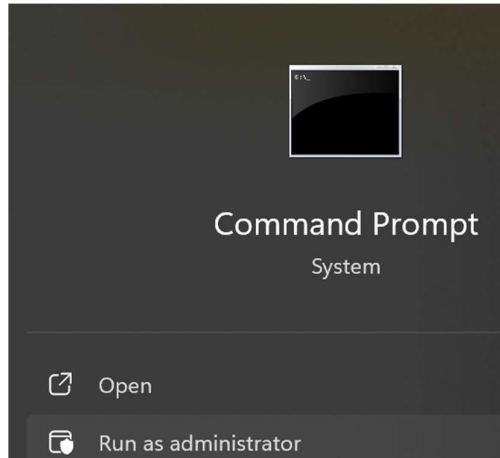


2.3 Cấu hình các file của Hadoop

2.3.1 Cấu hình địa chỉ IP

Mở command dưới quyền admin

Notepad C:\Windows\System32\Drivers\etc\hosts



Điền hostname

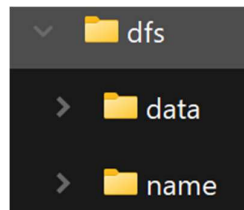
192.168.171.148 ngocphung-master

192.168.171.233 phuthanh-slave

192.168.171.201 trantien-slave

Tại ổ C tạo folder dfs

Trong folder dfs có 2 folder con là name và data



Tạo thêm folder tmp vào ổ C

2.3.2 Cấu hình core-site.xml (cấu hình trên cả master và slave)

Vào địa chỉ "C:\hadoop-3-3-6\etc\hadoop\core-site.xml"

```
<configuration>
  <property>
    <name>fs.default.name</name>
    <value>hdfs://ngocphung-master:9000</value>
  </property>
  <property>
    <name>dfs.permissions</name>
    <value>>false</value>
  </property>
  <property>
    <name>hadoop.tmp.dir</name>
    <value>/C:/tmp</value>
    <description>A base for other temporary directories.</description>
  </property>
</configuration>
```

```
<configuration>
  <property>
    <name>fs.default.name</name>
    <value>hdfs://ngocphung-master:9000</value>
  </property>
  <property>
    <name>dfs.permissions</name>
    <value>>false</value>
  </property>
  <property>
    <name>hadoop.tmp.dir</name>
    <value>/C:/tmp</value>
    <description>A base for other temporary directories.</description>
  </property>
</configuration>
```

2.3.3 Cấu hình mapred-site.xml (cấu hình trên cả master và slave)

Vào địa chỉ "C:\hadoop-3-3-6\etc\hadoop\mapred-site.xml"

```
<configuration>
  <property>
    <name>mapred.job.tracker</name>
    <value>ngocphung-master:9001</value>
  </property>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

```

<configuration>
  <property>
    <name>mapred.job.tracker</name>
    <value>ngocphung-master:9001</value>
  </property>

  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>

```

2.3.4 Cấu hình hdfs-site.xml (cấu hình trên cả master và slave)

Vào địa chỉ "C:\hadoop-3-3-6\etc\hadoop\hdfs-site.xml"

```

<configuration>
  <property>
    <name>dfs.name.dir</name>
    <value>/C:/dfs/name</value>
  </property>
  <property>
    <name>dfs.data.dir</name>
    <value>/C:/dfs/data</value>
  </property>
  <property>
    <name>dfs.replication</name>
    <value>4</value>
  </property>
  <property>
    <name>dfs.permissions</name>
    <value>>false</value>
  </property>
  <property>
    <name>dfs.datanode.use.datanode.hostname</name>
    <value>>false</value>
  </property>
  <property>
    <name>dfs.namenode.datanode.registration.ip-hostname-check</name>
    <value>>false</value>
  </property>
  <property>
    <name>dfs.namenode.http-address</name>
    <value>ngocphung-master:50070</value>
    <description>Your NameNode hostname for httpaccess.</description>
  </property>
  <property>
    <name>dfs.namenode.secondary.http-address</name>
    <value>ngocphung-master:50090</value>
    <description>Your Secondary NameNode hostname for httpaccess.</description>
  </property>
</configuration>

```

```

<configuration>
  <property>
    <name>dfs.name.dir</name>
    <value>/C:/dfs/name</value>
  </property>

  <property>
    <name>dfs.data.dir</name>
    <value>/C:/dfs/data</value>
  </property>

  <property>
    <name>dfs.replication</name>
    <value>4</value>
  </property>

  <property>
    <name>dfs.permissions</name>
    <value>>false</value>
  </property>

  <property>
    <name>dfs.datanode.use.datanode.hostname</name>
    <value>>false</value>
  </property>

  <property>
    <name>dfs.namenode.datanode.registration.ip-hostname-check</name>
    <value>>false</value>
  </property>

  <property>
    <name>dfs.namenode.http-address</name>
    <value>ngocphung-master:50070</value>
    <description>Your NameNode hostname for httpaccess.</description>
  </property>

  <property>
    <name>dfs.namenode.secondary.http-address</name>
    <value>ngocphung-master:50090</value>
    <description>Your Secondary NameNode hostname for httpaccess.</description>
  </property>
</configuration>

```

2.3.5 Cấu hình yarn-site.xml (cấu hình trên cả master và slave)

Vào trong địa chỉ "C:\hadoop-3-3-6\etc\hadoop\yarn-site.xml"

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
    <description>Long running service which executes on Node Manager(s) and
provides MapReduce Sort and Shuffle functionality.</description>
  </property>
  <property>
    <name>yarn.log-aggregation-enable</name>
    <value>true</value>
    <description>Enable log aggregation so application logs are moved onto hdfs and
are viewable via web ui after the application completed. The default location on hdfs is '/log' and
can be changed via yarn.nodemanager.remote-app-log-dir property</description>
  </property>
  <property>
    <name>yarn.resourcemanager.scheduler.address</name>
    <value>ngocphung-master:8030</value>
  </property>

  <property>
    <name>yarn.resourcemanager.resource-tracker.address</name>
    <value>ngocphung-master:8031</value>
  </property>
  <property>
    <name>yarn.resourcemanager.address</name>
    <value>ngocphung-master:8032</value>
  </property>
  <property>
    <name>yarn.resourcemanager.admin.address</name>
    <value>ngocphung-master:8033</value>
  </property>
  <property>
    <name>yarn.resourcemanager.webapp.address</name>
    <value>ngocphung-master:8088</value>
  </property>
</configuration>
```



```

<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
    <description>Long running service which executes on Node Manager(s) and provides MapReduce
Sort and Shuffle functionality.</description>
  </property>

  <property>
    <name>yarn.log-aggregation-enable</name>
    <value>true</value>
    <description>Enable log aggregation so application logs are moved onto hdfs and are viewable
via web ui after the application completed. The default location on hdfs is '/log' and can be changed via
yarn.nodemanager.remote-app-log-dir property</description>
  </property>

  <property>
    <name>yarn.resourcemanager.scheduler.address</name>
    <value>ngocphung-master:8030</value>
  </property>

  <property>
    <name>yarn.resourcemanager.resource-tracker.address</name>
    <value>ngocphung-master:8031</value>
  </property>

  <property>
    <name>yarn.resourcemanager.address</name>
    <value>ngocphung-master:8032</value>
  </property>

  <property>
    <name>yarn.resourcemanager.admin.address</name>
    <value>ngocphung-master:8033</value>
  </property>

  <property>
    <name>yarn.resourcemanager.webapp.address</name>
    <value>ngocphung-master:8088</value>
  </property>
</configuration>

```

2.3.6 Cấu hình hadoop-env.cmd (cấu hình trên cả master và slave)

Vào địa chỉ "C:\hadoop-3-3-6\etc\hadoop\hadoop-env.cmd"

Set up địa chỉ java giống vào java_home

```

@rem The java implementation to use. Required.
set JAVA_HOME=W:\Java\jdk-1.8

```

Dán lệnh này ở cuối file

```

set HADOOP_IDENT_STRING=%USERNAME%
set HADOOP_PREFIX=%HADOOP_HOME%
set HADOOP_CONF_DIR=%HADOOP_PREFIX%\etc\hadoop
set YARN_CONF_DIR=%HADOOP_CONF_DIR%
set PATH=%PATH%;%HADOOP_PREFIX%\bin

```

```

@rem A string representing this instance of hadoop. %USERNAME% by default.
set HADOOP_IDENT_STRING=%USERNAME%
set HADOOP_PREFIX=%HADOOP_HOME%
set HADOOP_CONF_DIR=%HADOOP_PREFIX%\etc\hadoop
set YARN_CONF_DIR=%HADOOP_CONF_DIR%
set PATH=%PATH%;%HADOOP_PREFIX%\bin

```

Edit file wokers (Làm trên máy master)

Vào địa chỉ "C:\hadoop-3-3-6\etc\hadoop\workers"

Trên máy master chỉ ghi tên máy slave

Edit file wokers (Làm trên máy slave)

Vào địa chỉ "C:\hadoop-3-3-6\etc\hadoop\workers"

Trên máy slave chỉ ghi tên máy của mình

2.3.7 Cập nhật file BIN

Tải từ link này: <https://github.com/s911415/apache-hadoop-3.1.0-winutils>

Tải về và extract ra

File name	Date modified	Type	Size
winutils-master	4/5/2024 9:33 AM	File folder	
winutils-master	4/5/2024 7:31 AM	Compressed (zip...	23,920 KB

Tìm folder gần với phiên bản của phiên bản mình dùng nhất

Em dùng hadoop-3.3.6 nên em sẽ dùng folder hadoop-3.3.5

Big Data > Week 10 > winutils-master > winutils-master >		
Sort	View	...
Name	Date modified	Type
hadoop-3.3.5	4/5/2024 9:34 AM	File folder

Copy file bin và đè lên file bin của "S:\Applications\Hadoop\hadoop-3-3-6"

Week 10 > winutils-master > winutils-master > hadoop-3.3.5 >			
Sort	View	...	
Name	Date modified	Type	Size
bin	4/5/2024 9:34 AM	File folder	

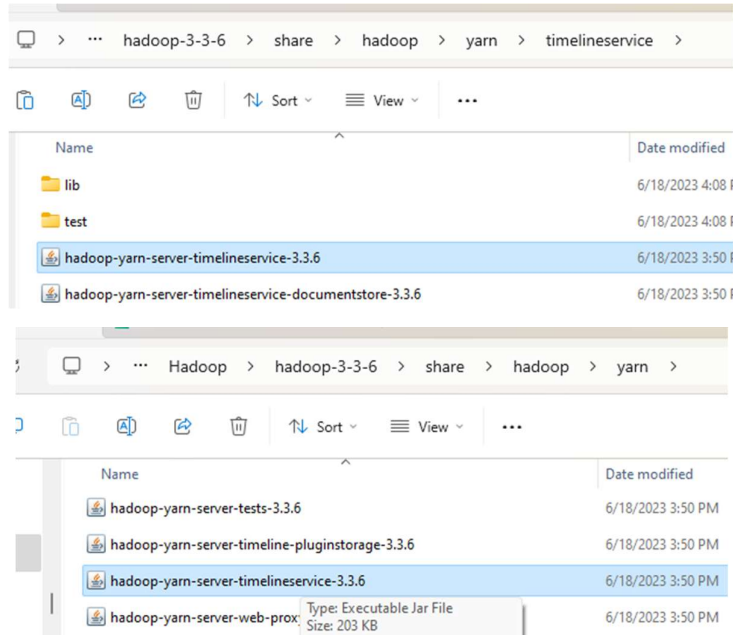
2.3.8 Format hệ thống

Hdfs namenode -format

```
S:\Applications\Hadoop\hadoop-3-3-6\bin>hdfs namenode -format
```

Sau đó ta Copy file hadoop-yarn-server-timelineservice-3.3.6

Và dán ra “S:\Applications\Hadoop\hadoop-3-3-6\share\hadoop\yarn”



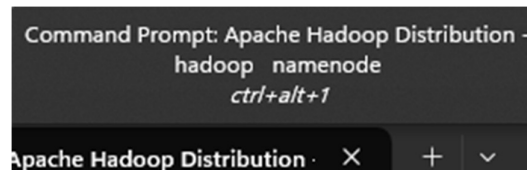
2.3.9 Chạy thử

Start-all

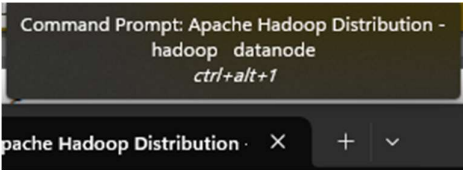
```
S:\Applications\Hadoop\hadoop-3-3-6\bin>start-all
This script is Deprecated. Instead use start-dfs.cmd and start-yarn.cmd
starting yarn daemons

S:\Applications\Hadoop\hadoop-3-3-6\bin>jps
2080 ResourceManager
3012 DataNode
13996 NameNode
17452 Jps
6076 NodeManager
```

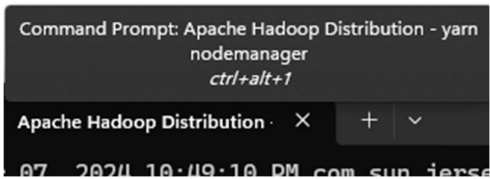
Namenode



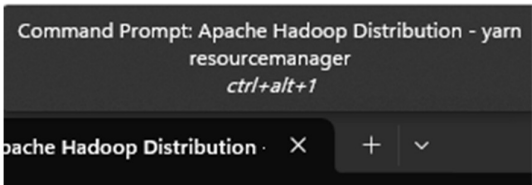
Datanode



Nodemanager



Resourcemanager



Kết quả, tạo Hadoop cluster thành công, bốn máy có trong hệ thống gồm 1 máy master và 3 máy slave



Nodes of the cluster

Logged in as: dr:who

Cluster

About

Nodes

Node Labels

Applications

NEW

NEW SAVING

SUBMITTED

ACCEPTED

RUNNING

FINISHED

FAILED

KILLED

Scheduler

Tools

Cluster Metrics

Apps Submitted: 0

Apps Pending: 0

Apps Running: 0

Apps Completed: 0

Containers Running: 0

Used Resources: <memory:0 B, vCores:0>

Total Resources: <memory:32 GB, vCores:32>

Reserved Resources: <memory:0 B, vCores:0>

Physical Mem Used %: 83

Physical VCoers Used %: 0

Cluster Nodes Metrics

Active Nodes: 4

Decommissioning Nodes: 0

Decommissioned Nodes: 0

Lost Nodes: 0

Unhealthy Nodes: 0

Rebooted Nodes: 0

Shutdown Nodes: 0

Scheduler Metrics

Scheduler Type: Capacity Scheduler

Scheduling Resource Type: [memory-mb (unit=M), vcores]

Minimum Allocation: <memory:1024, vCores:1>

Maximum Allocation: <memory:8192, vCores:4>

Maximum Cluster Application Priority: 0

Scheduler Busy %: 0

Show: 20 entries

Search:

Node Labels	Rack	Node State	Node Address	Node HTTP Address	Last health-update	Health-report	Containers	Allocation Tags	Mem Used	Mem Avail	Phys Mem Used %	VCoers Used	VCoers Avail	Phys VCoers Used %	Version
/default-rack		RUNNING	ngocphung-master:5762	ngocphung-master:8042	Thu May 16 17:45:16 +0700 2024		0		0 B	8 GB	91	0	8	3	3.3.6
/default-rack		RUNNING	192.168.56.1:5793	192.168.56.1:8042	Thu May 16 17:45:11 +0700 2024		0		0 B	8 GB	86	0	8	3	3.3.6
/default-rack		RUNNING	phuthanh-slave:55126	phuthanh-slave:8042	Thu May 16 17:43:26 +0700 2024		0		0 B	8 GB	78	0	8	1	3.3.6
/default-rack		RUNNING	phuthanh-slave:55414	phuthanh-slave:8042	Thu May 16 17:44:13 +0700 2024		0		0 B	8 GB	78	0	8	0	3.3.6

Showing 1 to 4 of 4 entries

First Previous 1 Next Last

2.4 Cài đặt Spark và Spark Cluster

2.4.1 Cài đặt môi trường

2.4.1.1 Cài đặt Java 11

Spark sử dụng JDK 11, cài đặt gói Java 11 và đặt biến môi trường

Mở công cụ "chỉnh sửa biến môi trường"



2.4.1.2 Cài đặt Python

Tải xuống python từ <https://www.python.org/downloads/> trang web và cài đặt chúng. Sau đó mở công cụ " chỉnh sửa biến môi trường " và khai báo biến môi trường



2.4.2 Cài đặt Spark

2.4.2.1 Tải xuống Spark

Tải spark tại địa chỉ sau <https://www.apache.org/dyn/closer.lua/spark/spark-3.5.3/spark-3.5.3-bin-hadoop3.tgz>

Sau khi tải về chúng ta giải nén thư mục



2.4.2.2 Thiết lập Spark

Tiếp theo chúng ta cần mở công cụ "chỉnh sửa biến môi trường" và khai báo biến môi trường cho spark



Cấu hình spark-defaults.conf (cho Master và Slave)

```
spark.master            spark://192.168.1.131:7077
spark.eventLog.enabled  true
spark.eventLog.dir       file:///c:/tmp/spark-events
spark.history.fs.logDirectory file:///c:/tmp/spark-events
spark.serializer        org.apache.spark.serializer.KryoSerializer
```

Cấu hình spark-env.cmd (cho Master)

```
set JAVA_HOME=W:\Java\jdk-1.8
set SPARK_HOME=C:\Spark\spark-3.5.3-bin-hadoop3
set HADOOP_HOME=C:\Hadoop\hadoop-3.3.6
set SPARK_MASTER_HOST=192.168.1.131
set SPARK_WORKER_CORES=8
set SPARK_LOCAL_IP=192.168.1.131
set
PATH=%JAVA_HOME%\bin;%SCALA_HOME%\bin;%SPARK_HOME%\bin;%HADOOP_HOME%\bin;%PATH%
set PYSPARK_PYTHON=C:\Python\python.exe
set PYSPARK_PYTHON=C:\Python\python.exe
set PYSPARK_DRIVER_PYTHON=C:\Python\python.exe
```

Cấu hình spark-env.cmd (cho Slave)

```
set JAVA_HOME=C:\Java-1.8\jdk-1.8
set SPARK_HOME=C:\Spark\spark-3.5.3-bin-hadoop3
set HADOOP_HOME=C:\Hadoop\hadoop-3.3.6
set SPARK_MASTER_HOST=192.168.1.131 set SPARK_WORKER_CORES=8
set SPARK_LOCAL_IP=192.168.1.120
set
PATH=%JAVA_HOME%\bin;%SCALA_HOME%\bin;%SPARK_HOME%\bin;%HADOOP_HOME%\bin;%PATH%
set PYSPARK_PYTHON=C:\Users\LG\AppData\Local\Programs\Python\Python311\python.exe
set PYSPARK_PYTHON=C:\Users\LG\AppData\Local\Programs\Python\Python311\python.exe
set
PYSPARK_DRIVER_PYTHON=C:\Users\LG\AppData\Local\Programs\Python\Python311\python.exe
```

Worker (cho Master)

```
192.168.1.131 ngocphung-master
192.168.1.120 tientran-slave
192.168.1.23 phuthanh-slave
192.168.1.172 vanhao-slave
```

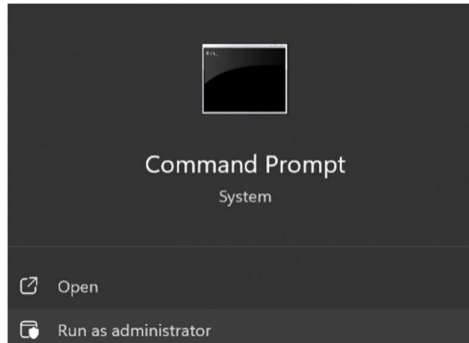
Worker (cho Slave)

192.168.1.120 tientran-slave

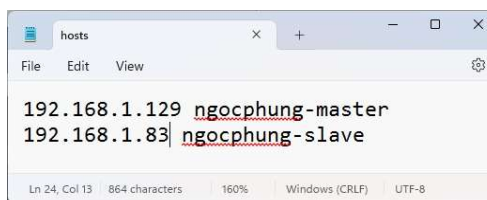
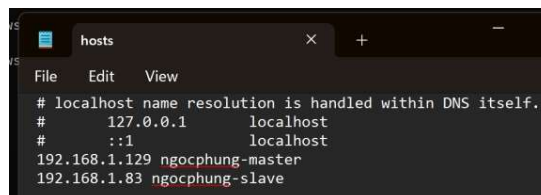
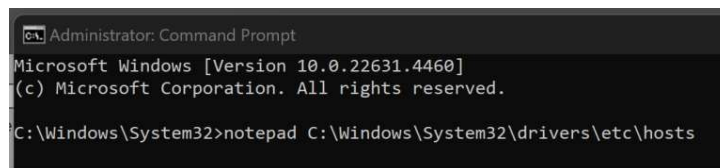
2.4.4.2 Đổi tên máy chủ

Đổi tên file máy chủ và máy phụ để dễ quản lý (thực hiện trên master và slave)

Chạy dấu nhắc lệnh với tư cách quản trị viên



Để dễ dàng quản lý địa chỉ máy nô lệ theo tên thay vì số. Gõ lệnh “ notepad C:\Windows\System32\drivers\etc\hosts ” để đặt tên cho từng ip

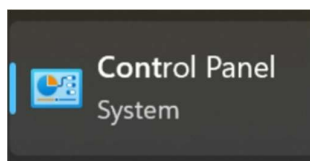


Điều này sẽ giúp chúng ta quản lý máy dễ dàng khi ping hay kiểm tra. Thay vì ping trực tiếp địa chỉ máy, chúng ta chỉ cần ping tên máy.

2.4.4.3 Tắt tường lửa

Tắt tường lửa của máy chủ và máy phụ (master và Workers)

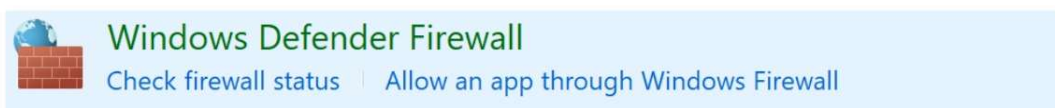
Mở “Bảng điều khiển”



Nhấp vào “Hệ thống và bảo mật”



Nhấp vào Tường lửa bảo vệ Windows Windows



Nhấp vào “Bật hoặc tắt Tường lửa của Bộ bảo vệ Windows”



Tắt đi bấm ok



2.4.4.4 Kiểm tra kết nối

Dùng máy ngocphung-master để ping máy ngocphung-slave

```
C:\Users\admin>ping ngocphung-slave

Pinging ngocphung-slave [192.168.1.83] with 32 bytes of data:
Reply from 192.168.1.83: bytes=32 time=12ms TTL=128
Reply from 192.168.1.83: bytes=32 time=12ms TTL=128
Reply from 192.168.1.83: bytes=32 time=4ms TTL=128
Reply from 192.168.1.83: bytes=32 time=6ms TTL=128

Ping statistics for 192.168.1.83:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 4ms, Maximum = 12ms, Average = 8ms
```

Dùng máy ngocphung-slave để ping máy ngocphung-master

```
C:\Users\phung>ping ngocphung-master

Pinging ngocphung-master [192.168.1.129] with 32 bytes of data:
Reply from 192.168.1.129: bytes=32 time=89ms TTL=128
Reply from 192.168.1.129: bytes=32 time=40ms TTL=128
Reply from 192.168.1.129: bytes=32 time=112ms TTL=128
Reply from 192.168.1.129: bytes=32 time=16ms TTL=128

Ping statistics for 192.168.1.129:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 16ms, Maximum = 112ms, Average = 64ms
```

2.4.4.5 Triển khai trên máy chủ (thực thi trên máy chủ)

Điều hướng đến Spark_Home bằng lệnh sau “cd %SPARK_HOME%\bin”

Khởi động máy chủ bằng lệnh sau “spark-class2.cmd

org.apache.spark.deploy.master.Master”

```
C:\Users\admin>cd %SPARK_HOME%\bin
C:\Spark\spark-3.5.1-bin-hadoop3\bin>spark-class2.cmd org.apache.spark.deploy.master.Master
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
24/12/03 02:36:12 INFO Master: Started daemon with process name: 19876@NgocPhung
24/12/03 02:36:13 INFO SecurityManager: Changing view acls to: admin
24/12/03 02:36:13 INFO SecurityManager: Changing modify acls to: admin
24/12/03 02:36:13 INFO SecurityManager: Changing view acls groups to:
24/12/03 02:36:13 INFO SecurityManager: Changing modify acls groups to:
24/12/03 02:36:13 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view perm
issions: admin; groups with view permissions: EMPTY; users with modify permissions: admin; groups with modify permission
s: EMPTY
24/12/03 02:36:15 INFO Utils: Successfully started service 'sparkMaster' on port 7077.
24/12/03 02:36:15 INFO Master: Starting Spark master at spark://192.168.1.129:7077
24/12/03 02:36:15 INFO Master: Running Spark version 3.5.1
24/12/03 02:36:15 INFO JettyUtils: Start Jetty 0.0.0.0:8080 for MasterUI
24/12/03 02:36:16 INFO Utils: Successfully started service 'MasterUI' on port 8080.
24/12/03 02:36:16 INFO MasterWebUI: Bound MasterWebUI to 0.0.0.0, and started at http://ngocphung-master:8080
24/12/03 02:36:16 INFO Master: I have been elected leader! New state: ALIVE
```

Worker ID	Address	State	Cores	Memory	Resources
-----------	---------	-------	-------	--------	-----------

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

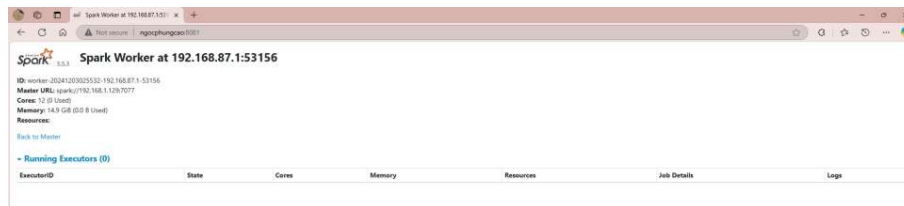
Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
----------------	------	-------	---------------------	------------------------	----------------	------	-------	----------

2.4.4.6 Triển khai trên máy Slave (thực thi trên máy Slave)

Điều hướng đến Spark_Home bằng lệnh sau “cd %SPARK_HOME%\bin”

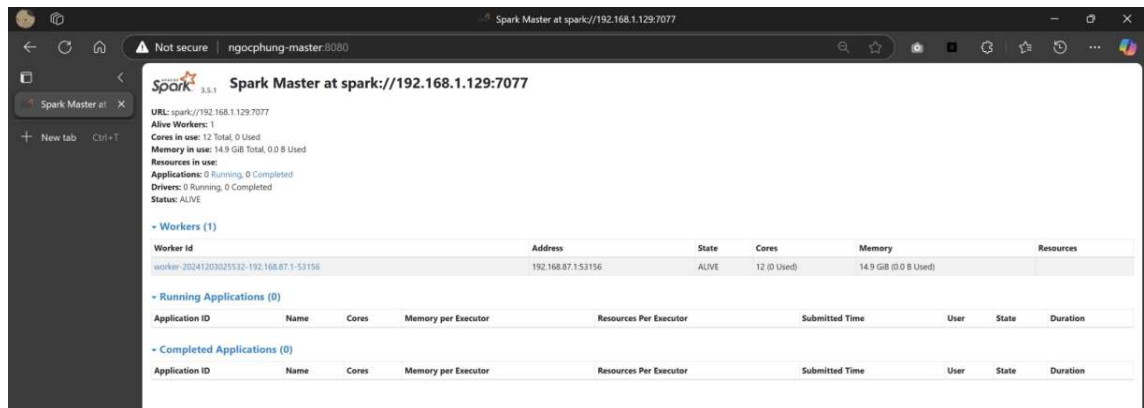
Khởi động máy chủ bằng lệnh sau “spark-class2.cmd org.apache.spark.deploy.
công nhân . Công nhân spark://192.168.1.129:7077 ”

```
C:\Users\phung>cd %SPARK_HOME%\bin
C:\Spark\spark-3.5.3-bin-hadoop3\bin>spark-class2.cmd org.apache.spark.deploy.worker.Worker spark://192.168.1.129:7077
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
24/12/03 02:55:30 INFO Worker: Started daemon with process name: 5024@NgocPhungCao
24/12/03 02:55:31 INFO SecurityManager: Changing view acls to: phung
24/12/03 02:55:31 INFO SecurityManager: Changing modify acls to: phung
24/12/03 02:55:31 INFO SecurityManager: Changing view acls groups to:
24/12/03 02:55:31 INFO SecurityManager: Changing modify acls groups to:
24/12/03 02:55:31 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view perm
issions: phung; groups with view permissions: EMPTY; users with modify permissions: phung; groups with modify permission
s: EMPTY
24/12/03 02:55:32 INFO Utils: Successfully started service 'sparkWorker' on port 53156.
24/12/03 02:55:32 INFO Worker: Worker decommisioning not enabled.
24/12/03 02:55:32 INFO Worker: Starting Spark worker 192.168.87.1:53156 with 12 cores, 14.9 GiB RAM
24/12/03 02:55:32 INFO Worker: Running Spark version 3.5.3
24/12/03 02:55:32 INFO Worker: Spark home: C:\Spark\spark-3.5.3-bin-hadoop3
24/12/03 02:55:32 INFO ResourceUtils: =====
24/12/03 02:55:32 INFO ResourceUtils: No custom resources configured for spark worker.
24/12/03 02:55:32 INFO ResourceUtils: =====
24/12/03 02:55:32 INFO JettyUtils: Start Jetty 0.0.0.0:8081 for WorkerUI
24/12/03 02:55:32 INFO Utils: Successfully started service 'WorkerUI' on port 8081.
24/12/03 02:55:32 INFO WorkerWebUI: Bound WorkerWebUI to 0.0.0.0, and started at http://NgocPhungCao:8081
24/12/03 02:55:32 INFO Worker: Connecting to master 192.168.1.129:7077...
24/12/03 02:55:32 INFO TransportClientFactory: Successfully created connection to /192.168.1.129:7077 after 164 ms (0 ms
spent in bootstraps)
24/12/03 02:55:32 INFO Worker: Successfully registered with master spark://192.168.1.129:7077
```



2.4.4.7 Kiểm tra máy master

Chúng ta có thể thấy trong phần worker, tên của máy khớp với địa chỉ IP của máy phụ.



CHƯƠNG 3: BÀI TẬP APACHE SPARK

Phần 1

Bài 30

Input:

- File log là một tệp văn bản đơn giản (*text file*), trong đó:
- Mỗi dòng tương ứng với một yêu cầu URL từ người dùng đến web server.

Ví dụ, nội dung có thể như sau:

```
66.249.69.97 --[24/Sep/2014:22:25:44 +0000] "GET http://www.google.com/bot.html"
66.249.69.97 --[24/Sep/2014:22:26:44 +0000] "GET http://www.google.com/how.html"
66.249.69.97 --[24/Sep/2014:22:28:44 +0000] "GET http://dbdmg.polito.it/course.html"
71.19.157.179 --[24/Sep/2014:22:30:12 +0000] "GET http://www.google.com/faq.html"
66.249.69.97 --[24/Sep/2014:31:28:44 +0000] "GET http://dbdmg.polito.it/thesis.html"
```

Output:

- Lọc ra những dòng chứa từ "google" (phân biệt hoặc không phân biệt chữ hoa/chữ thường, tùy theo yêu cầu).

```
66.249.69.97 --[24/Sep/2014:22:25:44 +0000] "GET http://www.google.com/bot.html"
66.249.69.97 --[24/Sep/2014:22:26:44 +0000] "GET http://www.google.com/how.html"
71.19.157.179 --[24/Sep/2014:22:30:12 +0000] "GET http://www.google.com/faq.html"
```

Code:

```
import os
import findspark
findspark.init()
import sys
from pathlib import Path
from pyspark.sql import SparkSession

if __name__ == "__main__":
    # Initialize SparkSession
    spark = SparkSession.builder.appName("Exercise-30").getOrCreate()

    # Get the current absolute path
    # Lấy đường dẫn tuyệt đối của thư mục chứa file
    absolute_path = Path(__file__).parent.absolute()

    # Define input and output paths
    input_path = os.path.join(absolute_path, "input.txt")
    output_path = "hdfs://localhost:9000/exercise-30"

    # Read input data
    # Đọc file input dưới dạng DataFrame và chuyển đổi sang RDD
    # Mỗi dòng là một hàng trong DataFrame, lấy giá trị của cột đầu tiên
    lines = spark.read.text(input_path).rdd.map(lambda row: row[0])

    # Filter lines containing the word 'google'
```

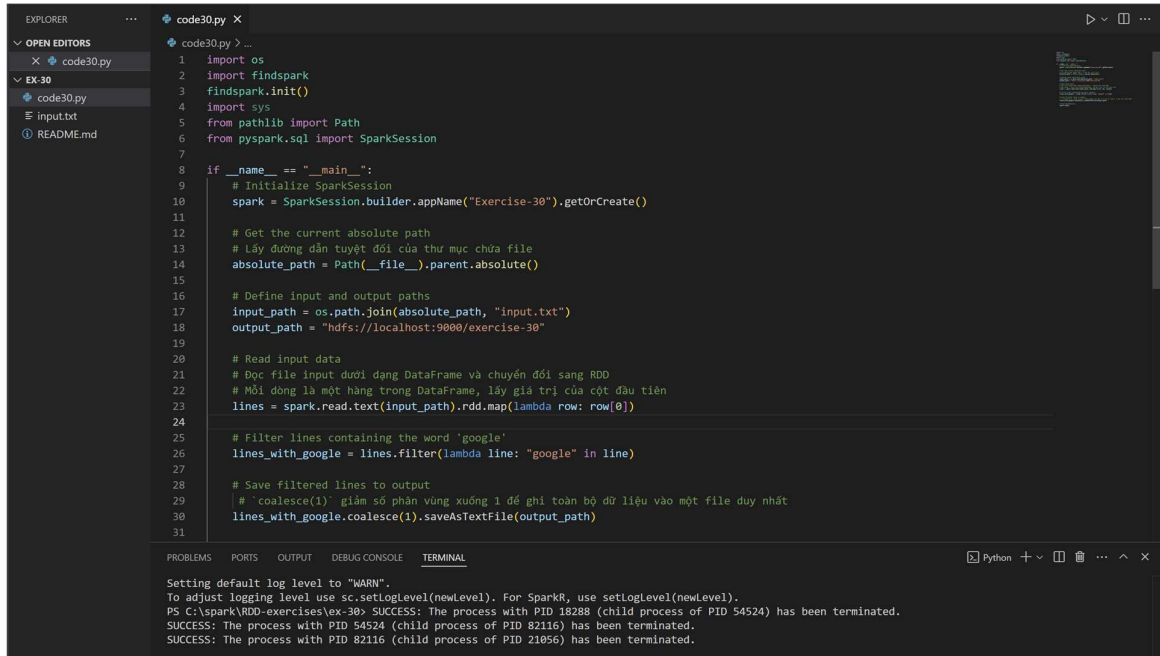
```
lines_with_google = lines.filter(lambda line: "google" in line)
```

```
# Save filtered lines to output
```

```
# `coalesce(1)` giảm số phân vùng xuống 1 để ghi toàn bộ dữ liệu vào một file duy nhất  
lines_with_google.coalesce(1).saveAsTextFile(output_path)
```

```
# Stop SparkSession
```

```
spark.stop()
```



```
code30.py
1 import os
2 import findspark
3 findspark.init()
4 import sys
5 from pathlib import Path
6 from pyspark.sql import SparkSession
7
8 if __name__ == "__main__":
9     # Initialize SparkSession
10     spark = SparkSession.builder.appName("Exercise-30").getOrCreate()
11
12     # Get the current absolute path
13     # Lấy đường dẫn tuyệt đối của thư mục chứa file
14     absolute_path = Path(__file__).parent.absolute()
15
16     # Define input and output paths
17     input_path = os.path.join(absolute_path, "input.txt")
18     output_path = "hdfs://localhost:9000/exercise-30"
19
20     # Read input data
21     # Đọc file input dưới dạng DataFrame và chuyển đổi sang RDD
22     # Mỗi dòng là một hàng trong DataFrame, lấy giá trị của cột đầu tiên
23     lines = spark.read.text(input_path).rdd.map(lambda row: row[0])
24
25     # Filter lines containing the word 'google'
26     lines_with_google = lines.filter(lambda line: "google" in line)
27
28     # Save filtered lines to output
29     # `coalesce(1)` giảm số phân vùng xuống 1 để ghi toàn bộ dữ liệu vào một file duy nhất
30     lines_with_google.coalesce(1).saveAsTextFile(output_path)
31
```

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
PS C:\spark\RDD-exercises\ex-30> SUCCESS: The process with PID 18288 (child process of PID 54524) has been terminated.
SUCCESS: The process with PID 54524 (child process of PID 82116) has been terminated.
SUCCESS: The process with PID 82116 (child process of PID 21056) has been terminated.
```

```
C:\Users\ASUS>hdfs dfs -ls /exercise-30
Found 2 items
-rw-r--r--  3 ASUS supergroup          0 2024-11-24 11:05 /exercise-30/_SUCCESS
-rw-r--r--  3 ASUS supergroup       257 2024-11-24 11:05 /exercise-30/part-000000

C:\Users\ASUS>hdfs dfs -cat /exercise-30/part-000000
66.249.69.97 --[24/Sep/2014:22:25:44 +0000] "GET http://www.google.com/bot.htmlGÇ¥
66.249.69.97 --[24/Sep/2014:22:26:44 +0000] "GET http://www.google.com/how.htmlGÇ¥
71.19.157.179 --[24/Sep/2014:22:30:12 +0000] "GET http://www.google.com/faq.htmlGÇ¥

C:\Users\ASUS>
```

Bài 31

Input:

- Một file văn bản (log file của web server) chứa các dòng nhật ký.
- Mỗi dòng đại diện cho một yêu cầu URL, bao gồm thông tin IP và URL được truy cập.

```
66.249.69.97 -- [24/Sep/2014:22:25:44 +0000] "GET http://www.google.com/bot.html"
66.249.69.97 -- [24/Sep/2014:22:26:44 +0000] "GET http://www.google.com/how.html"
66.249.69.97 -- [24/Sep/2014:22:28:44 +0000] "GET http://dbdmg.polito.it/course.html"
71.19.157.179 -- [24/Sep/2014:22:30:12 +0000] "GET http://www.google.com/faq.html"
66.249.69.95 -- [24/Sep/2014:31:28:44 +0000] "GET http://dbdmg.polito.it/thesis.html"
66.249.69.97 -- [24/Sep/2014:56:26:44 +0000] "GET http://www.google.com/how.html"
56.249.69.97 -- [24/Sep/2014:56:26:44 +0000] "GET http://www.google.com/how.html"
```

Output:

Danh sách các địa chỉ IP duy nhất (distinct IP addresses) đã thực hiện kết nối đến các URL chứa cụm từ "**www.google.com**". Lưu trong HDFS

```
66.249.69.97
71.19.157.179
56.249.69.97
```

Code:

```
import os
from pathlib import Path

from pyspark.sql import SparkSession

if __name__ == "__main__":

    # Create an instance of spark
    spark = SparkSession.builder.appName('Exercise-31').getOrCreate()

    # Current path
    absolute_path = Path().absolute()

    # Input path
    input_path = os.path.join(absolute_path, 'input.txt')

    # Output path
    output_path = "hdfs://localhost:9000/exercise-31"

    # Input data
    # Đọc file input vào Spark dưới dạng RDD
    # spark.read.text(input_path): Đọc file văn bản thành một DataFrame
    # .rdd: Chuyển đổi DataFrame sang RDD
    # .map(lambda x: x[0]): Lấy giá trị của cột đầu tiên (nội dung từng dòng)
    lines = spark.read.text(input_path).rdd.map(lambda x: x[0])

    # Filter out data not containing the work 'google'
    lines_w_google = lines.filter(lambda x: 'google' in x)
```

```

# Take unique IPs
# .map(lambda x: str.split(x, '')[0]): Lấy từ đầu tiên của mỗi dòng (địa chỉ IP)
# .distinct(): Loại bỏ các địa chỉ IP trùng lặp
unique_ips = lines_w_google.map(lambda x: str.split(x, '')[0]).distinct()

# Save data on the output file
# coalesce(1): Giảm số lượng tệp đầu ra xuống 1 tệp duy nhất
# saveAsTextFile(output_path): Lưu kết quả vào thư mục output
unique_ips.coalesce(1).saveAsTextFile(output_path)

# Stop spark
spark.stop()

```

The screenshot shows a VS Code editor with a Python script named `code_31.py` and its terminal output. The script uses PySpark to read data from `input.txt`, filter out lines containing 'google', and save the unique IP addresses to `output_path`. The terminal output shows the successful execution of the script and the termination of the Spark processes.

```

EXPLORER
  code_31.py
  log4j2.properties
  README.md

code_31.py
1 import os
2 from pathlib import Path
3
4 from pyspark.sql import SparkSession
5
6
7 if __name__ == "__main__":
8
9     # Create an instance of spark
10    spark = SparkSession.builder.appName('Exercise-31').getOrCreate()
11
12    # Current path
13    absolute_path = Path().absolute()
14
15    # Input path
16    input_path = os.path.join(absolute_path, 'input.txt')
17
18    # Output path
19    output_path = "hdfs://localhost:9000/exercise-31"
20
21    # Input data
22    # Đọc file input vào Spark dưới dạng RDD
23    # spark.read.text(input_path): Đọc file văn bản thành một DataFrame
24    # .rdd: Chuyển đổi DataFrame sang RDD
25    # .map(lambda x: x[0]): Lấy giá trị của cột đầu tiên (nội dung từng dòng)
26    lines = spark.read.text(input_path).rdd.map(lambda x: x[0])
27
28    # Filter out data not containing the word 'google'
29    lines_w_google = lines.filter(lambda x: 'google' in x)
30
31    # Take unique IPs

```

```

PROBLEMS  PORTS  OUTPUT  DEBUG CONSOLE  TERMINAL
at java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:511)
at java.util.concurrent.FutureTask.run(FutureTask.java:266)
at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
at java.lang.Thread.run(Thread.java:748)
SUCCESS: The process with PID 27976 (child process of PID 38504) has been terminated.
SUCCESS: The process with PID 38504 (child process of PID 46536) has been terminated.
SUCCESS: The process with PID 46536 (child process of PID 17632) has been terminated.

```

```

C:\Users\ASUS>hdfs dfs -ls /exercise-31
Found 2 items
-rw-r--r--  3 ASUS supergroup          0 2024-11-24 16:58 /exercise-31/_SUCCESS
-rw-r--r--  3 ASUS supergroup        40 2024-11-24 16:58 /exercise-31/part-00000

C:\Users\ASUS>hdfs dfs -cat /exercise-31/part-00000
66.249.69.97
71.19.157.179
56.249.69.97

C:\Users\ASUS>

```

Bài 32

Input:

Một tập hợp các tệp văn bản CSV (dạng có cấu trúc). Mỗi dòng của tệp CSV chứa thông tin về **giá trị PM10 hàng ngày** được đo từ một cảm biến.

Định dạng của mỗi dòng: sensorId, date, PM10_value

Trong đó:

- sensorId: ID của cảm biến.
- date: Ngày đo giá trị.
- PM10_value: Giá trị PM10 đo được trong ngày ($\mu\text{g}/\text{m}^3$).

```
s1,2016-01-01,20.5
s2,2016-01-01,30.1
s1,2016-01-02,60.2
s2,2016-01-02,20.4
s1,2016-01-03,55.5
s2,2016-01-03,52.5
```

Output:

Giá trị PM10 lớn nhất trong tất cả các giá trị đo từ các cảm biến.

Kết quả được in trực tiếp ra **standard output** (màn hình console).

```
60.2
```

Code:

```
import sys
import os
from pathlib import Path

from pyspark.sql import SparkSession

def split_data(line):
    """
    Return the temperature of each line
    """
    data = str.split(line, ',') # Chia dòng thành một danh sách với dấu phẩy là ký tự phân tách
    return data[2] # Trả về giá trị PM10 (vị trí thứ 3 trong dòng)

if __name__ == "__main__":

    # Create an instance of spark
    spark = SparkSession.builder.appName('Exercise-32').getOrCreate()

    # Current path
    absolute_path = Path().absolute()

    # Input path
    input_path = os.path.join(absolute_path, 'input.csv')
```



```

# Get a spark context
sc = spark.sparkContext

# Input data from CSV file
lines = sc.textFile(input_path)

# Extract temperature PM10 from sensors data
# .map(split_data): Áp dụng hàm `split_data` cho từng dòng dữ liệu
temperatures = lines.map(split_data)

# Take the maximum temperature
max_temp = temperatures.reduce(lambda x,y : x if x > y else y)

# Print the result on the standard output
print(max_temp)

# Stop spark
spark.stop()

```

The screenshot shows a VS Code editor with a file named `code32.py`. The script defines a `split_data` function and a main block that initializes a Spark session, sets the current path, defines the input path, and runs the Spark code from the first block. The terminal at the bottom shows the execution output, including log level settings and success messages for process termination.

```

7 def split_data(line):
8     """
9     """
10    data = str.split(line, ',') # Chia dòng thành một danh sách với dấu phẩy là ký tự phân tách
11    return data[2] # Trả về giá trị PM10 (vị trí thứ 3 trong dòng)
12
13
14
15 if __name__ == "__main__":
16
17     # Create an instance of spark
18     spark = SparkSession.builder.appName('Exercise-32').getOrCreate()
19
20     # Current path
21     absolute_path = Path().absolute()
22
23     # Input path
24     input_path = os.path.join(absolute_path, 'input.csv')
25
26     # Get a spark context
27     sc = spark.sparkContext
28
29     # Input data from CSV file
30     lines = sc.textFile(input_path)

```

Terminal Output:

```

PS C:\spark\RDD-exercises\ex-32> & C:/Users/ASUS/anaconda3/python.exe c:/spark/RDD-exercises/ex-32/code32.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
60.2
PS C:\spark\RDD-exercises\ex-32> SUCCESS: The process with PID 54928 (child process of PID 18164) has been terminated.
SUCCESS: The process with PID 18164 (child process of PID 86856) has been terminated.
SUCCESS: The process with PID 86856 (child process of PID 8548) has been terminated.

```

Bài 34

Input:

Một tập hợp các tệp văn bản CSV (dạng có cấu trúc). Mỗi dòng của tệp CSV chứa thông tin về **giá trị PM10 hàng ngày** được đo từ một cảm biến.

Định dạng của mỗi dòng: sensorId, date, PM10_value

Trong đó:

- sensorId: ID của cảm biến.
- date: Ngày đo giá trị.
- PM10_value: Giá trị PM10 đo được trong ngày ($\mu\text{g}/\text{m}^3$).

Output:

Kết quả đầu ra là tất cả các dòng trong tệp CSV có giá trị PM10 lớn nhất.

Code:

```
import sys
import os
from pathlib import Path

from pyspark.sql import SparkSession

def split_data(line):
    """
    Return the temperature of each line
    """
    data = str.split(line, ',') # Chia dòng thành một danh sách với dấu phẩy là ký tự phân tách
    return float(data[2]) # Trả về giá trị PM10 (vị trí thứ 3 trong dòng)

if __name__ == "__main__":

    # Create an instance of spark
    spark = SparkSession.builder.appName('Exercise-34').getOrCreate()

    # Current path
    absolute_path = Path().absolute()

    # Input path
    input_path = os.path.join(absolute_path, 'input.csv')

    # Output path
    output_path = "hdfs://localhost:9000/exercise-34"

    # Get a spark context
    sc = spark.sparkContext

    # Input data from CSV file
    lines = sc.textFile(input_path)

    # First get the maximum temperature
```

Sử dụng map để lấy PM10 của mỗi dòng và top(1) để lấy giá trị lớn nhất, rồi lấy phần tử đầu tiên

```
max_temp = lines.map(split_data).max()
```

Lọc ra các dòng có giá trị PM10 bằng với giá trị lớn nhất

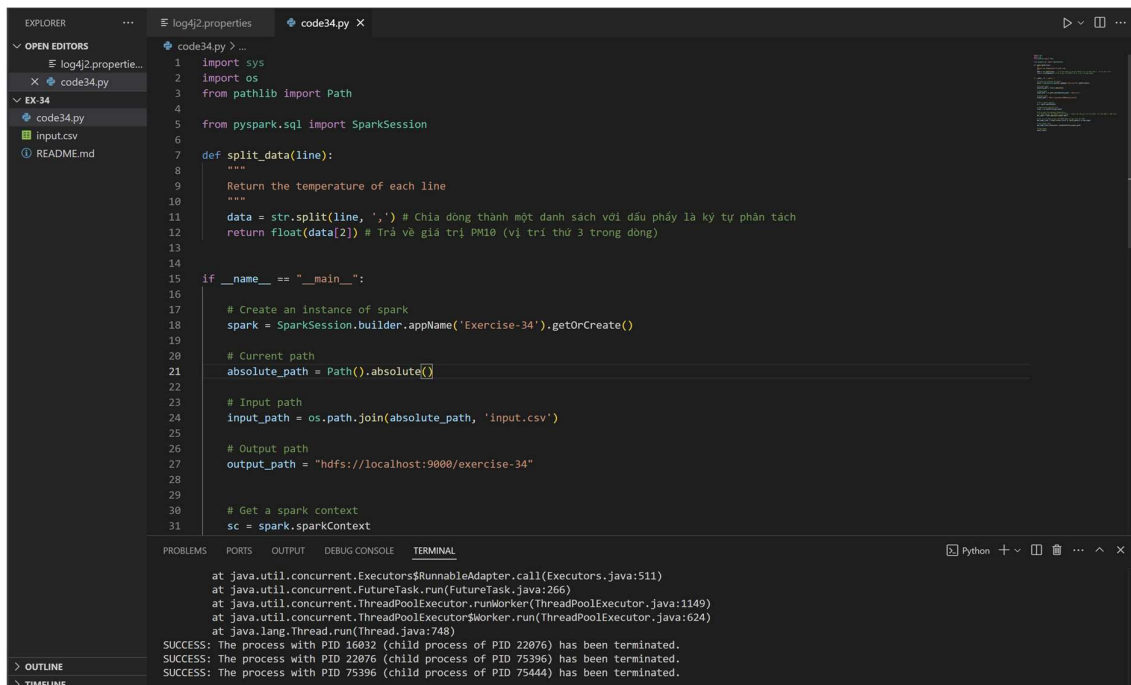
```
max_temp_lines = lines.filter(lambda x: split_data(x) == max_temp)
```

Lưu output hdfs

```
max_temp_lines.coalesce(1).saveAsTextFile(output_path)
```

Stop spark

```
spark.stop()
```



```
code34.py
1 import sys
2 import os
3 from pathlib import Path
4
5 from pyspark.sql import SparkSession
6
7 def split_data(line):
8     """
9     Return the temperature of each line
10    """
11    data = str.split(line, ',') # Chia dòng thành một danh sách với dấu phẩy là ký tự phân tách
12    return float(data[2]) # Trả về giá trị PM10 (vị trí thứ 3 trong dòng)
13
14
15 if __name__ == "__main__":
16
17     # Create an instance of spark
18     spark = SparkSession.builder.appName('Exercise-34').getOrCreate()
19
20     # Current path
21     absolute_path = Path().absolute()
22
23     # Input path
24     input_path = os.path.join(absolute_path, 'input.csv')
25
26     # Output path
27     output_path = "hdfs://localhost:9000/exercise-34"
28
29
30     # Get a spark context
31     sc = spark.sparkContext
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
at java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:511)
at java.util.concurrent.FutureTask.run(FutureTask.java:266)
at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
at java.lang.Thread.run(Thread.java:748)
SUCCESS: The process with PID 16032 (child process of PID 22076) has been terminated.
SUCCESS: The process with PID 22076 (child process of PID 75396) has been terminated.
SUCCESS: The process with PID 75396 (child process of PID 75444) has been terminated.
```

```
C:\Users\ASUS>hdfs dfs -ls /exercise-34
Found 2 items
-rw-r--r--  3 ASUS supergroup          0 2024-11-24 17:12 /exercise-34/_SUCCESS
-rw-r--r--  3 ASUS supergroup        38 2024-11-24 17:12 /exercise-34/part-00000

C:\Users\ASUS>hdfs dfs -cat /exercise-34/part-00000
s1,2016-01-02,60.2
s1,2016-01-03,60.2

C:\Users\ASUS>
```

Bài 43 Phân tích dữ liệu trạm chia sẻ xe đạp

Input:

Tệp txt về trạng thái trạm:

Gồm các dòng dữ liệu với định

dạng: stationId, date, hour, minute, num_of_bikes, num_of_free_slots

- ✓ stationId: Mã định danh trạm.
- ✓ date: Ngày ghi nhận.
- ✓ hour: Giờ ghi nhận (0-23).
- ✓ minute: Phút ghi nhận.
- ✓ num_of_bikes: Số lượng xe có sẵn tại trạm.
- ✓ num_of_free_slots: Số lượng chỗ trống tại trạm.

Lưu ý:

- Mỗi dòng là một mẫu dữ liệu thu thập được cách nhau 5 phút.
- Một số dữ liệu có thể bị thiếu do lỗi kỹ thuật.
- Tổng số trạm: 100.

Tệp txt về danh sách lân cận:

- Gồm các dòng dữ liệu với định dạng: stationIdx, list_of_neighbors
- Ví dụ: s1,s2 s3 nghĩa là trạm s1 có trạm lân cận là s2 và s3.

Output:

1. Tỷ lệ trạng thái nguy cấp của từng trạm:

- Định nghĩa trạng thái nguy cấp: Trạm có số chỗ trống nhỏ hơn một ngưỡng do người dùng cung cấp, ví dụ: num_of_free_slots < 3
- Tỷ lệ nguy cấp:

$$\text{Percentage critical situations} = \frac{\text{Số lần trạm Si quá tải}}{\text{Tổng số lần lấy mẫu của trạm Si}}$$

Kết quả lưu vào HDFS:

- Các trạm có tỷ lệ trạng thái nguy cấp > 80%.

- Định dạng: Mỗi dòng chứa stationId và tỷ lệ nguy cấp, sắp xếp giảm dần theo tỷ lệ.

2. Tỷ lệ nguy cấp theo cặp (khoảng thời gian, trạm):

- Định nghĩa khoảng thời gian: Chia ngày thành 6 khoảng:
 - [0-3], [4-7], [8-11], [12-15], [16-19], [20-23].
- Tỷ lệ nguy cấp cho mỗi cặp (timeslot, station):

Percentage critical situations for pair (timeslot, station)

$$= \frac{(\text{số lần cặp (timeslot, station) quá tải})}{(\text{tổng số lần lấy mẫu trong khung giờ của station})}$$

- Kết quả lưu vào HDFS:
 - Các cặp có tỷ lệ trạng thái nguy cấp > 80%.
 - Định dạng: Mỗi dòng chứa cặp (timeslot, stationId) và tỷ lệ nguy cấp, sắp xếp giảm dần.

3. Chọn các dòng dữ liệu với điều kiện:

- Trạng thái trạm đầy đủ: Trạm Si tại thời điểm đó **đầy hoàn toàn**:
 - num_of_free_slots = 0 (không còn chỗ trống).
- Các trạm lân cận cũng đầy đủ:
 - Mỗi trạm lân cận có num_of_free_slots = 0 trong cùng thời điểm.
- Lưu các dòng thỏa mãn điều kiện vào HDFS và in tổng số dòng đó.

Code:

```
from pyspark import SparkContext, SparkConf
import operator as op
import os
import argparse

# Khởi tạo trình phân tích cú pháp để nhận tham số từ dòng lệnh
args = argparse.ArgumentParser()

# Tham số '-t' xác định ngưỡng (threshold) cho các trường hợp nguy cấp
args.add_argument('-t', default=3, help='Threshold for critical situations', type=int)

# Hàm kiểm tra nếu số lượng slot trống thấp hơn ngưỡng
def is_critical(free_slot, threshold):
    return (1, 1) if int(free_slot) < int(threshold) else (0, 1)
    # (1, 1): slot trống < threshold, (0, 1): slot trống >= threshold

# Xử lý dữ liệu đọc được để đánh dấu tình trạng nguy cấp
def reading_map_critical_func(x: str, threshold: int) -> list:
    data = x.split(',') # Tách chuỗi dữ liệu thành danh sách
    free_slot = data[5] # Lấy giá trị slot trống
    return [data[0], is_critical(free_slot, threshold)] # Trả về (station_id, tình trạng)

# Ánh xạ dữ liệu thành khung thời gian 4 giờ
def mapping_hour(x, threshold):
    data = x.split(',')
    min_hour = 4 * (int(data[2]) // 4) # Tìm khung giờ nhỏ nhất (bội số của 4)
    max_hour = min_hour + 3 # Khung giờ lớn nhất
    free_slot = data[5]
    return [f'{data[0]}_{min_hour}-{max_hour}', is_critical(free_slot, threshold)]
    # Trả về dạng station_id_[min-max], tình trạng

# Ánh xạ dữ liệu với timestamp
def mapping_timestamp_func(x):
    data = x.split(',')
    time_stamp = f'{data[1]} {data[2]}:{data[3]}' # Ghép ngày và giờ
    return [time_stamp, x] # Trả về (timestamp, dữ liệu gốc)

# Ánh xạ neighbor từ tệp
def neighbor_map_func(x: str):
    data = x.split(',')
    return [data[0], tuple(data[1].split(' '))] # Trả về (station_id, danh sách neighbor)

# Lọc ra các dòng có số lượng slot bằng 0
def zero_slot_filter(x):
    data = x.split(',')
    free_slot = int(data[5])
    return free_slot == 0 # Chỉ giữ lại các dòng có slot trống = 0

# Hàm cộng giá trị tuple
def add_func(val1, val2):
    return (val1[0] + val2[0], val1[1] + val2[1]) # Cộng giá trị tuple từng phần tử

# Tính phần trăm tình trạng nguy cấp
def percentage_map_func(x):
    return (x[0], x[1][0] / x[1][1]) # Tính tỷ lệ: số lần nguy cấp / tổng số lần
```

```

# Điểm khởi đầu của chương trình
if __name__ == '__main__':
    arg = args.parse_args() # Parse tham số
    threshold = arg.t # Lấy giá trị threshold từ tham số

    # Cấu hình Spark
    conf = (
        SparkConf()
        .setAppName("Exercise-43")
        .setMaster("local[4]") # Chạy cục bộ với 4 luồng
        .set("spark.hadoop.fs.defaultFS", "file:///") # Sử dụng hệ thống file cục bộ
    )

    sc = SparkContext(conf=conf).getOrCreate()

    path = os.path.dirname(os.path.abspath(__file__)) # Lấy đường dẫn file hiện tại

    # Đường dẫn tệp input
    neighbor_input = os.path.join(path, 'neighbors.txt')
    reading_input = os.path.join(path, 'readings.txt')

    # Đường dẫn tệp output
    percentage_critical_situations_output = "hdfs://localhost:9000/exercise-43/percentage_critical_situations"
    percentage_critical_situations_ts_output = "hdfs://localhost:9000/exercise-43/percentage_critical_situations_ts"
    station_full_output = "hdfs://localhost:9000/exercise-43/station_full"

    neighbor = sc.textFile(neighbor_input) # Đọc tệp neighbor
    reading = sc.textFile(reading_input).cache() # Đọc tệp reading và cache lại để tái sử dụng

    ## Phần 1: Tính phần trăm tình trạng nguy cấp cho từng station
    station_count_pair_rdd = reading.map(lambda x: reading_map_critical_func(x, threshold)) #
    Ánh xạ với hàm critical
    station_total_count_pair = station_count_pair_rdd.reduceByKey(add_func) # Gom nhóm và
    cộng các giá trị
    station_percentage_pair = station_total_count_pair.map(percentage_map_func) \
        .filter(lambda x: x[1] >= 0.8).sortBy(lambda x: x[1], ascending=False) # Lọc và sắp xếp
    theo tỷ lệ

    station_percentage_pair.coalesce(1).saveAsTextFile(percentage_critical_situations_output) #
    Lưu kết quả

    ## Phần 2: Tính phần trăm tình trạng nguy cấp theo khung giờ
    station_timeslot_rdd = reading.map(lambda x: mapping_hour(x, threshold)) # Ánh xạ dữ liệu
    thành khung giờ
    station_timeslot_percentage = station_timeslot_rdd.reduceByKey(add_func) \
        .map(percentage_map_func) \
        .filter(lambda x: x[1] > 0.8).sortBy(lambda x: x[1], ascending=False) # Lọc và sắp xếp
    theo tỷ lệ

    station_timeslot_percentage.coalesce(1).saveAsTextFile(percentage_critical_situations_ts_output
    ) # Lưu kết quả

    ## Phần 3: Tìm các station có slot trống = 0 và tất cả neighbor cũng trống

```

```

zero_slot_station = reading.filter(zero_slot_filter).map(mapping_timestamp_func) # Lọc các
station có slot = 0
zero_slot_group_by_timestamp = zero_slot_station.groupByKey().mapValues(list) # Nhóm theo
timestamp

```

```

neighbor_map = neighbor.map(neighbor_map_func) # Ánh xạ tệp neighbor
neighbors = {k: v for k, v in neighbor_map.collect()} # Chuyển RDD thành dictionary
neighbor_broadcast = sc.broadcast(neighbors) # Phát broadcast dictionary neighbor

```

Hàm kiểm tra xem tất cả neighbor của station có trống không

```

def filter_func(data: list, neighbor_broadcast):
    stations = []
    for line in data:
        stations.append(line.split(',')[0]) # Lấy danh sách station

```

for station in stations:

for neighbor in neighbor_broadcast.value[station]:

if neighbor not in stations: # Nếu neighbor không thuộc danh sách -> False
return False

return True # Nếu tất cả neighbor có trong danh sách -> True

```

full_stations = zero_slot_group_by_timestamp.filter(lambda x: filter_func(x[1],
neighbor_broadcast)) # Lọc station thỏa mãn

```

```

full_stations.coalesce(1).saveAsTextFile(station_full_output) # Lưu kết quả

```

```

EXPLORER
log4j2.properties X mycode.py X part-00000
mycode.py > ...
72 path = os.path.dirname(os.path.abspath(__file__)) # Lấy đường dẫn file hiện tại
73
74 # Đường dẫn tệp input
75 neighbor_input = os.path.join(path, 'neighbors.txt')
76 reading_input = os.path.join(path, 'readings.txt')
77
78 # Đường dẫn tệp output
79 percentage_critical_situations_output = "hdfs://localhost:9000/exercise-43/percentage_critical_situations"
80 percentage_critical_situations_ts_output = "hdfs://localhost:9000/exercise-43/percentage_critical_situations_ts"
81 station_full_output = "hdfs://localhost:9000/exercise-43/station_full"
82
83 neighbor = sc.textFile(neighbor_input) # Đọc tệp neighbor
84 reading = sc.textFile(reading_input).cache() # Đọc tệp reading và cache lại để tái sử dụng
85
86 ## Phần 1: Tính phần trăm tình trạng nguy cấp cho từng station
87 station_count_pair_rdd = reading.map(lambda x: reading_map_critical_func(x, threshold)) # Ánh xạ với hàm critical
88 station_total_count_pair = station_count_pair_rdd.reduceByKey(add_func) # Gom nhóm và cộng các giá trị
89 station_percentage_pair = station_total_count_pair.map(percentage_map_func) \
90     .filter(lambda x: x[1] >= 0.8).sortBy(lambda x: x[1], ascending=False) # Lọc và sắp xếp theo tỷ lệ
91
92 station_percentage_pair.coalesce(1).saveAsTextFile(percentage_critical_situations_output) # Lưu kết quả
93
94 ## Phần 2: Tính phần trăm tình trạng nguy cấp theo khung giờ
95 station_timeslot_rdd = reading.map(lambda x: mapping_hour(x, threshold)) # Ánh xạ dữ liệu thành khung giờ
96 station_timeslot_percentage = station_timeslot_rdd.reduceByKey(add_func) \
97     .map(percentage_map_func) \
98     .filter(lambda x: x[1] > 0.8).sortBy(lambda x: x[1], ascending=False) # Lọc và sắp xếp theo tỷ lệ
99
100 station_timeslot_percentage.coalesce(1).saveAsTextFile(percentage_critical_situations_ts_output) # Lưu kết quả
101
102 ## Phần 3: Tìm các station có slot trống = 0 và tất cả neighbor cũng trống
103
TERMINAL
at java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:511)
at java.util.concurrent.FutureTask.run(FutureTask.java:266)
at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
at java.lang.Thread.run(Thread.java:748)
SUCCESS: The process with PID 28924 (child process of PID 63852) has been terminated.
SUCCESS: The process with PID 63852 (child process of PID 85820) has been terminated.
SUCCESS: The process with PID 85820 (child process of PID 46620) has been terminated.

```



```

C:\Users\ASUS>hdfs dfs -ls /exercise-43
Found 3 items
drwxr-xr-x - ASUS supergroup 0 2024-11-24 17:18 /exercise-43/percentage_critical_situations
drwxr-xr-x - ASUS supergroup 0 2024-11-24 17:18 /exercise-43/percentage_critical_situations_ts
drwxr-xr-x - ASUS supergroup 0 2024-11-24 17:18 /exercise-43/station_full

C:\Users\ASUS>hdfs dfs -ls /exercise-43/percentage_critical_situations
Found 2 items
-rw-r--r-- 3 ASUS supergroup 0 2024-11-24 17:18 /exercise-43/percentage_critical_situations/_SUCCESS
-rw-r--r-- 3 ASUS supergroup 12 2024-11-24 17:18 /exercise-43/percentage_critical_situations/part-00000

C:\Users\ASUS>hdfs dfs -cat /exercise-43/percentage_critical_situations/part-00000
('s4', 1.0)

C:\Users\ASUS>hdfs dfs -cat /exercise-43/percentage_critical_situations_ts/part-00000
('s4_[12-15]', 1.0)
('s4_[0-3]', 1.0)

C:\Users\ASUS>hdfs dfs -cat /exercise-43/station_full/part-00000
C:\Users\ASUS>

```

Phần 2

Bài 35

Đọc file CSV từ HDFS, trích xuất nhiệt độ từ mỗi ngày và xác định nhiệt độ cao nhất trong tập dữ liệu. Sau đó, nó lọc ra tất cả các ngày có nhiệt độ bằng giá trị cao nhất này và ghi danh sách các ngày đó vào một file trên HDFS, mỗi ngày xuất hiện trên một dòng.

Input

```

C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-35/input/input.csv
s1,2016-01-01,20.5
s2,2016-01-01,30.1
s1,2016-01-02,60.2
s2,2016-01-02,20.4
s1,2016-01-03,60.2
s2,2016-01-03,52.5

```

Output

```

C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-35/output/part-00000
2016-01-02
2016-01-03

```

Code

```

import findspark
# Khởi tạo Spark từ SPARK_HOME
findspark.init("C:\Spark\spark-3.5.1-bin-hadoop3")
import sys
import os
from pathlib import Path
# Import SparkSession để làm việc với Spark
from pyspark.sql import SparkSession
# Hàm xử lý từng dòng trong file CSV
def split_data(line):
    """
    Return the temperature of each line
    """
    # Tách một dòng thành danh sách các giá trị, ngăn cách bởi dấu phẩy
    data = str.split(line, ',')
    return data[2]
# Điểm bắt đầu của chương trình
if __name__ == "__main__":
    # Khởi tạo một SparkSession (một ứng dụng Spark)

```

```

spark = SparkSession.builder.appName('Exercise-35').getOrCreate()
sc = spark.sparkContext
# Thư mục chứa file code.py
absolute_path = Path().absolute()
# Đường dẫn đầy đủ đến input.csv
input_path = "hdfs://ngocphung-master:9000/spark/ex-35/input/input.csv"

# Đọc file CSV và tạo RDD, mỗi dòng trong file là một phần tử trong RDD
# VD: ["s1,2016-01-01,20.5", "s2,2016-01-01,30.1", "s3,2016-01-01,25.3"]
lines = sc.textFile(input_path)

# Áp dụng hàm `split_data` để lấy giá trị nhiệt độ từ mỗi dòng
# Kết quả là một RDD chỉ chứa các giá trị nhiệt độ
# Lấy giá trị lớn nhất từ RDD (top(1) trả về danh sách có 1 phần tử, [0] lấy giá trị đầu tiên)
# VD: max_temp: "30.1"
max_temp = lines.map(split_data).top(1)[0]

# Lọc các ngày có nhiệt độ cao nhất
# VD: `max_temp_dates`: ["2016-01-01"] nếu nhiệt độ cao nhất là 30.1 vào ngày "2016-01-01"
max_temp_dates = (
    lines.filter(lambda x: split_data(x) == max_temp) # Lọc các dòng có nhiệt độ bằng
    `max_temp`
    .map(lambda x: str.split(x, ',')[1]) # Lấy cột ngày (cột thứ 2) từ các dòng đã lọc
    ) # Giữ max_temp_dates là một RDD

# Xuất kết quả ra HDFS, hợp nhất thành 1 file
max_temp_dates.coalesce(1).saveAsTextFile('hdfs://ngocphung-master:9000/spark/ex-35/output')
# Dừng SparkSession để giải phóng tài nguyên
spark.stop()

```

```

spark-submit --master local[*] hdfs://ngocphung-master:9000/spark/ex-35/code.py

```

```

C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -ls /spark/ex-35/output
Found 2 items
-rw-r--r--  3 admin supergroup      0 2024-11-23 22:38 /spark/ex-35/output/_SUCCESS
-rw-r--r--  3 admin supergroup    22 2024-11-23 22:38 /spark/ex-35/output/part-000000
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-35/output/part-000000
2016-01-02
2016-01-03

```

Bài 36

Đọc một file CSV chứa dữ liệu nhiệt độ, trong đó mỗi dòng bao gồm thông tin về cảm biến, ngày đo và nhiệt độ. Chương trình xử lý file để tách cột nhiệt độ, sau đó tính trung bình của tất cả các giá trị nhiệt độ. Kết quả cuối cùng là nhiệt độ trung bình, được định dạng với 2 chữ số thập phân và xuất ra.

Input

```
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-36/input/input.csv
s1,2016-01-01,20.5
s2,2016-01-01,30.1
s1,2016-01-02,60.2
s2,2016-01-02,20.4
s1,2016-01-03,55.5
s2,2016-01-03,52.5
```

Output

```
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-36/output/part-00000
39.87
```

Code

```
import findspark
# Khởi tạo Spark từ SPARK_HOME
findspark.init("C:\Spark\spark-3.5.1-bin-hadoop3")
import sys
import os
from pathlib import Path
# Import SparkSession để làm việc với Spark
from pyspark.sql import SparkSession
# Hàm xử lý từng dòng trong file CSV
def split_data(line):
    """
    Return the temperature of each line
    """
    # Tách một dòng thành danh sách các giá trị, ngăn cách bởi dấu phẩy
    data = str.split(line, ',')
    return float(data[2])

# Điểm bắt đầu của chương trình
if __name__ == "__main__":
    # Khởi tạo một SparkSession (một ứng dụng Spark)
    spark = SparkSession.builder.appName('Exercise-36').getOrCreate()
    sc = spark.sparkContext
    # Thư mục chứa file code.py
    current_dir = os.path.dirname(os.path.abspath(__file__))

    # Đường dẫn đầy đủ đến input.csv
    input_path = "hdfs://ngocphung-master:9000/spark/ex-36/input/input.csv"

    # Đọc file CSV và tạo RDD, mỗi dòng trong file là một phần tử trong RDD
    # ["s1,2016-01-01,20.5", "s2,2016-01-01,30.1", "s3,2016-01-01,25.3"]
    lines = sc.textFile(input_path)
```

```
# Áp dụng hàm `split_data` để lấy cột nhiệt độ (cột thứ 3), và tính trung bình các giá trị đó
# VD: [20.5, 30.1, 25.3]
# mean_temp = (20.5 + 30.1 + 25.3) / 3 = 25.3
mean_temp = lines.map(split_data).mean()

# Tạo một RDD chứa giá trị trung bình
# VD: ["25.30"]
result_rdd = sc.parallelize(["%2.2f" % mean_temp])

# Lưu file ở địa chỉ chỉ định
result_rdd.coalesce(1).saveAsTextFile('hdfs://ngocphung-master:9000/spark/ex-36/output')
# Dừng SparkSession để giải phóng tài nguyên
spark.stop()
```

```
spark-submit --master local[*] hdfs://ngocphung-master:9000/spark/ex-36/code.py
```

```
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -ls /spark/ex-36/output
Found 2 items
-rw-r--r-- 3 admin supergroup 0 2024-11-23 17:57 /spark/ex-36/output/_SUCCESS
-rw-r--r-- 3 admin supergroup 6 2024-11-23 17:57 /spark/ex-36/output/part-00000

C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-36/output/part-00000
39.87
```

Bài 37

Đọc dữ liệu từ file CSV từ HDFS và tách từng dòng để lấy cột nhiệt độ. Sau đó, sử dụng hàm top(1) để tìm nhiệt độ cao nhất trong toàn bộ dữ liệu. Tiếp theo, các dòng chứa nhiệt độ cao nhất được lọc ra để lấy thông tin ngày đo ứng với nhiệt độ đó. Cuối cùng, kết quả nhiệt độ cao nhất được ghi nhận từ mỗi cảm biến (sensor) và ngày xảy ra nhiệt độ đó.

Input

```
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-37/input/input.csv
s1,2016-01-01,20.5
s2,2016-01-01,30.1
s1,2016-01-02,60.2
s2,2016-01-02,20.4
s1,2016-01-03,55.5
s2,2016-01-03,52.5
```

Output

```
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-37/output/part-00000
('s1', '60.2')
('s2', '52.5')
```

Code

```
import findspark
# Khởi tạo Spark từ SPARK_HOME
findspark.init("C:\Spark\spark-3.5.1-bin-hadoop3")
import sys
import os
from pathlib import Path
```

```

# Import SparkSession để làm việc với Spark
from pyspark.sql import SparkSession
# Hàm xử lý từng dòng trong file CSV
def split_data(line):
    """
    Split each line into columns
    """
    # Tách một dòng thành danh sách các giá trị, ngăn cách bởi dấu phẩy
    data = str.split(line, ',')
    return [data[0], data[2]]
if __name__ == "__main__":
    # Khởi tạo một SparkSession (một ứng dụng Spark)
    spark = SparkSession.builder.appName('Exercise-37').getOrCreate()
    sc = spark.sparkContext
    # Thư mục chứa file code.py
    absolute_path = Path().absolute()
    # Đường dẫn đầy đủ đến input.csv
    input_path = "hdfs://ngocphung-master:9000/spark/ex-37/input/input.csv"
    # Đọc file CSV và tạo RDD, mỗi dòng trong file là một phần tử trong RDD
    lines = sc.textFile(input_path)
    # VD: ["s1,2016-01-01,20.5", "s2,2016-01-01,30.1"] → [("s1", "20.5"), ("s2", "30.1")].
    # Tách từng dòng dữ liệu thành cặp giá trị (sensorID, temp).
    # Kết quả là một RDD chứa các cặp (sensorID, temp) để dễ dàng xử lý
    sensor_temp = lines.map(split_data)
    # VD: [("s1", "20.5"), ("s1", "30.1"), ("s2", "25.5"), ("s2", "52.5")] → [("s1", "30.1"), ("s2",
    "52.5")]
    # nhóm các giá trị theo sensorID và áp dụng hàm giảm (reduce) để tìm nhiệt độ cao nhất cho mỗi
    cảm biến.
    # bằng cách so sánh giá trị nhiệt độ (x và y) và giữ lại giá trị lớn hơn.
    # thu thập kết quả từ RDD và chuyển thành một danh sách Python chứa các cặp (sensorID,
    max_temp).
    max_temp_by_sensor = sensor_temp.reduceByKey(lambda x,y : x if x > y else y)
    # Xuất kết quả ra HDFS, hợp nhất thành 1 file
    max_temp_by_sensor.coalesce(1).saveAsTextFile('hdfs://ngocphung-master:9000/spark/ex-
    37/output')
    # Dừng SparkSession để giải phóng tài nguyên
    spark.stop()

```

```
spark-submit --master local[*] hdfs://ngocphung-master:9000/spark/ex-36/code.py
```

```

C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -ls /spark/ex-37/output
Found 2 items
-rw-r--r--  3 admin supergroup          0 2024-11-24 01:22 /spark/ex-37/output/_SUCCESS
-rw-r--r--  3 admin supergroup        30 2024-11-24 01:22 /spark/ex-37/output/part-000000

C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-37/output/part-000000
('s1', '60.2')
('s2', '52.5')

```

Bài 38

Xác định số lượng cảm biến (sensor) ghi nhận được nhiệt độ lớn hơn 50 trong nhiều lần đo khác nhau, sau đó đếm số lần xuất hiện của từng cảm biến. Kết quả trả về các cảm biến xuất hiện ít nhất 2 lần với nhiệt độ lớn hơn 50.

VD: ('s1', 2) cho thấy cảm biến s1 ghi nhận được nhiệt độ lớn hơn 50 trong 2 lần đo là ngày s1,2016-01-02,60.2 và s1,2016-01-03,55.5

Input

```
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-38/input/input.csv
s1,2016-01-01,20.5
s2,2016-01-01,30.1
s1,2016-01-02,60.2
s2,2016-01-02,20.4
s1,2016-01-03,55.5
s2,2016-01-03,52.5
```

Output

```
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-38/output/part-00000
('s1', 2)
```

Code

```
import findspark
# Khởi tạo Spark từ SPARK_HOME
findspark.init("C:\Spark\spark-3.5.1-bin-hadoop3")
import sys
import os
from pathlib import Path
from pyspark.sql import SparkSession
from operator import add

def split_data_add_counter(line):
    """
    Split each line into columns
    """
    data = str.split(line, ',')
    return [data[0], 1]

if __name__ == "__main__":
    # Khởi tạo một SparkSession (một ứng dụng Spark)
    spark = SparkSession.builder.appName('Exercise-38').getOrCreate()
    sc = spark.sparkContext
    # Thư mục chứa file code.py
    absolute_path = Path().absolute()
    # Đường dẫn đầy đủ đến input.csv
    input_path = "hdfs://ngocphung-master:9000/spark/ex-38/input/input.csv"

    # Đọc file CSV và tạo RDD, mỗi dòng trong file là một phần tử trong RDD
    # ["s1,2016-01-01,20.5", "s2,2016-01-01,30.1"]
    lines = sc.textFile(input_path)
```

```

# Lọc các dòng có nhiệt độ cao hơn ngưỡng 50
# VD: ["s2,2016-01-01,55.1", "s1,2016-01-02,60.0"]
lines_above = lines.filter(lambda x: float(str.split(x,')[2]) > 50)

# Map từng dòng (sensorID) với một bộ đếm (1)
# VD: `counter_readings`: [("s2", 1), ("s1", 1)]
counter_readings = lines_above.map(split_data_add_counter)

# Tính tổng số lần xuất hiện của mỗi sensorID bằng `reduceByKey`
# Sau đó lọc các sensorID có số đếm > 1
# `reduceByKey(add)` tính tổng số lần xuất hiện của mỗi sensorID
# VD: `reduceByKey` : [("s1", 2), ("s2", 1)]
# Sau `filter(lambda x: x[1] > 1): [("s1", 2)]
readings_above_threshold = counter_readings.reduceByKey(add).filter(lambda x: x[1] > 1)

# Xuất kết quả ra HDFS, hợp nhất thành 1 file
readings_above_threshold.coalesce(1).saveAsTextFile('hdfs://ngocphung-master:9000/spark/ex-38/output')
# Dừng SparkSession để giải phóng tài nguyên
spark.stop()

```

```
spark-submit --master local[*] hdfs://ngocphung-master:9000/spark/ex-36/code.py
```

```

C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -ls /spark/ex-38/output/
Found 2 items
-rw-r--r--  3 admin supergroup          0 2024-11-24 01:37 /spark/ex-38/output/_SUCCESS
-rw-r--r--  3 admin supergroup        10 2024-11-24 01:37 /spark/ex-38/output/part-00000
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-38/output/part-00000
('s1', 2)

```

Bài 44

Input

```
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-44/input/movies.txt
movie1,Toy Story (1995),Animation
movie2,Jumanji (1995),Adventure
movie3,Grumpier Old Men (1995),Comedy
movie4,Waiting to Exhale (1995),Comedy
movie5,Father of the Bride Part II (1995),Comedy
movie6,Heat (1995),Action
movie7,Sabrina (1995),Comedy
movie8,Tom and Huck (1995),Adventure
movie9,Sudden Death (1995),Action
movie10,GoldenEye (1995),Action
```

```
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-44/input/watchedmovies.txt
user1,movie1,201606061500,201606061650
user1,movie3,201606061800,201606061834
user1,movie4,201609061500,201609061605
user1,movie5,201610061100,201610061450
user2,movie6,201610081800,201610081845
user2,movie3,201610091800,201610091834
user2,movie4,201611051100,201611051105
```

```
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-44/input/preferences.txt
user1,Animation
user1,Comedy
user2,Action
```

Output

```
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-44/output/part-00000
user2
```

Code

```
from pyspark import SparkContext, SparkConf
if __name__ == "__main__":
    # Nhận các tham số đầu vào
    inputPathWatched = "hdfs://ngocphung-master:9000/spark/ex-44/input/watchedmovies.txt"
    inputPathPreferences = "hdfs://ngocphung-master:9000/spark/ex-44/input/preferences.txt"
    inputPathMovies = "hdfs://ngocphung-master:9000/spark/ex-44/input/movies.txt"
    outputPath = "hdfs://ngocphung-master:9000/spark/ex-44/output"
    threshold = 0.3 # Thay thế giá trị threshold phù hợp
    # Tạo SparkContext với cấu hình ứng dụng
    conf = SparkConf().setAppName("Spark Exercise #44")
    sc = SparkContext(conf=conf)
    # Đọc file watched movies
    watchedRDD = sc.textFile(inputPathWatched)
    # Tạo cặp (movieID, userID) từ file watchedmovies
    # Chia từng dòng trong file watchedmovies.txt bằng dấu phẩy (,)
    # VD: [("movie1", "user1"), ("movie3", "user1"), ("movie4", "user1")]
    movieUserPairRDD = watchedRDD.map(lambda line: (line.split(",")[1], line.split(",")[0]))
    # Đọc file movies
    moviesRDD = sc.textFile(inputPathMovies)
    # Tạo cặp (movieID, genre) từ file movies
    # Chia từng dòng trong file movies.txt bằng dấu phẩy (,)
    # VD: [("movie1", "Animation"), ("movie2", "Adventure")]
    movieGenrePairRDD = moviesRDD.map(lambda line: (line.split(",")[0], line.split(",")[2]))
    # Thực hiện join watched movies với movies
    # VD: [("movie1", ("user1", "Animation")), ("movie3", ("user1", "Comedy"))]
    joinWatchedGenreRDD = movieUserPairRDD.join(movieGenrePairRDD)
    # Tạo cặp (userID, genre)
```



```

# Lấy userID và genre từ joinWatchedGenreRDD.
# VD: [("user1", "Animation"), ("user1", "Comedy"), ("user1", "Comedy")]
usersWatchedGenresRDD = joinWatchedGenreRDD.map(lambda userMovie: (userMovie[1][0],
userMovie[1][1]))
# Đọc file preferences
preferencesRDD = sc.textFile(inputPathPreferences)
# Tạo RDD userLikedGenresRDD (userid, genre)
# Tạo cặp (userID, liked genre) từ file preferences
# VD: [("user1", "Animation"), ("user1", "Comedy"), ("user2", "Action")]
userLikedGenresRDD = preferencesRDD.map(lambda line: (line.split(",")[0], line.split(",")[1]))
# Cogroup watched genres và liked genres
# [ ("user1", (["Animation", "Comedy", "Comedy", "Comedy"], ["Animation", "Comedy"])),
#   ("user2", (["Action", "Comedy", "Comedy"], ["Action"]))]
userWatchedLikedGenres = usersWatchedGenresRDD.cogroup(userLikedGenresRDD)
# Lọc người dùng có hồ sơ lệch lạc
# user1: Tất cả phim đều thuộc thể loại yêu thích (Animation, Comedy) --> nên không bị lọc.
# user2: Xem 3 phim nhưng chỉ 1 phim thuộc thể loại yêu thích (Action). Tỷ lệ không yêu thích =
2/3 > 0.3,
# --> nên được coi là hồ sơ lệch.
misleadingUsersListsRDD = userWatchedLikedGenres.filter(
    lambda userGenres: (
        lambda likedGenres, watchedGenres: (
            sum(1 for genre in watchedGenres if genre not in likedGenres) >
            threshold * len(list(watchedGenres))
        )
    )(set(userGenres[1][1]), userGenres[1][0])
)
# Lấy userid của người dùng có hồ sơ lệch
misleadingUsersRDD = misleadingUsersListsRDD.keys()
# Lưu kết quả
misleadingUsersRDD.coalesce(1).saveAsTextFile(outputPath)
# Dừng SparkContext
sc.stop()

```

```
spark-submit --master local[*] hdfs://ngocphung-master:9000/spark/ex-44/code.py
```

```

C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -ls /spark/ex-44/output/
Found 2 items
-rw-r--r--  3 admin supergroup      0 2024-11-24 02:08 /spark/ex-44/output/_SUCCESS
-rw-r--r--  3 admin supergroup      6 2024-11-24 02:08 /spark/ex-44/output/part-00000
C:\Hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /spark/ex-44/output/part-00000
user2

```

Phần 3

Bài 33

Đề bài yêu cầu tìm ra 3 giá trị PM10 cao nhất từ một tập hợp các tệp CSV, mỗi tệp chứa dữ liệu ngày và giá trị PM10 của các cảm biến. Kết quả là các giá trị PM10 cao nhất cần được in ra trên đầu ra chuẩn.

Input

Dữ liệu đầu vào là một tệp CSV chứa thông tin về các cảm biến (sensorId), ngày (date), và giá trị PM10 tương ứng

```
E:\Bigdata_Analysis\Final\Exercises\ex33>type input.csv
s1,2016-01-01,20.5
s2,2016-01-01,30.1
s1,2016-01-02,60.2
s2,2016-01-02,20.4
s1,2016-01-03,55.5
s2,2016-01-03,52.5
```

Output

```
PS E:\Bigdata_Analysis\Final\Exercises> & C:/Users/LG/AppData/Local/Programs/Python/Python
PS E:\Bigdata_Analysis\Final\Exercises> & C:/Users/LG/AppData/Local/Programs/Python/Python
311/python.exe e:/Bigdata_Analysis/Final/Exercises/ex33/ex33.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel
).
60.2
55.5
52.5
PS E:\Bigdata_Analysis\Final\Exercises> SUCCESS: The process with PID 18096 (child process
of PID 13720) has been terminated.
SUCCESS: The process with PID 13720 (child process of PID 18128) has been terminated.
SUCCESS: The process with PID 18128 (child process of PID 14220) has been terminated.
```

Code

```
import sys
import os
from pathlib import Path
import findspark

from pyspark.sql import SparkSession

findspark.init()
def split_data(line):
    """
    Return the temperature of each line
    """
    data = str.split(line, ',')
    return data[2] # Trả về trường thứ 3

#Input: "sensor1,2023-11-22,35.6"
#Output: "35.6"
```

```

if __name__ == "__main__":

    spark = SparkSession.builder.appName('Exercise-33').getOrCreate()
    absolute_path = Path().absolute()
    input_path = os.path.join(absolute_path, "E:\Bigdata_Analysis\Final\Exercises\ex33\input.csv")

    sc = spark.sparkContext # tạo SparkContext để thực hiện các thao tác RDD

    lines = sc.textFile(input_path) #Đọc tệp CSV dưới dạng RDD. Mỗi dòng trong tệp sẽ là một phần tử trong RDD
    #Input :sensor1,2023-11-22,35.6
    #sensor2,2023-11-22,36.1
    #sensor3,2023-11-22,34.8
    #OUT["sensor1,2023-11-22,35.6", "sensor2,2023-11-22,36.1", "sensor3,2023-11-22,34.8"]

    temperatures = lines.map(split_data) # Áp dụng hàm split_data lên từng dòng trong RDD để lấy cột thứ

    top_3 = temperatures.top(3) # Lấy 3 phần tử lớn nhất từ RDD temperatures

    # Print the temperatures
    print("\n".join(top_3))

    # Stop spark
    spark.stop()

```

Bài 40

Đề bài yêu cầu xử lý dữ liệu PM10 từ một tập hợp các tệp CSV, trong đó mỗi dòng chứa giá trị PM10 của một cảm biến vào từng ngày. Mục tiêu là xác định số ngày có giá trị PM10 vượt quá 50 (được gọi là "critical days") cho mỗi cảm biến và sắp xếp các cảm biến theo số lượng ngày này. Kết quả sẽ được lưu trữ trong HDFS dưới dạng một tệp, với mỗi dòng chứa số ngày vượt quá 50 và ID của cảm biến tương ứng, và các cảm biến được sắp xếp giảm dần theo số lượng ngày này.

Input

Dữ liệu đầu vào là một tệp CSV chứa thông tin về các cảm biến (sensorId), ngày (date), và giá trị PM10 tương ứng

```

E:\Bigdata_Analysis\Final\Exercises\ex40>type input.csv
s1,2016-01-01,20.5
s2,2016-01-01,30.1
s1,2016-01-02,60.2
s2,2016-01-02,20.4
s1,2016-01-03,55.5
s2,2016-01-03,52.5

```

Output

Số ngày có giá trị PM10 vượt quá 50 cho mỗi cảm biến, với mỗi dòng chứa (số ngày, sensorId), sắp xếp theo số ngày giảm dần.

```
C:\hadoop-3-3-6\bin>hdfs dfs -cat /Spark/ex40/output/part-00001
(2, 's1')
(1, 's2')

C:\hadoop-3-3-6\bin>
```

Code

```
import sys
import os
from pathlib import Path
from operator import add
import findspark

from pyspark.sql import SparkSession

findspark.init()

def split_data(line):
    """
    Phân tách dữ liệu từ mỗi dòng, trả về (sensorId, 1) nếu PM10 > 50, ngược lại trả về (sensorId, 0)
    """
    data = str.split(line, ',') # Chia dữ liệu dòng theo dấu phẩy
    sensor_id = data[0] # sensorId ở cột đầu tiên
    pm10_value = float(data[2]) # Giá trị PM10 ở cột thứ ba
    return (sensor_id, 1 if pm10_value > 50 else 0) # Nếu PM10 > 50, đánh dấu là 1, ngược lại là 0

def swap_parts(line):
    """
    Hoán đổi giá trị của sensorId và số ngày để thuận tiện cho việc sắp xếp theo số ngày (critical days)
    """
    return (line[1], line[0]) # Trả về tuple (số ngày, sensorId)

if __name__ == "__main__":
    # Khởi tạo Spark session
    spark = SparkSession.builder.appName('Exercise-40').getOrCreate()

    # Đường dẫn tuyệt đối tới thư mục làm việc hiện tại
    absolute_path = Path().absolute()

    # Đường dẫn tới tệp CSV đầu vào
    input_path = os.path.join(absolute_path, "E:\Bigdata_Analysis\Final\Exercises\ex40\input.csv")

    # Lấy Spark context từ session
    sc = spark.sparkContext

    # Đọc dữ liệu từ tệp CSV
    lines = sc.textFile(input_path) # Đọc tệp dưới dạng RDD

    # Chuyển đổi dữ liệu thành (sensorId, 1) hoặc (sensorId, 0) dựa trên giá trị PM10
    lines_key_value = lines.map(split_data)
```

```

# Tính tổng số ngày PM10 > 50 cho mỗi sensorId
sensors_counter = lines_key_value.reduceByKey(add) # Tính tổng cho mỗi sensorId
sensors_counter = sensors_counter.map(swap_parts) # Hoán đổi để sắp xếp theo số ngày
sensors_counter = sensors_counter.sortByKey(ascending=False) # Sắp xếp theo số ngày giảm
dẫn

# Lưu kết quả ra HDFS
output_path = os.path.join(absolute_path, "hdfs://localhost:9000/Spark/ex40") # Đường dẫn tới
thư mục lưu kết quả
sensors_counter.saveAsTextFile(output_path) # Lưu kết quả dưới dạng tệp văn bản trên HDFS

# In kết quả ra màn hình (tùy chọn)
for sensor in sensors_counter.collect():
    print(sensor)

# Dừng Spark session
spark.stop()

```

Bài 42

Yêu cầu của bài tập thực hiện việc kết nối các câu hỏi với các câu trả lời của chúng. Cụ thể sẽ làm việc với hai tệp dữ liệu: một tệp chứa danh sách các câu hỏi, mỗi câu hỏi có định dạng QuestionId, Timestamp, TextOfTheQuestion; và một tệp chứa danh sách các câu trả lời, với định dạng AnswerId, QuestionId, Timestamp, TextOfTheAnswer. Mục tiêu của bài tập là tạo ra một tệp kết quả, trong đó mỗi dòng chứa một câu hỏi và danh sách các câu trả lời của câu hỏi đó, được nhóm lại theo QuestionId.

Input

questions.txt chứa câu hỏi với định dạng: QuestionId, Timestamp, TextOfTheQuestion. Ví dụ: Q1, 2015-01-01, What is ...?.

answers.txt chứa câu trả lời với định dạng: AnswerId, QuestionId, Timestamp, TextOfTheAnswer. Ví dụ: A1, Q1, 2015-01-02, It is ...

```

E:\Bigdata_Analysis\Final\Exercises\ex42>type questions.txt
Q1,2015-01-01,What is ..?
Q2,2015-01-03,Who invented ..
E:\Bigdata_Analysis\Final\Exercises\ex42>type answers.txt
A1,Q1,2015-01-02,It is ..
A2,Q2,2015-01-03,John Smith
A3,Q1,2015-01-05,I think it is ..

```

Output

Kết quả đầu ra nhóm câu hỏi theo QuestionId và liệt kê các câu trả lời tương ứng.

```
C:\hadoop-3-3-6\bin>hdfs dfs -ls /Spark/ex42/output
Found 2 items
-rw-r--r--   3 LG supergroup          0 2024-11-25 12:38 /Spark/ex42/output/_SUCCESS
-rw-r--r--   3 LG supergroup       106 2024-11-25 12:38 /Spark/ex42/output/part-00000

C:\hadoop-3-3-6\bin>hdfs dfs -ls /Spark/ex42/output/part-00000
-rw-r--r--   3 LG supergroup       106 2024-11-25 12:38 /Spark/ex42/output/part-00000

C:\hadoop-3-3-6\bin>hdfs dfs -cat /Spark/ex42/output/part-00000
('Q2', ([ 'Who invented ..?', ['John Smith']]))
('Q1', ([ 'What is ..?', ['It is ..', 'I think it is ..']]))
```

Code

```
import sys
import os
from pathlib import Path
from operator import add
import findspark

from pyspark.sql import SparkSession

findspark.init()

def split_func(x:str, index:list):
    """
    Phân tách mỗi dòng theo dấu phẩy và chỉ lấy các phần tử theo chỉ số trong list `index`.
    """
    data = x.split(',') # Tách các phần tử trong mỗi dòng theo dấu phẩy
    return [data[i] for i in index] # Lấy phần tử theo chỉ số chỉ định trong list `index`

if __name__ == '__main__':
    # Khởi tạo Spark session
    spark = SparkSession.builder.appName('Exercise-42').getOrCreate()

    # Lấy SparkContext từ session
    sc = spark.sparkContext

    # Đường dẫn tuyệt đối tới thư mục làm việc hiện tại
    absolute_path = Path().absolute()

    # Đường dẫn tới tệp CSV câu hỏi và câu trả lời
    questions_path = os.path.join(absolute_path,
    "E:\Bigdata_Analysis\Final\Exercises\ex42\questions.txt"
    answers_path = os.path.join(absolute_path,
    "E:\Bigdata_Analysis\Final\Exercises\ex42\answers.txt")

    # Đường dẫn để lưu kết quả đầu ra vào HDFS
    output_path = "hdfs://localhost:9000/Spark/ex42/output" # Lưu kết quả trên HDFS

    # Đọc dữ liệu từ tệp câu hỏi và câu trả lời
    questions = sc.textFile(questions_path) # Đọc câu hỏi từ tệp 'questions.txt'
    answers = sc.textFile(answers_path) # Đọc câu trả lời từ tệp 'answers.txt'

    # Phân tách câu hỏi thành (QuestionId, TextOfTheQuestion)
    questions_map = questions.map(lambda x: split_func(x, [0, 2])) # Chỉ lấy QuestionId và
    TextOfTheQuestion
    # Ví dụ:
    # (Q1, [What is ...?])
    # (Q2, [Who invented ..?])
```

```

# Phân tách câu trả lời thành (QuestionId, TextOfTheAnswer)
answers_map = answers.map(lambda x: split_func(x, [1, 3])) # Chỉ lấy QuestionId và
TextOfTheAnswer
# Ví dụ:
# (Q1, [It is ..])
# (Q2, [John Smith])
# (Q1, [I think it is ..])

# Nhóm câu hỏi theo QuestionId và chuyển giá trị thành danh sách
questions_group = questions_map.groupByKey().mapValues(list) # Group câu hỏi theo
QuestionId
# Ví dụ:
# (Q1, [What is ...?])
# (Q2, [Who invented ..?])

# Nhóm câu trả lời theo QuestionId và chuyển giá trị thành danh sách
answers_group = answers_map.groupByKey().mapValues(list) # Group câu trả lời theo
QuestionId
# Ví dụ:
# (Q1, [It is ..., I think it is ..])
# (Q2, [John Smith])

# Kết nối câu hỏi và câu trả lời qua QuestionId
result = questions_group.join(answers_group) # Kết nối câu hỏi và câu trả lời theo QuestionId
# Kết quả
# (Q1, ([What is ...?], [It is ..., I think it is ..]))
# (Q2, ([Who invented ..?], [John Smith]))

# Lưu kết quả vào HDFS (lưu ra một thư mục trong HDFS)
result.coalesce(1).saveAsTextFile(output_path) # Lưu kết quả vào HDFS, coalesce(1) để tạo 1
tệp duy nhất
# Kết quả được lưu trên HDFS tại đường dẫn '/Spark/ex42/output'

# In kết quả ra màn hình (tùy chọn)
# for question in result.collect(): # Thu thập kết quả và in ra
#     print(question) # In ra mỗi câu hỏi và danh sách câu trả lời tương ứng
# Kết quả in ra trên màn hình:
# (Q1, ([What is ...?], [It is ..., I think it is ..]))
# (Q2, ([Who invented ..?], [John Smith]))

# Dừng Spark session
spark.stop()

```

Bài 46

Đề bài yêu cầu phân tích chuỗi thời gian nhiệt độ từ các bản ghi có dấu thời gian Unix. Mỗi bản ghi gồm hai phần: dấu thời gian và nhiệt độ tương ứng. Dữ liệu được đo với khoảng cách giữa các bản ghi là 1 phút. Nhiệm vụ là tìm ra tất cả các cửa sổ thời gian 3 bản ghi liên tiếp mà trong đó nhiệt độ tăng dần qua từng bản ghi, sau đó lưu kết quả vào HDFS. Cụ thể, mỗi cửa sổ 3 bản ghi phải có nhiệt độ ở mỗi bản ghi tiếp theo cao hơn bản ghi trước đó.

Input

Một tệp chứa các bản ghi nhiệt độ, mỗi bản ghi bao gồm dấu thời gian (Unix timestamp) và nhiệt độ, cách nhau bằng dấu phẩy, với mẫu số liệu là 1 phút.

```
E:\Bigdata_Analysis\Final\Exercises\ex46>type input.txt
1451606400,12.1
1451606460,12.2
1451606520,13.5
1451606580,14.0
1451606640,14.0
1451606700,15.5
1451606760,15.0
```

Output

Lọc ra các cửa sổ gồm 3 bản ghi liên tiếp có xu hướng nhiệt độ tăng dần và lưu kết quả vào HDFS.

```
C:\hadoop-3-3-6\bin>hdfs dfs -cat /Spark/ex46/output/part-00000
1451606400,12.1,1451606460,12.2,1451606520,13.5
1451606460,12.2,1451606520,13.5,1451606580,14.0
```

Code

```
from pyspark import SparkConf, SparkContext
import os
import shutil
from pathlib import Path
import findspark
from pyspark.sql import SparkSession

# Khởi tạo Spark session
findspark.init()

if __name__ == '__main__':
    # Tạo Spark session
    spark = SparkSession.builder.appName('Exercise-46').getOrCreate()
    # Lấy SparkContext từ SparkSession
    sc = spark.sparkContext

    # Đường dẫn tuyệt đối tới thư mục làm việc hiện tại
    absolute_path = Path().absolute()
```



```

# Đường dẫn tới tệp đầu vào
input_path = os.path.join(absolute_path, "E:/Bigdata_Analysis/Final/Exercises/ex46/input.txt") #
Đảm bảo đường dẫn tới tệp input.txt đúng
# Output path lưu vào HDFS
output_path = "hdfs://localhost:9000/Spark/ex46/output" # Lưu kết quả trên HDFS

# Đọc dữ liệu đầu vào từ file
temperature = sc.textFile(input_path) # Đọc tệp 'input.txt' dưới dạng RDD
# Input:
# 1451606400,12.1
# 1451606460,12.2
# 1451606520,13.5
# 1451606580,14.0
# 1451606640,14.0
# 1451606700,15.5
# 1451606760,15.0

# Kích thước cửa sổ trượt
window_size = 3 # Chọn cửa sổ trượt có kích thước 3

def flat_map_func(x): # Tạo các cửa sổ trượt 3 bản ghi
    index = x[1] # Lấy chỉ số của dòng (index)
    return [(index - i, x[0]) for i in range(window_size) if index - i >= 0]
# Ví dụ: nếu x = ('1451606400,12.1', 0), với window_size = 3
# Trả về [(0, '1451606400,12.1'), (1, '1451606460,12.2'), (2, '1451606520,13.5')]
# Dựa trên index và kích thước cửa sổ trượt

# Thực hiện thao tác sliding window
sliding = (
    temperature.zipWithIndex() # Kết hợp mỗi dòng với chỉ số của nó
    .flatMap(flat_map_func) # Tạo các cửa sổ trượt (3 phần tử)
    .groupByKey() # Nhóm lại theo key (chỉ số của cửa sổ)
    .mapValues(list) # Chuyển giá trị thành danh sách
    .sortByKey() # Sắp xếp theo chỉ số cửa sổ
    .map(lambda x: x[1]) # Chỉ lấy giá trị (danh sách cửa sổ)
    .filter(lambda x: len(x) == window_size) # Lọc các cửa sổ có đủ 3 bản ghi
    .filter(lambda x: x[0].split(',')[1] < x[1].split(',')[1] < x[2].split(',')[1]) # Lọc các cửa sổ có
    giá trị nhiệt độ tăng dần
    .map(lambda x: ','.join(x)) # Nối các phần tử trong cửa sổ thành chuỗi
)

# Output:
# ["1451606400,12.1,1451606460,12.2,1451606520,13.5"]
# ["1451606460,12.2,1451606520,13.5,1451606580,14.0"]

# Lưu kết quả vào HDFS
sliding.coalesce(1).saveAsTextFile(output_path) # Lưu kết quả vào HDFS với 1 tệp duy nhất

# Dừng Spark session
spark.stop()

```

Bài 50

Đề bài yêu cầu xử lý tệp CSV chứa thông tin về các hồ sơ cá nhân, trong đó mỗi hồ sơ bao gồm tên, họ và tuổi. Mục tiêu là tạo ra một cột duy nhất, trong đó mỗi dòng là sự kết hợp của tên và họ của các cá nhân từ tệp đầu vào.

Input

Một tệp CSV chứa các thông tin name,surname,age của từng cá nhân.

```
E:\Bigdata_Analysis\Final\Exercises\ex50>type input.csv
name,surname,age
Paolo,Garza,42
Luca,Boccia,41
Maura,Bianchi,16
```

Output

```
C:\hadoop-3-3-6\bin>hdfs dfs -ls /Spark/ex50
Found 2 items
drwxr-xr-x   - LG supergroup          0 2024-11-25 13:29 /Spark/ex50/output_dataframe_mode
drwxr-xr-x   - LG supergroup          0 2024-11-25 13:29 /Spark/ex50/output_sql_mode

C:\hadoop-3-3-6\bin>hdfs dfs -ls /Spark/ex50/output_dataframe_mode
Found 2 items
-rw-r--r--   3 LG supergroup          0 2024-11-25 13:29 /Spark/ex50/output_dataframe_mode/_SUCCESS
-rw-r--r--   3 LG supergroup        55 2024-11-25 13:29 /Spark/ex50/output_dataframe_mode/part-00000-1de727db-31a8-4043-8504-6c0cfce1add7-c000.csv

C:\hadoop-3-3-6\bin>hdfs dfs -cat /Spark/ex50/output_dataframe_mode/part-00000-1de727db-31a8-4043-8504-6c0cfce1add7-c000.csv
name_surname
Paolo Garza
Luca Boccia
Maura Bianchi

C:\hadoop-3-3-6\bin>hdfs dfs -ls /Spark/ex50/output_sql_mode/part-00000-ec80a4b7-6c4b-41c8-aa5e-2e715601bf93-c000.csv
-rw-r--r--   3 LG supergroup        55 2024-11-25 13:29 /Spark/ex50/output_sql_mode/part-00000-ec80a4b7-6c4b-41c8-aa5e-2e715601bf93-c000.csv

C:\hadoop-3-3-6\bin>hdfs dfs -cat /Spark/ex50/output_sql_mode/part-00000-ec80a4b7-6c4b-41c8-aa5e-2e715601bf93-c000.csv
name_surname
Paolo Garza
Luca Boccia
Maura Bianchi
```

Kết quả chứa cột duy nhất name_surname kết hợp tên và họ của mỗi cá nhân từ cột

Code

```
import sys
import os
from pathlib import Path
from operator import add
import findspark
from pyspark.sql import SparkSession
from pyspark.sql.types import IntegerType, StringType
from pyspark.sql.functions import udf

# Khởi tạo Spark
findspark.init()

# Định nghĩa một UDF (User Defined Function) để kết hợp tên và họ thành một chuỗi
@udf
def parse_columns(name, surname):
    """
    Gộp tên và họ lại thành một chuỗi 'name surname'
    """
```

```

return name + " " + surname # Trả về chuỗi tên và họ kết hợp với dấu cách

def perform_analysis_dataframe(df, output_path):
    """
    Phân tích này chỉ sử dụng API DataFrame của Spark
    """
    # Tạo cột mới 'name_surname' bằng cách sử dụng hàm UDF `parse_columns`
    df_result = df.withColumn("name_surname", parse_columns(df.name, df.surname)) \
        .select("name_surname") # Chọn cột 'name_surname'
    # Ví dụ: Nếu dữ liệu đầu vào là:
    # name surname age
    # Paolo Garza 42
    # Luca Boccia 41
    # Maura Bianchi 16
    # Sau khi áp dụng UDF, ta sẽ có cột 'name_surname' chứa:
    # Paolo Garza
    # Luca Boccia
    # Maura Bianchi

    # Lưu kết quả vào một tệp CSV với chỉ một phân vùng để xuất ra một tệp duy nhất
    df_result.repartition(1).write.csv(output_path, mode="overwrite", header=True)
    # Kết quả được lưu vào output_path. Ví dụ, nếu output_path là
    "hdfs://localhost:9000/Spark/ex50/output_dataframe_mode",
    # kết quả sẽ được lưu vào HDFS tại đường dẫn đó.

def perform_analysis_sql(session, df, output_path):
    """
    Phân tích này sử dụng SQL queries và tempViews
    """
    # Đăng ký hàm UDF vào Spark session để có thể sử dụng trong SQL
    session.udf.register("name_surname_func", parse_columns)

    # Tạo một temporary view từ DataFrame, có thể truy vấn bằng SQL
    df.createOrReplaceTempView("users")

    # Thực hiện truy vấn SQL sử dụng UDF đã đăng ký
    df_2 = session.sql("SELECT name_surname_func(name, surname) AS name_surname FROM
users;")
    # Truy vấn SQL này sẽ gộp các cột name và surname lại với nhau thành cột name_surname
    # Sau khi thực thi truy vấn SQL:
    # name_surname
    # Paolo Garza
    # Luca Boccia
    # Maura Bianchi

    # Lưu kết quả vào một tệp CSV với một phân vùng
    df_2.repartition(1).write.csv(output_path, mode='overwrite', header=True)
    # Kết quả sẽ được lưu vào đường dẫn output_path trên HDFS.

    # Xóa temporary view sau khi sử dụng
    session.catalog.dropTempView("users")

if __name__ == "__main__":
    # Tạo một Spark session
    spark = SparkSession.builder.appName('Exercise-50').getOrCreate()

```

```

# Đường dẫn hiện tại (để tìm file đầu vào và đầu ra)
absolute_path = Path().absolute()

# Đường dẫn tệp đầu vào (file CSV chứa dữ liệu)
input_path = os.path.join(absolute_path, "E:/Bigdata_Analysis/Final/Exercises/ex50/input.csv")
# input.csv có nội dung như sau:
# name,surname,age
# Paolo,Garza,42
# Luca,Boccia,41
# Maura,Bianchi,16

# Đường dẫn đầu ra cho phương pháp sử dụng DataFrame API và SQL, bây giờ là trên HDFS
output_df_path = "hdfs://localhost:9000/Spark/ex50/output_dataframe_mode"
output_sql_path = "hdfs://localhost:9000/Spark/ex50/output_sql_mode"

# Đọc tệp CSV vào DataFrame
df = spark.read.csv(input_path, header=True, inferSchema=True)
# Dữ liệu sẽ được đọc và chuyển thành DataFrame có cấu trúc như sau:
# +-----+-----+---+
# | name|surname|age|
# +-----+-----+---+
# |Paolo| Garza| 42|
# | Luca|Boccia| 41|
# |Maura|Bianchi| 16|
# +-----+-----+---+

# Thực hiện phân tích sử dụng DataFrame API
perform_analysis_dataframe(df, output_df_path)

# Thực hiện phân tích sử dụng SQL
perform_analysis_sql(spark, df, output_sql_path)

# Dừng Spark session
spark.stop()

```

Phần 4

Câu 39

Input:

- Một tệp CSV văn bản chứa giá trị hàng ngày của PM10 cho một tập hợp các cảm biến
 - Mỗi dòng của tệp có định dạng như sau: sensorId,date,PM10 value ($\mu\text{g}/\text{m}^3$)n

Output:

- Một file lưu trữ trong HDFS chứa một dòng cho mỗi cảm biến.
 - Mỗi dòng chứa một sensorId và danh sách các ngày có giá trị PM10 lớn hơn 50 cho cảm biến đó.

```

import os
from pathlib import Path
from pyspark.sql import SparkSession
def split_data(line):
    """
    Chia mỗi dòng thành các cột dựa trên dấu phẩy và chỉ lấy 2 cột đầu tiên (sensorId, date)
    """
    data = str.split(line, ',')
    return [data[0], data[1]]

def filter_by_threshold(line):
    """
    Lọc các dòng có giá trị PM10 lớn hơn 50
    """
    temp = float(str.split(line, ',')[2])
    return temp > 50

if __name__ == "__main__":

    # Khởi tạo một phiên bản của SparkSession, là điểm khởi đầu để làm việc
    # với Spark SQL và DataFrame API trong Spark.
    spark = SparkSession.builder.appName('Exercise-39').getOrCreate()

    # Lấy đường dẫn tuyệt đối của thư mục làm việc hiện tại sử dụng class Path từ thư viện pathlib.
    absolute_path = Path().absolute()

    # Tạo đường dẫn đầu vào bằng cách kết hợp đường dẫn tuyệt đối với tên tệp tin đầu vào.
    input_path = os.path.join(absolute_path, 'input.csv')

    # Lấy Spark context từ Spark session.
    # Spark context (sc) là điểm khởi đầu cho các thao tác Spark ở mức thấp.
    sc = spark.sparkContext

    # Phương thức textFile đọc một tệp văn bản từ đường dẫn được chỉ định
    # và trả về một RDD của các chuỗi, mỗi chuỗi là một dòng từ tệp.
    lines = sc.textFile(input_path)

    # Lọc tất cả các dòng có giá trị PM10 lớn hơn 50 và trả về (sensorId, date)
    # Phương thức filter trả về một RDD mới chỉ chứa các phần tử thỏa mãn hàm điều kiện
    (filter_by_threshold).
    # Phương thức map áp dụng hàm split_data cho mỗi phần tử của RDD và trả về một RDD mới
    với các kết quả.
    lines_above = lines.filter(filter_by_threshold).map(split_data)

    # Nhóm các bản ghi đã lọc và ánh xạ theo các khóa của chúng (sensor ID)
    # bằng phương thức groupByKey. Phương thức mapValues sau đó được sử dụng
    # để chuyển đổi các giá trị đã nhóm thành các danh sách. Cuối cùng,
    # phương thức collect được gọi để lấy toàn bộ RDD dưới dạng một danh sách các tuple về
    chương trình điều khiển.
    # Mỗi tuple chứa một khóa (sensor ID) và một danh sách các giá trị (ngày).
    lines_above_by_key = lines_above.groupByKey().mapValues(list)

    # Hợp nhất các phân vùng thành một và lưu kết quả vào HDFS
    lines_above_by_key.coalesce(1).saveAsTextFile('hdfs://localhost:9000/exercise-39')

```

```
# In kết quả ra màn hình chuẩn
print(lines_above_by_key.collect())

# Dừng spark
spark.stop()
```

```
org.apache.hadoop#hadoop-client-runtime;3.3.4 from central in [default]
org.apache.kafka#kafka-clients;3.4.1 from central in [default]
org.apache.spark#spark-sql-kafka-0-10_2.12;3.5.1 from central in [default]
org.apache.spark#spark-token-provider-kafka-0-10_2.12;3.5.1 from central in [default]
org.lz4#lz4-java;1.8.0 from central in [default]
org.slf4j#slf4j-api;2.0.7 from central in [default]
org.xerial.snappy#snappy-java;1.1.10.3 from central in [default]

-----
|               | modules | artifacts |
| conf | number | search | dwnlded | evicted | number | dwnlded |
|-----|-----|-----|-----|-----|-----|-----|
| default | 11 | 0 | 0 | 0 | 11 | 0 |

:: retrieving :: org.apache.spark#spark-submit-parent-46e19672-3225-497b-ae86-4cc88bca40f3
confs: [default]
0 artifacts copied, 11 already retrieved (0kB/26ms)
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
[('s1', ['2016-01-02', '2016-01-03']), ('s2', ['2016-01-03'])]
PS D:\OneDrive-QuangNgai\BigData_Analysis\spark-exercise\Ques_39> SUCCESS: The process with PID 12596 (child process of PID 12620) has been terminated.
SUCCESS: The process with PID 12620 (child process of PID 5836) has been terminated.
SUCCESS: The process with PID 5836 (child process of PID 11520) has been terminated.
[]
```

```
D:\Workspace\hadoop\hadoop-3.3.6\bin>hdfs dfs -ls /
Found 1 items
drwxr-xr-x - Admin supergroup 0 2024-11-17 11:32 /exercise-39

D:\Workspace\hadoop\hadoop-3.3.6\bin>hdfs dfs -ls /exercise-39
Found 2 items
-rw-r--r-- 3 Admin supergroup 0 2024-11-17 11:32 /exercise-39/_SUCCESS
-rw-r--r-- 3 Admin supergroup 60 2024-11-17 11:32 /exercise-39/part-00000

D:\Workspace\hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /exercise-39/part-00000
('s1', ['2016-01-02', '2016-01-03'])
('s2', ['2016-01-03'])

D:\Workspace\hadoop\hadoop-3.3.6\bin>
```

Câu 39 - Critical dates analysis - bis

Input:

- Một tệp CSV văn bản chứa giá trị hàng ngày của PM10 cho một tập hợp các cảm biến
 - Mỗi dòng của tệp có định dạng như sau: sensorId,date,PM10 value ($\mu\text{g}/\text{m}^3$)
n

Output:

- Một file lưu trữ trong HDFS chứa một dòng cho mỗi cảm biến.
 - Mỗi dòng chứa một sensorId và danh sách các ngày có giá trị PM10 lớn hơn 50 cho cảm biến đó.
 - Các cảm biến không bao giờ có giá trị PM10 lớn hơn 50 cũng phải được bao gồm trong kết quả (với một tập hợp rỗng).

```

import os
from pathlib import Path
from pyspark.sql import SparkSession
def split_data(line):
    """
    Returns sensorId,date of readings with temperature above 50
    """
    data = str.split(line, ',')
    key = data[0]
    value = data[1] if filter_by_threshold(float(data[2])) else None
    return [key, value]

def filter_by_threshold(temperature):
    """
    Just a filter
    """
    return temperature > 50

def map_values_func(data_list):
    """
    Removes all None values from the list
    Loại bỏ tất cả các giá trị None khỏi danh sách
    """
    data = list(data_list)
    return [x for x in data if x != None]

if __name__ == "__main__":

    # Tạo một phiên bản của SparkSession, là điểm khởi đầu để làm việc
    # với Spark SQL và DataFrame API trong Spark.
    spark = SparkSession.builder.appName('Exercise-39-bis').getOrCreate()

    # Lấy đường dẫn tuyệt đối của thư mục làm việc hiện tại sử dụng class Path từ thư viện pathlib.
    absolute_path = Path().absolute()

    # Tạo đường dẫn đầu vào bằng cách kết hợp đường dẫn tuyệt đối với tên tệp tin đầu vào.
    input_path = os.path.join(absolute_path, 'input.csv')

    # Lấy Spark context từ Spark session.
    # Spark context (sc) là điểm khởi đầu cho các thao tác Spark ở mức thấp.
    sc = spark.sparkContext

    # Phương thức textFile đọc một tệp văn bản từ đường dẫn được chỉ định
    # và trả về một RDD của các chuỗi, mỗi chuỗi là một dòng từ tệp.
    lines = sc.textFile(input_path)

    # Ánh xạ mỗi dòng dữ liệu thành (sensorId, date) nếu giá trị PM10 lớn hơn 50
    lines_key_value = lines.map(split_data)

    # Nhóm các bản ghi theo sensorId và loại bỏ các giá trị None, sau đó sắp xếp theo sensorId
    lines_above_by_key =
lines_key_value.groupByKey().mapValues(map_values_func).sortByKey()

    # Hợp nhất các phân vùng thành một và lưu kết quả vào HDFS
    lines_above_by_key.coalesce(1).saveAsTextFile('hdfs://localhost:9000/exercise-39-bis')

```

```
# In kết quả ra màn hình chuẩn
print(lines_above_by_key.collect())
```

```
# Dừng Spark
spark.stop()
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER COMMENTS
:: resolution report :: resolve 376ms :: artifacts d1 90ms
:: modules in use:
com.google.code.findbugs#jsr305;3.0.0 from central in [default]
commons-logging#commons-logging;1.1.3 from central in [default]
org.apache.commons#commons-pool2;2.11.1 from central in [default]
org.apache.hadoop#hadoop-client-api;3.3.4 from central in [default]
org.apache.hadoop#hadoop-client-runtime;3.3.4 from central in [default]
org.apache.kafka#kafka-clients;3.4.1 from central in [default]
org.apache.spark#spark-sql-kafka-0-10_2.12;3.5.1 from central in [default]
org.apache.spark#spark-token-provider-kafka-0-10_2.12;3.5.1 from central in [default]
org.lz4#lz4-java;1.8.0 from central in [default]
org.slf4j#slf4j-api;2.0.7 from central in [default]
org.xerial.snappy#snappy-java;1.1.10.3 from central in [default]

-----
| conf | modules | artifacts | | | | |
|---|---|---|---|---|---|---|
|      | number | search | dwld | evicted | number | dwld |
|-----|-----|-----|
| default | 11 | 0 | 0 | 0 | 11 | 0 |
-----

:: retrieving :: org.apache.spark#spark-submit-parent-096e0115-4266-4cb1-a838-7ffa6c5e612
conf: [default]
0 artifacts copied, 11 already retrieved (0kB/23ms)
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
[('s1', ['2016-01-02', '2016-01-03']), ('s2', ['2016-01-03']), ('s3', [])]
PS D:\OneDrive-QuangNgai\BigData_Analysis\spark-exercise\Ques_39_bis> SUCCESS: The process with PID 9696 (child process of PID 9816) has been terminated.
SUCCESS: The process with PID 9816 (child process of PID 6184) has been terminated.
SUCCESS: The process with PID 6184 (child process of PID 1248) has been terminated.
[]
```

```
D:\Workspace\hadoop\hadoop-3.3.6\bin>hdfs dfs -ls /
Found 2 items
drwxr-xr-x - Admin supergroup 0 2024-11-17 11:32 /exercise-39
drwxr-xr-x - Admin supergroup 0 2024-11-17 11:47 /exercise-39-bis

D:\Workspace\hadoop\hadoop-3.3.6\bin>hdfs dfs -ls /exercise-39-bis
Found 2 items
-rw-r--r-- 3 Admin supergroup 0 2024-11-17 11:47 /exercise-39-bis/_SUCCESS
-rw-r--r-- 3 Admin supergroup 71 2024-11-17 11:47 /exercise-39-bis/part-00000

D:\Workspace\hadoop\hadoop-3.3.6\bin>hdfs dfs -cat /exercise-39-bis/part-00000
('s1', ['2016-01-02', '2016-01-03'])
('s2', ['2016-01-03'])
('s3', [])

D:\Workspace\hadoop\hadoop-3.3.6\bin>
```

Câu 41- Top-k most critical sensors

Input:

- Một tệp CSV văn bản chứa giá trị hàng ngày của PM10 cho một tập hợp các cảm biến
 - Mỗi dòng của tệp có định dạng như sau: sensorId,date,PM10 value ($\mu\text{g}/\text{m}^3$)
n
- Giá trị của k
 - Đây là tham số đầu vào của ứng dụng

Output:

- Một file lưu trong HDFS chứa danh sách top-k cảm biến quan trọng
 - Một cảm biến được coi là quan trọng nếu nó có nhiều ngày mà giá trị PM10 đo được lớn hơn 50.
 - Mỗi dòng trong kết quả đầu ra sẽ chứa: Số ngày quan trọng (số ngày có giá trị PM10 lớn hơn 50) và sensorId.

```
import sys
import os
from pathlib import Path
from operator import add

from pyspark.sql import SparkSession

def check_argv_value(vals):
    if len(vals) <= 1:
        print("ERR: Please provide a valid value for K", file=sys.stderr)
        exit(-1)
    try:
        k = int(vals[1])
    except ValueError:
        print("ERR: {} is not a valid value for K".format(vals[1]), file=sys.stderr)
        exit(-1)
    return k

def split_data(line):
    """
    Trả về sensorId,1 của các bản ghi có PM10 lớn hơn 50
    """
    data = str.split(line, ',')
    key = data[0]
    temperature = float(data[2])
    return [key, 1 if temperature > 50 else 0]

def swap_parts(line):
    """
    Đổi chỗ key và value và trả về
    """
    return (line[1], line[0])

if __name__ == "__main__":
    # Kiểm tra giá trị đầu vào k từ danh sách các tham số dòng lệnh và kiểm tra xem nó có hợp lệ
    # hay không
    # Nếu không hợp lệ, in thông báo lỗi và thoát chương trình. Ngược lại, trả về giá trị k.
    k = check_argv_value(sys.argv)

    # Tạo một phiên bản của SparkSession, là điểm khởi đầu để làm việc
    # với Spark SQL và DataFrame API trong Spark.
    spark = SparkSession.builder.appName('Exercise-41').getOrCreate()

    # Lấy đường dẫn tuyệt đối của thư mục làm việc hiện tại sử dụng class Path từ thư viện pathlib.
    absolute_path = Path().absolute()

    # Tạo đường dẫn đầu vào bằng cách kết hợp đường dẫn tuyệt đối với tên tệp tin đầu vào.
```

```

input_path = os.path.join(absolute_path, 'input.csv')

# Lấy Spark context từ Spark session.
# Spark context (sc) là điểm khởi đầu cho các thao tác Spark ở mức thấp.
sc = spark.sparkContext

# Phương thức readFile đọc một tệp văn bản từ đường dẫn được chỉ định
# và trả về một RDD của các chuỗi, mỗi chuỗi là một dòng từ tệp.
lines = sc.readFile(input_path)

# Ánh xạ mỗi dòng dữ liệu thành (sensorId, 1) nếu giá trị PM10 lớn hơn 50
lines_key_value = lines.map(split_data)

# Đếm số lượng các bản ghi có giá trị PM10 lớn hơn 50 cho mỗi sensorId
sensors_counter =
lines_key_value.reduceByKey(add).map(swap_parts).sortByKey(ascending=False)

# Lấy top k cảm biến quan trọng nhất
top_k_results = sensors_counter.take(k)

# Chuyển top k kết quả thành RDD
top_k_rdd = sc.parallelize(top_k_results)

# Hợp nhất phân vùng và lưu kết quả vào thư mục output
top_k_rdd.coalesce(1).saveAsTextFile('hdfs://localhost:9000/exercise-41')

# In kết quả ra màn hình chuẩn
print(sensors_counter.collect()[:k])

# Dừng Spark
spark.stop()

```

Output chương trình với k=1

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  JUPYTER  COMMENTS
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
[[2, 's1']]
PS D:\OneDrive-QuangNgai\BigData_Analysis\spark-exercise\Ques_41> SUCCESS: The process with PID 12992 (child process of PID 9852) has been terminated.

SUCCESS: The process with PID 9852 (child process of PID 13092) has been terminated.
SUCCESS: The process with PID 13092 (child process of PID 19296) has been terminated.

```

```

D:\Workspace\hadoop\hadoop-3.3.6\sbin>hdfs dfs -ls /
Found 3 items
drwxr-xr-x   - Admin supergroup            0 2024-11-17 11:32 /exercise-39
drwxr-xr-x   - Admin supergroup            0 2024-11-17 11:47 /exercise-39-bis
drwxr-xr-x   - Admin supergroup            0 2024-11-17 16:06 /exercise-41

D:\Workspace\hadoop\hadoop-3.3.6\sbin>hdfs dfs -ls /exercise-41
Found 2 items
-rw-r--r--   3 Admin supergroup            0 2024-11-17 16:06 /exercise-41/_SUCCESS
-rw-r--r--   3 Admin supergroup           10 2024-11-17 16:06 /exercise-41/part-00000

D:\Workspace\hadoop\hadoop-3.3.6\sbin>hdfs dfs -cat /exercise-41/part-00000
(2, 's1')

D:\Workspace\hadoop\hadoop-3.3.6\sbin>

```

Câu 45 - Profile update

Input:

- Tập văn bản chứa danh sách các bộ phim đã được xem bởi người dùng của dịch vụ video theo yêu cầu
 - Mỗi dòng của tập chứa thông tin về một lần xem phim: userid,movieid,start-timestamp,end-timestamp
 - Người dùng có id userid đã xem bộ phim có id movieid từ thời điểm start-timestamp đến thời điểm end-timestamp.
- Tập văn bản thứ hai chứa danh sách các sở thích của mỗi người dùng
 - Mỗi dòng của tập chứa thông tin về một sở thích: userid,movie-genre
 - Người dùng có id userid thích thể loại phim movie-genre.
- Tập văn bản thứ ba chứa danh sách các bộ phim kèm theo thông tin liên quan
 - Mỗi dòng của tập chứa thông tin về một bộ phim: movieid,title,movie-genre
 - Chỉ có một dòng cho mỗi bộ phim, tức là mỗi bộ phim chỉ có một thể loại duy nhất.

Output:

- Chọn cho mỗi người dùng có hồ sơ gây hiểu lầm (theo định nghĩa của Bài tập #44 - người dùng có hồ sơ gây hiểu lầm nếu vượt quá ngưỡng % trong số những bộ phim anh ấy/cô ấy đã xem không liên quan đến một thể loại phim anh ấy/cô ấy thích) danh sách các thể loại phim không nằm trong sở thích của họ và được liên kết với ít nhất 2 bộ phim mà người dùng đã xem
- Lưu kết quả vào một tập HDFS
 - Mỗi dòng của tập đầu ra được liên kết với một cặp (người dùng, thể loại gây hiểu lầm được chọn) liên quan đến họ.
 - Định dạng là userid, selected (misleading) genre
 - Người dùng liên kết với một danh sách các thể loại được chọn sẽ được liên kết với nhiều dòng của tập đầu ra.

```
from pyspark.sql import SparkSession
import os
from pathlib import Path

def filter_misleading_users(listWatchedLikedGenres, threshold):
```

```

likedGenres = list(listWatchedLikedGenres[1][1])
numWatchedMovies = 0
notLiked = 0

for watchedGenre in listWatchedLikedGenres[1][0]:
    numWatchedMovies += 1
    if watchedGenre not in likedGenres:
        notLiked += 1

return (float(notLiked) > threshold * float(numWatchedMovies))

def select_misleading_genres(listWatchedLikedGenres):
    likedGenres = list(listWatchedLikedGenres[1])
    numGenres = {}

    for watchedGenre in listWatchedLikedGenres[0]:
        if watchedGenre not in likedGenres:
            if watchedGenre in numGenres:
                numGenres[watchedGenre] += 1
            else:
                numGenres[watchedGenre] = 1

    selectedGenres = [genre for genre in numGenres if numGenres[genre] >= 2]

    return selectedGenres

if __name__ == "__main__":

    # Khởi tạo SparkSession
    spark = SparkSession.builder.appName("Exercise-45").getOrCreate()

    # Lấy đường dẫn tuyệt đối của thư mục hiện tại
    absolute_path = Path().absolute()

    # Tạo các đường dẫn đầu vào và đầu ra
    inputPathWatched = os.path.join(absolute_path, 'watchedmovies.txt')
    inputPathPreferences = os.path.join(absolute_path, 'preferences.txt')
    inputPathMovies = os.path.join(absolute_path, 'movies.txt')
    outputPath = 'hdfs://localhost:9000/exercise-45'
    threshold = 0.3

    # Lấy SparkContext từ SparkSession
    sc = spark.sparkContext

    # Đọc file phim đã xem - watched movies và tạo RDD
    watchedRDD = sc.textFile(inputPathWatched)
    # Tạo cặp (movieID, userID) từ file watchedmovies
    movieUserPairRDD = watchedRDD.map(lambda line: (line.split(",")[1], line.split(",")[0]))
    """
        [(movie1, user1),
         (movie3, user1),
         (movie4, user1),
         (movie5, user1),
         (movie6, user2),
         (movie3, user2),
         (movie4, user2)]
    """

```

```

# Đọc file phim - movies và tạo RDD
moviesRDD = sc.textFile(inputPathMovies)
# Tạo cặp (movieID, genre) từ file movies
movieGenrePairRDD = moviesRDD.map(lambda line: (line.split(",")[0], line.split(",")[2]))
"""
    [(movie1, Animation),
     (movie2, Adventure),
     (movie3, Comedy),
     (movie4, Comedy),
     (movie5, Comedy),
     (movie6, Action),
     (movie7, Comedy),
     (movie8, Adventure),
     (movie9, Action),
     (movie10, Action)]
"""

# Join phim đã xem với phim
joinWatchedGenreRDD = movieUserPairRDD.join(movieGenrePairRDD)
"""
[
  (movie1, (user1, Animation)),
  (movie3, (user1, Comedy)),
  (movie3, (user2, Comedy)),
  (movie4, (user1, Comedy)),
  (movie4, (user2, Comedy)),
  (movie5, (user1, Comedy)),
  (movie6, (user2, Action))
]
"""

# Chọn chỉ userid (là key) và genre (là value)
usersWatchedGenresRDD = joinWatchedGenreRDD.map(lambda userMovie: (userMovie[1][0],
userMovie[1][1]))
"""
    (user1, Animation),
    (user1, Comedy),
    (user2, Comedy),
    (user1, Comedy),
    (user2, Comedy),
    (user1, Comedy),
    (user2, Action)
"""

# Đọc file preferences và tạo RDD
preferencesRDD = sc.textFile(inputPathPreferences)
userLikedGenresRDD = preferencesRDD.map(lambda line: (line.split(",")[0], line.split(",")[1]))
"""
[
  (user1, Animation),
  (user1, Comedy),
  (user2, Action)
]
"""

```

```

# Cogroup danh sách các thể loại đã xem và thích cho mỗi người dùng
# Key là userid, Value chứa danh sách các thể loại phim (có thể lặp lại) của các bộ phim đã xem
# và danh sách các thể loại phim yêu thích.
userWatchedLikedGenres = usersWatchedGenresRDD.cogroup(userLikedGenresRDD)
"""
[
  (user1, ([Animation, Comedy, Comedy, Comedy], [Animation, Comedy])),
  (user2, ([Comedy, Comedy, Action], [Action]))
]
"""

# Lọc những người dùng có hồ sơ gây hiểu lầm (số lượng phim đã xem liên quan đến thể loại
không được thích có lớn hơn % ngưỡng tổng số phim đã xem hay không.)
misleadingUsersListsRDD = userWatchedLikedGenres.filter(lambda listWatchedLikedGenres:
filter_misleading_users(listWatchedLikedGenres, threshold))
"""
[
  (user2, ([Comedy, Comedy, Action], [Action]))
]
"""

# Chọn ra thể loại phim mà không nằm trong thể loại yêu thích của người dùng
# Xuất hiện ít nhất 2 lần trong danh sách các thể loại phim mà người dùng đã xem
# Do dùng flatMapValues nên mỗi phần tử trong selectedGenres sẽ được kết hợp với user2
misleadingUserGenrePairRDD = misleadingUsersListsRDD.flatMapValues(lambda
listWatchedLikedGenres: select_misleading_genres(listWatchedLikedGenres))
"""
[
  (user2, Comedy)
]
"""

# Lưu kết quả ra file
misleadingUserGenrePairRDD.coalesce(1).saveAsTextFile(outputPath)

sc.stop()

```

Output với ngưỡng là 0.3

```

python code.py
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
PS D:\OneDrive-QuangNgai\BigData_Analysis\spark-exercise\Ques_45> SUCCESS: The process with PID 18396 (child process of PID 6380) has
been terminated.
SUCCESS: The process with PID 6380 (child process of PID 3328) has been terminated.
SUCCESS: The process with PID 3328 (child process of PID 14904) has been terminated.
□

```

```
D:\Workspace\hadoop\hadoop-3.3.6\sbin>hdfs dfs -ls /
Found 5 items
drwxr-xr-x - Admin supergroup          0 2024-11-24 10:44 /exercise-35
drwxr-xr-x - Admin supergroup          0 2024-11-17 11:32 /exercise-39
drwxr-xr-x - Admin supergroup          0 2024-11-17 11:47 /exercise-39-bis
drwxr-xr-x - Admin supergroup          0 2024-11-17 16:06 /exercise-41
drwxr-xr-x - Admin supergroup          0 2024-11-24 17:56 /exercise-45

D:\Workspace\hadoop\hadoop-3.3.6\sbin>hdfs dfs -ls /exercise-45
Found 2 items
-rw-r--r--  3 Admin supergroup          0 2024-11-24 17:56 /exercise-45/_SUCCESS
-rw-r--r--  3 Admin supergroup        20 2024-11-24 17:56 /exercise-45/part-00000

D:\Workspace\hadoop\hadoop-3.3.6\sbin>hdfs dfs -cat /exercise-45/part-00000
('user2', 'Comedy')

D:\Workspace\hadoop\hadoop-3.3.6\sbin>
```

Câu 49

Input:

- Một tệp CSV chứa danh sách các hồ sơ người dùng
 - Tiêu đề: name,age,gender
 - Mỗi dòng của tệp chứa thông tin về một người dùng

Output:

- Một tệp CSV chứa một dòng cho mỗi hồ sơ. Thuộc tính age ban đầu được thay thế bằng một thuộc tính mới gọi là rangeage có kiểu chuỗi.
 - $\text{rangeage} = "[" + (\text{age}/10)*10 + "-" + (\text{age}/10)*10+9 + "]"$

```
import os
from pathlib import Path

from pyspark.sql import SparkSession
# Thư viện pyspark.sql.functions cung cấp các hàm để làm việc với DataFrame.
# udf là viết tắt của User Defined Function,
# cho phép bạn định nghĩa các hàm tùy chỉnh để sử dụng trong DataFrame.
from pyspark.sql.functions import udf

@udf
def parse_age(age):
    # Hàm này nhận vào một giá trị age và trả về một chuỗi biểu thị khoảng tuổi.
    return "[{}-{}]".format(int(age/10)*10, int(age/10)*10+9)

def perform_analysis_dataframe(df, output_path):
    """
    Thực hiện phân tích này chỉ bằng cách sử dụng DataFrame APIs
    """
    # Thêm một cột mới rangeage vào DataFrame df bằng cách áp dụng hàm parse_age lên cột age.
    # Sau đó chọn các cột name, surname, rangeage, hợp nhất các phân vùng và lưu kết quả vào
    # đường dẫn output_path.
    df_result = df.withColumn("rangeage", parse_age(df.age)).select("name", "surname",
"rangeage")
```

```

df_result.repartition(1).write.csv(output_path, mode="overwrite", header=True)

def perform_analysis_sql(session, df, output_path):
    """
    Thực hiện phân tích này bằng cách sử dụng các truy vấn SQL và tempViews
    """

    # Đăng ký hàm parse_age vào session
    session.udf.register("range_age_func", parse_age)

    # Tạo một view tạm thời liên kết với dữ liệu đầu vào
    df.createOrReplaceTempView("users")

    # Submit SQL queries on the view
    # Tạo truy vấn sql chọn các cột name, surname, range_age_func(age),
    # hợp nhất các phân vùng và lưu kết quả vào đường dẫn output_path.
    df_2 = session.sql("select name, surname, range_age_func(age) as rangeage FROM users;")
    df_2.repartition(1).write.csv(output_sql_path, mode='overwrite', header=True)

    # Xóa view tạm thời
    session.catalog.dropTempView("users")

if __name__ == "__main__":

    # Tạo một phiên bản của SparkSession, là điểm khởi đầu để làm việc
    # với Spark SQL và DataFrame API trong Spark.
    spark = SparkSession.builder.appName('Exercise-49').getOrCreate()

    # Lấy đường dẫn tuyệt đối của thư mục làm việc hiện tại sử dụng class Path từ thư viện pathlib.
    absolute_path = Path().absolute()

    # Tạo đường dẫn đầu vào bằng cách kết hợp đường dẫn tuyệt đối với tên tệp tin đầu vào.
    input_path = os.path.join(absolute_path, 'input.csv')

    # Tạo đường dẫn đầu ra bằng cách kết hợp đường dẫn tuyệt đối với tên thư mục đầu ra.
    output_df_path = os.path.join(absolute_path, 'result_dataframe_mode')
    output_sql_path = os.path.join(absolute_path, 'result_sql_mode')

    # Đọc dữ liệu từ tệp tin csv vào DataFrame df bằng phương thức read.
    df = spark.read.csv(input_path, header=True, inferSchema=True)

    # Thực hiện phân tích bằng cách sử dụng DataFrame
    perform_analysis_dataframe(df, output_df_path)

    # Thực hiện phân tích bằng cách sử dụng SQL
    perform_analysis_sql(spark, df, output_sql_path)

    # Dừng phiên làm việc với Spark.
    spark.stop()

```


Ques_49

> __pycache__

result_dataframe_mode

._SUCCESS

._SUCCESS.crc

.part-00000-c0de8e0a-f2a0-4f30-a9bb-edebc7a6308-c000.csv.crc

part-00000-c0de8e0a-f2a0-4f30-a9bb-edebc7a6308-c000.csv

result_sql_mode

._SUCCESS

._SUCCESS.crc

.part-00000-d9fc3f42-c0b0-4ae3-8e8f-b8e5560be517-c000.csv.crc

part-00000-d9fc3f42-c0b0-4ae3-8e8f-b8e5560be517-c000.csv

code.py

input.csv

Ques_49.ipynb

README.md

> OUTLINE

> TIMELINE

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

JUPYTER

COMMENTS

SUCCESS: The process with PID 9088 (child process of PID 14256) has been terminated.

SUCCESS: The process with PID 14256 (child process of PID 10872) has been terminated.

SUCCESS: The process with PID 10872 (child process of PID 9448) has been terminated.

PS D:\OneDrive-Quang\gai\BigData Analysis\spark-exercise\Ques_49>

code.py

part-00000-c0de8e0a-f2a0-4f30-a9bb-edebc7a6308-c000.csv X

Ques_49 > result_dataframe_mode > part-00000-c0de8e0a-f2a0-4f30-a9bb-edebc7a6308-c000.csv

1 name,surname,rangeage

2 Paolo,Garza,[40-49]

3 Luca,Boccia,[40-49]

4 Maura,Bianchi,[10-19]

5

code.py

part-00000-d9fc3f42-c0b0-4ae3-8e8f-b8e5560be517-c000.csv X

Ques_49 > result_sql_mode > part-00000-d9fc3f42-c0b0-4ae3-8e8f-b8e5560be517-c000.csv

1 name,surname,rangeage

2 Paolo,Garza,[40-49]

3 Luca,Boccia,[40-49]

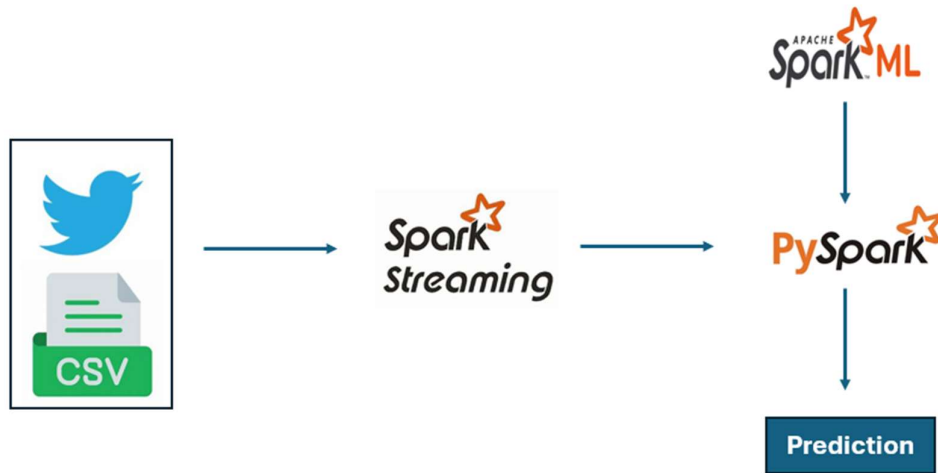
4 Maura,Bianchi,[10-19]

5

CHƯƠNG 4: ỨNG DỤNG SPARK ĐỂ XÂY DỰNG MÔ HÌNH DỰ ĐOÁN CẢM XÚC

4.1 Giới thiệu bài toán

Dự án này liên quan đến việc sử dụng PySpark, SparkML để xây dựng mô hình phân tích Sentiment Analysis trên dữ liệu Twitter.



Cụ thể:

1. Dữ liệu huấn luyện: `twitter_training.csv` chứa 74,682 dòng, mỗi dòng tương ứng với một tweet.
2. Dữ liệu kiểm tra: `twitter_validation.csv` chứa 998 dòng với cấu trúc tương tự.
3. Yêu cầu chính:
 - Xây dựng mô hình học máy sử dụng PySpark.
 - Tích hợp Kafka để xử lý các tweet trực tiếp (live tweets).

1. Phân tích dữ liệu

Dữ liệu gồm các cột:

2. Tweet ID (int): ID duy nhất của mỗi tweet.
3. Entity (string): Một thực thể liên quan đến nội dung của tweet (vd: một tổ chức, một địa điểm, hay một cá nhân).
4. Sentiment (string - Target): Nhãn cảm xúc của tweet, thường là Positive, Negative, hoặc Neutral.
5. Tweet content (string): Nội dung đầy đủ của tweet.

6. Yêu cầu xử lý

1. Tiền xử lý dữ liệu:

- Làm sạch dữ liệu:
 - Loại bỏ các ký tự không cần thiết (URLs, hashtags, mentions).
 - Xử lý dữ liệu trống hoặc bị lỗi.
 - Tokenization: Chia nhỏ câu thành các từ (tokens).
 - Loại bỏ stopwords: Loại bỏ các từ không mang nhiều ý nghĩa (vd: "the", "is").
 - Stemming/Lemmatization: Chuẩn hóa từ về gốc.
2. Phân tích dữ liệu:
- Kiểm tra phân phối của cột Sentiment (vd: số lượng tweet Positive/Negative/Neutral).
 - Hiển thị các từ phổ biến nhất trong các nhóm Sentiment khác nhau.
3. Xây dựng mô hình:
- Sử dụng PySpark MLlib để xây dựng pipeline học máy:
 - Biến đổi dữ liệu văn bản thành đặc trưng số (sử dụng TF-IDF hoặc Word2Vec).
 - Chọn mô hình phù hợp, ví dụ: Logistic Regression, Naive Bayes, hoặc Random Forest.
 - Huấn luyện mô hình trên tập `twitter_training.csv`.
 - Đánh giá mô hình trên tập kiểm tra `twitter_validation.csv`.

4.2 Xử lý và làm sạch dữ liệu

4.2.1 Xử lý dữ liệu

```
nltk.download('stopwords')
# define English stopwords
stop_words = stopwords.words('english')
conf = SparkConf()
spark = SparkSession.builder.appName("Tweets Sentiment with in PySpark").getOrCreate()
```

Phần này hai nhiệm vụ chính trong một pipeline xử lý dữ liệu văn bản với Spark và NLP

nltk.download('stopwords'): Tải danh sách các từ dừng (stopwords) tiếng Anh từ thư viện nltk. Các từ dừng là các từ phổ biến như "and", "is", "the", v.v., thường không mang nhiều ý nghĩa trong việc phân tích văn bản.

stop_words: Lưu danh sách các từ dừng tiếng Anh vào biến *stop_words*.

SparkConf() và *setMaster*: Cấu hình Spark, chỉ định master node là địa chỉ cụ thể `spark://192.168.1.91:7077`.

Tạo *SparkSession* và đọc dữ liệu

```
train_set = spark.read.csv('twitter_training.csv', header = False, inferSchema = True)
val_set = spark.read.csv('twitter_validation.csv', header = False, inferSchema = True)
```

SparkSession.builder: Khởi tạo phiên Spark với tên ứng dụng "Tweets Sentiment with in PySpark".

read.csv: Đọc dữ liệu từ file CSV.

header=False: Tập tin không có dòng tiêu đề.

inferSchema=True: Tự động suy luận kiểu dữ liệu của các cột

Hai tập dữ liệu:

train_set: Tập huấn luyện.

val_set: Tập kiểm tra.

Hiển thị cấu trúc và nội dung tập dữ liệu huấn luyện

```
train_set.printSchema()
train_set.show(5, False)
```

printSchema(): Hiển thị cấu trúc dữ liệu (kiểu dữ liệu của từng cột).

show(5, False): Hiển thị 5 dòng dữ liệu đầu tiên, không cắt chuỗi (False).

```

root
|-- _c0: integer (nullable = true)
|-- _c1: string (nullable = true)
|-- _c2: string (nullable = true)
|-- _c3: string (nullable = true)

+---+-----+-----+-----+
|_c0|_c1      |_c2      |_c3      |
+---+-----+-----+-----+
|2401|Borderlands|Positive|im getting on borderlands and i will murder you all ,|
|2401|Borderlands|Positive|I am coming to the borders and I will kill you all,|
|2401|Borderlands|Positive|im getting on borderlands and i will kill you all,|
|2401|Borderlands|Positive|im coming on borderlands and i will murder you all,|
|2401|Borderlands|Positive|im getting on borderlands 2 and i will murder you me all,|
+---+-----+-----+-----+
only showing top 5 rows

```

Đổi tên cột

```

columns = ['id', 'company', 'label', 'text']
for i, col_name in enumerate(columns):
    train_set = train_set.withColumnRenamed('_c{}'.format(i), col_name)
    val_set = val_set.withColumnRenamed('_c{}'.format(i), col_name)

```

Thực hiện việc **đặt tên cột** (rename columns) cho hai DataFrame `train_set` và `val_set` được tải từ tệp CSV

- `columns = ['id', 'company', 'label', 'text']`: Đây là danh sách tên cột mới sẽ được gán cho DataFrame.
- Sau đó dùng vòng lặp để đổi tên các cột từ định dạng mặc định `'_c0', '_c1', ...` # sang tên có ý nghĩa hơn từ danh sách `'columns'`.

Kiểm tra dữ liệu

```

train_set.printSchema()
train_set.show(5, False)
train_set.count()

```

printSchema và **show**: Kiểm tra cấu trúc và dữ liệu sau khi đổi tên.

count(): Đếm số lượng dòng trong tập dữ liệu.

```

root
|-- id: integer (nullable = true)
|-- company: string (nullable = true)
|-- label: string (nullable = true)
|-- text: string (nullable = true)

+---+-----+-----+-----+
|id |company |label |text |
+---+-----+-----+-----+
|2401|Borderlands|Positive|im getting on borderlands and i will murder you all ,|
|2401|Borderlands|Positive|I am coming to the borders and I will kill you all,|
|2401|Borderlands|Positive|im getting on borderlands and i will kill you all,|
|2401|Borderlands|Positive|im coming on borderlands and i will murder you all,|
|2401|Borderlands|Positive|im getting on borderlands 2 and i will murder you me all,|
+---+-----+-----+-----+
only showing top 5 rows

74682

```

Xử lý giá trị thiếu

```

# drop missing value of text column
train_set = train_set.dropna(subset=['text'])
val_set = val_set.dropna(subset=['text'])
train_set.count()

```

Hàm Dropna dùng để loại bỏ các dòng mà cột text có giá trị null (thiếu dữ liệu).

Sau khi loại bỏ đếm lại số dòng trong tập dữ liệu train

```
# drop missing value of text column
train_set = train_set.dropna(subset=['text'])
val_set = val_set.dropna(subset=['text'])
train_set.count()
```

73996

Biến đổi nhãn (label)

```
# mapping label with index
label_indexer = StringIndexer(inputCol='label', outputCol='int_label')

label_indexer_model = label_indexer.fit(train_set)
train_set = label_indexer_model.transform(train_set)
val_set = label_indexer_model.transform(val_set)

# extract label mapping
label_mapping = label_indexer_model.labels

for i, val in enumerate(label_mapping):
    print(f'index {i} --> label {val}')

train_set.show(5, False)
```

label_indexer: Sử dụng StringIndexer để chuyển đổi các nhãn văn bản trong cột 'label' thành chỉ số số nguyên với cột đầu vào: 'label' (nhãn dạng text, ví dụ: 'positive', 'negative', 'neutral') và cột đầu ra: 'int_label' (chỉ số số nguyên tương ứng)

hàm fit() để huấn luyện mô hình StringIndexer trên tập train_set để gán chỉ số cho các nhãn (Gán chỉ số số nguyên cho mỗi nhãn, theo thứ tự giảm dần của tần suất xuất hiện).

Ví dụ:

"positive" → 0

"negative" → 1

"neutral" → 2

Hàm **transform()** để áp dụng mô hình để tạo thêm cột **int_label** trong train_set và val_set.

label_mapping để trích xuất ánh xạ giữa nhãn văn bản và chỉ số số nguyên (label mapping) sau đó in ra ánh xạ nhãn để kiểm tra.

```
# mapping label with index
label_indexer = StringIndexer(inputCol='label', outputCol='int_label')

label_indexer_model = label_indexer.fit(train_set)
train_set = label_indexer_model.transform(train_set)
val_set = label_indexer_model.transform(val_set)

# extract label mapping
label_mapping = label_indexer_model.labels

for i, val in enumerate(label_mapping):
    print(f'index {i} --> label {val}')

train_set.show(5, False)
```

```
index 0 --> label Negative
index 1 --> label Positive
index 2 --> label Neutral
index 3 --> label Irrelevant
```

id	company	label	text	int_label
2481	Borderlands	Positive	im getting on borderlands and i will murder you all ,	1.0
2481	Borderlands	Positive	i am coming to the borders and i will kill you all,	1.0
2481	Borderlands	Positive	im getting on borderlands and i will kill you all,	1.0
2481	Borderlands	Positive	im coming on borderlands and i will murder you all,	1.0
2481	Borderlands	Positive	im getting on borderlands 2 and i will murder you me all,	1.0

only showing top 5 rows

Chuẩn hóa văn bản

Hàm **clean_text** giúp chuẩn hóa văn bản bằng cách:

Loại bỏ URL.

Loại bỏ mentions (@) và hashtags (#).

Chuyển văn bản về chữ thường.

Loại bỏ các ký tự không phải chữ cái hoặc khoảng trắng.

```
# cleaning text
def clean_text(df, inputCol='text', outputCol='text'):
    # remove link https, http, ...
    df = df.withColumn(outputCol, regexp_replace(df[inputCol],
    r'https?://S+|www\\.S+|S+\\.com\\S+|youtu\\.be\\S+', ''))

    # remove starting with @ or #
    df = df.withColumn(outputCol, regexp_replace(df[inputCol], r'(@|#)w+', ''))

    # convert text to lowercase
    df = df.withColumn(outputCol, lower(df[inputCol]))

    # remove non-alpha characters
    df = df.withColumn(outputCol, regexp_replace(df[inputCol], r'^a-zA-Z\\s', ''))

    return df
```

Bước 1: Xóa các liên kết URL (http, https, www, .com, youtube links, v.v.)

Sử dụng `regex_replace` để thay thế các URL bằng chuỗi rỗng (")

- `https?://`: URL bắt đầu bằng `http://` hoặc `https://`.
- `\S+`: Một hoặc nhiều ký tự không phải khoảng trắng.
- `www\S+`: URL bắt đầu với `www..`
- `S+\.com\S+`: Các URL có đuôi `.com`.
- `youtu.be\S+`: URL của YouTube.

Bước 2: Xóa các từ bắt đầu bằng `@` (mentions) hoặc `#` (hashtags).

`regex_replace` để thay thế các từ bắt đầu bằng `@` hoặc `#` bằng chuỗi rỗng.

- `@/#`: Tìm các từ bắt đầu bằng `@` hoặc `#`.
- `\w+`: Một hoặc nhiều ký tự chữ hoặc số.

Bước 3: Chuyển văn bản thành chữ thường bang lower

Bước 4: Xóa các ký tự không phải chữ cái (non-alpha) và khoảng trắng

`regex_replace` để giữ lại chỉ các ký tự chữ cái (a-z, A-Z) và khoảng trắng (`\s`).

- `^`: Phủ định (nghĩa là bất kỳ ký tự nào không thuộc nhóm sau nó).
- `a-zA-Z`: Các ký tự chữ cái (hoa hoặc thường).
- `\s`: Các khoảng trắng.

```
# cleaning text
def clean_text(df, inputCol='text', outputCol='text'):
    # remove link https, http, ...
    df = df.withColumn(outputCol, regexp_replace(df[inputCol], r'https?://\S+|\S+\.com|\S+youtu.be/\S+', ''))
    # remove starting with @ or #
    df = df.withColumn(outputCol, regexp_replace(df[inputCol], r'(@#)\w+', ''))
    # convert text to lowercase
    df = df.withColumn(outputCol, lower(df[inputCol]))
    # remove non-alpha characters
    df = df.withColumn(outputCol, regexp_replace(df[inputCol], r'[^a-zA-Z\s]', ''))

    return df

[ ] cleaned_train_set = clean_text(train_set, inputCol='text', outputCol='text')
cleaned_val_set = clean_text(val_set, inputCol='text', outputCol='text')
cleaned_train_set.show(5, False)
```

id	company	label	text	int_label
2401	Borderlands	Positive	im getting on borderlands and i will murder you all	1.0
2401	Borderlands	Positive	i am coming to the borders and i will kill you all	1.0
2401	Borderlands	Positive	im getting on borderlands and i will kill you all	1.0
2401	Borderlands	Positive	im coming on borderlands and i will murder you all	1.0
2401	Borderlands	Positive	im getting on borderlands and i will murder you me all	1.0

only showing top 5 rows

Xây dựng Pipeline cho mô hình học máy

Giới thiệu

Trong phần này, chúng ta sẽ đi qua các bước xử lý dữ liệu và xây dựng các pipeline để huấn luyện hai mô hình học máy: Logistic Regression và RandomForest. Các bước bao gồm việc chuyển đổi dữ liệu văn bản thành các đặc trưng dạng số, loại bỏ từ kết thúc, và xây dựng các mô hình học máy.

Các bước xử lý dữ liệu trong pipeline

Tokenizer

Mục đích:

Chuyển đổi dữ liệu văn bản trong cột **text** thành các từ (token).

Cấu hình:

```
tokenizer = Tokenizer(inputCol='text', outputCol='token')
```

inputCol: Tên cột chứa văn bản gốc (text).

outputCol: Tên cột mới sẽ chứa các token (token).

StopWordsRemover

Mục đích:

Loại bỏ các từ dừng (stop words) từ cột chứa token, giúp giảm nhiễu trong dữ liệu.

Cấu hình:

```
stopword_remover = StopWordsRemover(inputCol='token', outputCol='filtered_token',  
stopWords=stop_words)
```

inputCol: Tên cột chứa các token (token).

outputCol: Tên cột mới sẽ chứa các token sau khi loại bỏ từ dừng (filtered_token).

stopWords: Danh sách các từ dừng được cung cấp bởi biến stop_words.

CountVectorizer

Mục đích:

Chuyển đổi các token đã lọc thành các đặc trưng dạng số, sử dụng biểu diễn Bag of Words.

Cấu hình:

```
count_vectorizer = CountVectorizer(inputCol='filtered_token', outputCol='features',  
vocabSize=10000, minDF=5)
```

inputCol: Tên cột chứa các token đã lọc (filtered_token).

outputCol: Tên cột mới sẽ chứa các đặc trưng dạng số (features).

vocabSize: Kích thước từ vựng tối đa là 10,000 từ.

minDF: Tần suất xuất hiện tối thiểu của một từ là 5.

Xây Dựng Mô Hình

Logistic Regression

Logistic Regression được chọn vì tính đơn giản và hiệu quả của nó trong việc giải quyết các bài toán phân loại, đặc biệt là khi dữ liệu có mối quan hệ tuyến tính giữa các đặc trưng và nhãn. Nó cũng có khả năng giải thích cao giúp người dùng hiểu rõ mỗi

quan hệ giữa các đặc trưng và kết quả phân loại. Hơn nữa với dữ liệu văn bản lớn, Logistic Regression có thể hoạt động ổn định và cho kết quả nhanh chóng.

Mục đích:

Huấn luyện mô hình Logistic Regression để phân loại dữ liệu.

Cấu hình:

```
lr = LogisticRegression(labelCol='int_label', featuresCol='features', maxIter= 100)
```

labelCol: Tên cột chứa nhãn (`int_label`).

featuresCol: Tên cột chứa các đặc trưng (`features`).

maxIter: Số vòng lặp tối đa để huấn luyện mô hình là 100.

Random Forest

Random Forest được chọn vì khả năng xử lý dữ liệu phi tuyến tính và các mối quan hệ phức tạp trong dữ liệu. Thuật toán này sử dụng tập hợp các cây quyết định, giúp mô hình có thể phân loại tốt hơn trong các trường hợp có nhiều yếu tố ảnh hưởng. Random Forest cũng có ưu điểm về độ chính xác cao, đặc biệt khi dữ liệu có sự thay đổi lớn giữa các đặc trưng.

Mục đích:

Huấn luyện mô hình RandomForest để phân loại dữ liệu.

Cấu hình:

```
rf = RandomForestClassifier(labelCol='int_label', featuresCol='features', numTrees=100, maxDepth=10)
```

labelCol: Tên cột chứa nhãn (`int_label`).

featuresCol: Tên cột chứa các đặc trưng (`features`).

numTrees: Số lượng cây trong rừng là 100.

maxDepth: Độ sâu tối đa của mỗi cây là 10.

Xây Dựng Pipeline

Pipeline cho Logistic Regression

Mục đích:

Kết hợp các bước xử lý dữ liệu và huấn luyện mô hình Logistic Regression thành một pipeline.

Cấu hình:

```
lr_pipeline = Pipeline(stages=[tokenizer, stopword_remover, count_vectorizer, lr])
```

Các bước trong pipeline bao gồm: Tokenizer, StopWordsRemover, CountVectorizer, Logistic Regression.

Pipeline cho RandomForest

Mục đích:

Kết hợp các bước xử lý dữ liệu và huấn luyện mô hình RandomForest thành một pipeline.

Cấu hình:

```
rf_pipeline = Pipeline(stages=[tokenizer, stopwordsremover, count_vectorizer, rf])
```

Các bước trong pipeline bao gồm: Tokenizer, StopWordsRemover, CountVectorizer, RandomForest.

Đánh giá mô hình

Sau khi huấn luyện mô hình nhóm sẽ sử dụng accuracy để đánh giá:

```
from pyspark.ml.evaluation import MulticlassClassificationEvaluator
evaluator = MulticlassClassificationEvaluator(labelCol="int_label", predictionCol="prediction",
metricName="accuracy")
lr_accuracy = evaluator.evaluate(lr_predictions)
rf_accuracy = evaluator.evaluate(rf_predictions)
print(f"Logistic Regression Accuracy on Training Set: {lr_accuracy}")
print(f"RandomForest Accuracy on Training Set: {rf_accuracy}")
```

Sử dụng MulticlassClassificationEvaluator từ thư viện của PySpark để đánh giá mô hình với ba tham số chính: labelCol="int_label" chỉ định cột chứa nhãn thực tế của dữ liệu, predictionCol="prediction" chỉ định cột chứa các giá trị dự đoán của mô hình và metricName="accuracy" để tính toán độ chính xác của mô hình.

Kết quả Accuracy của 2 mô hình:

```
Logistic Regression Accuracy on Training Set: 0.900089194010487
RandomForest Accuracy on Training Set: 0.45413265581923346
```

Kết quả cho thấy mô hình Logistic Regression có độ chính xác vượt trội so với Random Forest trên tập huấn luyện với một sự chênh lệch lớn trong accuracy (90.01% so với 45.41%). Điều này cho thấy Logistic Regression là lựa chọn phù hợp cho bài toán này.

Tiếp tục áp dụng mô hình Logistic Regression đã huấn luyện trên tập kiểm tra (validation set):

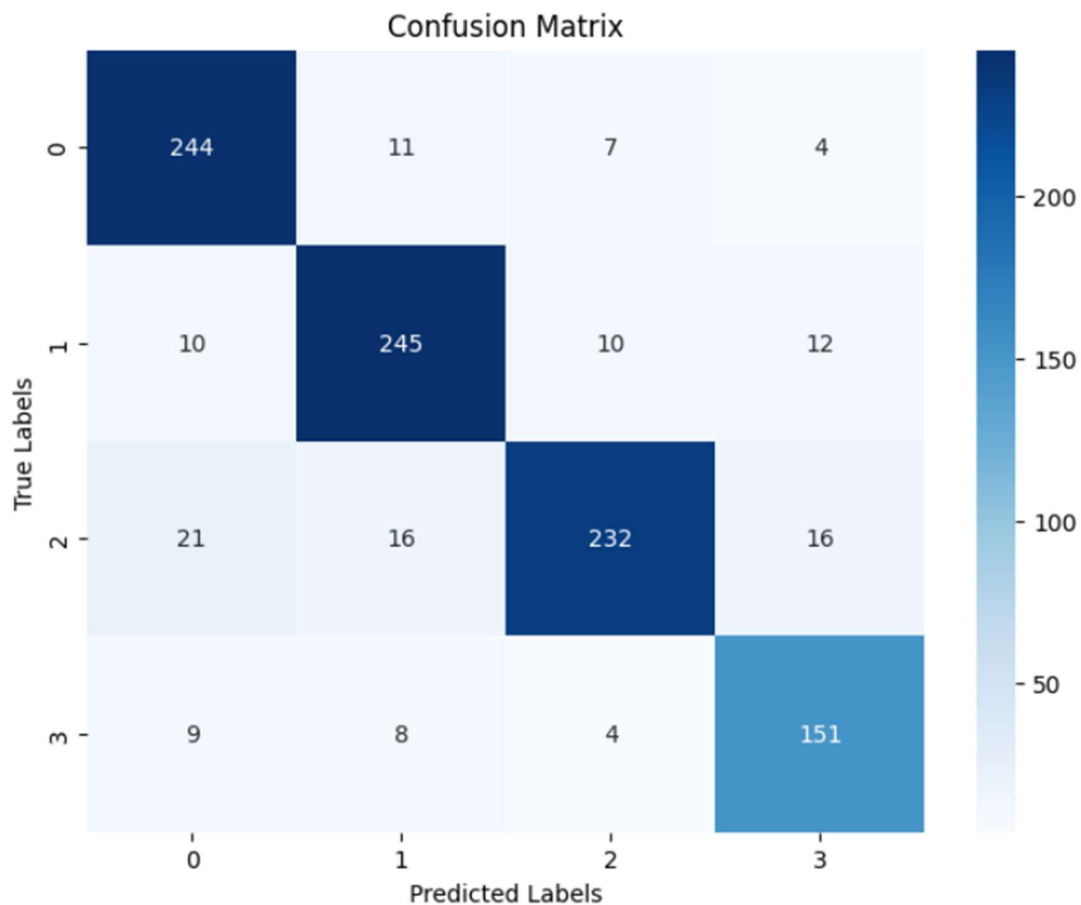
```
# apply model to validation set
processed_val_set = lr_model.transform(cleaned_val_set)

# Evaluate accuracy
evaluator = MulticlassClassificationEvaluator(labelCol="int_label", predictionCol="prediction",
metricName="accuracy")
accuracy = evaluator.evaluate(processed_val_set)
print("Accuracy:", accuracy)
```

Accuracy: 0.872

Mô hình Logistic Regression đạt accuracy = 0.872 (87,2%) cho thấy mô hình có thể tổng quát tốt và phân loại chính xác các bài viết trong tập kiểm tra.

Vẽ Confusion Matrix sẽ giúp hiểu rõ hơn về cách mô hình phân loại đúng và sai từng lớp (positive, negative, neutral, Irrelevant).



Lớp Positive có số lượng dự đoán chính xác cao với 244 mẫu. Một số mẫu bị nhầm lẫn với các lớp khác, bao gồm 11 mẫu nhầm thành lớp Negative, 7 mẫu nhầm thành lớp Neutral, và 4 mẫu nhầm thành lớp Irrelevant. Chứng tỏ mô hình hoạt động hiệu quả với lớp Positive, nhờ đặc trưng của lớp này rõ ràng và dễ phân biệt.

Lớp Negative có hiệu suất tốt nhất với 245 mẫu được dự đoán chính xác. Tuy nhiên, có 10 mẫu bị nhầm thành lớp Positive, 10 mẫu thành lớp Neutral, và 12 mẫu thành lớp Irrelevant. Sự nhầm lẫn giữa lớp Negative và Positive có thể do đặc trưng giữa hai lớp này có sự tương đồng.

Lớp Neutral có 232 mẫu được dự đoán đúng, nhưng vẫn có nhiều nhầm lẫn. Cụ thể, 21 mẫu bị nhầm thành lớp Positive, 16 mẫu thành lớp Negative, và 16 mẫu thành lớp Irrelevant. Hiệu suất thấp hơn của lớp này cho thấy đặc trưng của lớp Neutral chưa đủ rõ ràng để phân biệt với các lớp khác.

Lớp Irrelevant là lớp khó phân loại nhất, với chỉ 151 mẫu được dự đoán chính xác. Có 9 mẫu nhầm thành lớp Positive, 8 mẫu nhầm thành lớp Negative, và 4 mẫu nhầm thành lớp Neutral. Cho thấy đặc trưng của lớp Irrelevant chưa đủ mạnh để tách biệt rõ ràng khỏi các lớp còn lại.

Mô hình Logistic Regression đã thể hiện hiệu quả cao với độ chính xác tổng thể 87.2%, đặc biệt là trong việc phân loại các bài viết thuộc lớp Positive và Negative. Tuy nhiên, lớp Neutral và Irrelevant vẫn còn những hạn chế, với tỷ lệ nhầm lẫn đáng kể. Để cải thiện hiệu suất, cần tập trung vào cân bằng dữ liệu giữa các lớp, tăng cường đặc trưng văn bản, và thử nghiệm các thuật toán phân loại tiên tiến hơn. Những cải tiến này không chỉ giúp cải thiện khả năng phân loại mà còn đảm bảo tính tổng quát hóa tốt hơn của mô hình trên dữ liệu thực tế.

4.2.2 Spark Streaming

Xử lý dữ liệu streaming sử dụng Apache Spark để phân tích cảm xúc của các tweet trong thời gian thực. Cụ thể, mã sẽ đọc các bài tweet từ một thư mục chứa các file CSV, áp dụng các bước tiền xử lý để làm sạch văn bản (bao gồm việc loại bỏ URL, ký tự đặc biệt, và chuyển đổi văn bản thành chữ thường), sau đó sử dụng một mô hình học máy đã được huấn luyện sẵn để phân loại cảm xúc của từng tweet vào các nhóm: Positive (Tích cực), Negative (Tiêu cực), Neutral (Trung lập), và Irrelevant (Không liên quan). Quá trình này được thực hiện theo dạng streaming (xử lý theo từng dòng dữ liệu) và mỗi batch dữ liệu được xử lý một cách độc lập, với kết quả dự đoán được in ra cho từng tweet, bao gồm cả nội dung gốc và nội dung đã làm sạch. Kết quả phân loại sẽ giúp người sử dụng hiểu được cảm xúc chung của các tweet theo thời gian thực, phục vụ cho

các ứng dụng như phân tích xu hướng trên mạng xã hội, giám sát cảm xúc người dùng hoặc phân tích dữ liệu truyền thông. Toàn bộ quá trình chạy liên tục cho đến khi người dùng dừng lại, giúp hệ thống có thể xử lý và phân tích dữ liệu liên tục mà không bị gián đoạn.

```
import findspark # Thư viện findspark giúp tìm kiếm và cấu hình đường dẫn Spark khi sử dụng
Spark trong Python
findspark.init('C:\Spark\spark-3.5.3-bin-hadoop3') # Khởi tạo Spark với đường dẫn tới thư mục cài
đặt Spark (ở đây là C:\Spark\spark-3.5.3-bin-hadoop3)
import pyspark # Nhập khẩu thư viện pyspark để sử dụng SparkContext và các tính năng Spark
khác
sc = pyspark.SparkContext() # Tạo một SparkContext, điểm bắt đầu cho mọi chương trình Spark
import re # Nhập khẩu thư viện regex (regular expression) để xử lý các chuỗi văn bản
from pyspark.sql import SparkSession # Import SparkSession từ pyspark.sql, giúp tạo phiên làm
việc với Spark SQL
from pyspark.sql.functions import col, udf # Import col và udf (user-defined functions) từ
pyspark.sql.functions để thao tác với cột và tạo hàm tự định nghĩa
from pyspark.sql.types import StringType, StructType, StructField # Import các kiểu dữ liệu để
khai báo schema cho DataFrame
from pyspark.ml import PipelineModel # Import PipelineModel để tải mô hình đã huấn luyện trong
Spark MLlib
# Định nghĩa schema cho file CSV
schema = StructType([ # Định nghĩa cấu trúc của các trường trong file CSV
    StructField("id", StringType(), True), # Trường "id" là kiểu String và có thể là null
    StructField("company", StringType(), True), # Trường "company" là kiểu String và có thể là null
    StructField("label", StringType(), True), # Trường "label" là kiểu String và có thể là null
    StructField("tweet", StringType(), True), # Trường "tweet" là kiểu String và có thể là null
])

# Tạo SparkSession và cấu hình cho master cluster
spark = SparkSession.builder.appName("Spark
Streaming").master("spark://192.168.1.66:7077").config("spark.executor.memory",
"4g").config("spark.executor.cores", "4").getOrCreate()
# Đọc dữ liệu từ file CSV
csv_dir = "C:\Spark\spark_streaming\data" # Đường dẫn tới thư mục chứa dữ liệu CSV
df = spark \
    .readStream \
    .schema(schema) \
    .option("header", "false") \
    .csv(csv_dir)
# Hàm để làm sạch văn bản (tiền xử lý văn bản)
def clean_text(text): # Hàm này nhận vào một chuỗi văn bản và trả về văn bản đã được làm sạch
    if text is not None: # Nếu văn bản không phải là None
        text = re.sub(r'https?://\S+|www\.\S+|\S+\.com\S+|youtu\.be/\S+', "", text) # Loại bỏ các URL
        text = re.sub(r'(@|#)\w+', "", text) # Loại bỏ các ký hiệu @ và # trong văn bản
        text = text.lower() # Chuyển toàn bộ văn bản thành chữ thường
        text = re.sub(r'^a-zA-Z\s|', "", text) # Loại bỏ các ký tự không phải là chữ cái hoặc khoảng
trắng
        text = re.sub(r'\s+', ' ', text).strip() # Loại bỏ khoảng trắng thừa
        return text # Trả về văn bản đã làm sạch
    else:
        return "" # Nếu văn bản là None, trả về chuỗi rỗng
# Định nghĩa UDF (User Defined Function) để áp dụng hàm làm sạch văn bản
clean_text_udf = udf(clean_text, StringType()) # UDF sẽ trả về kiểu String sau khi làm sạch
# Tải mô hình đã huấn luyện từ file
```

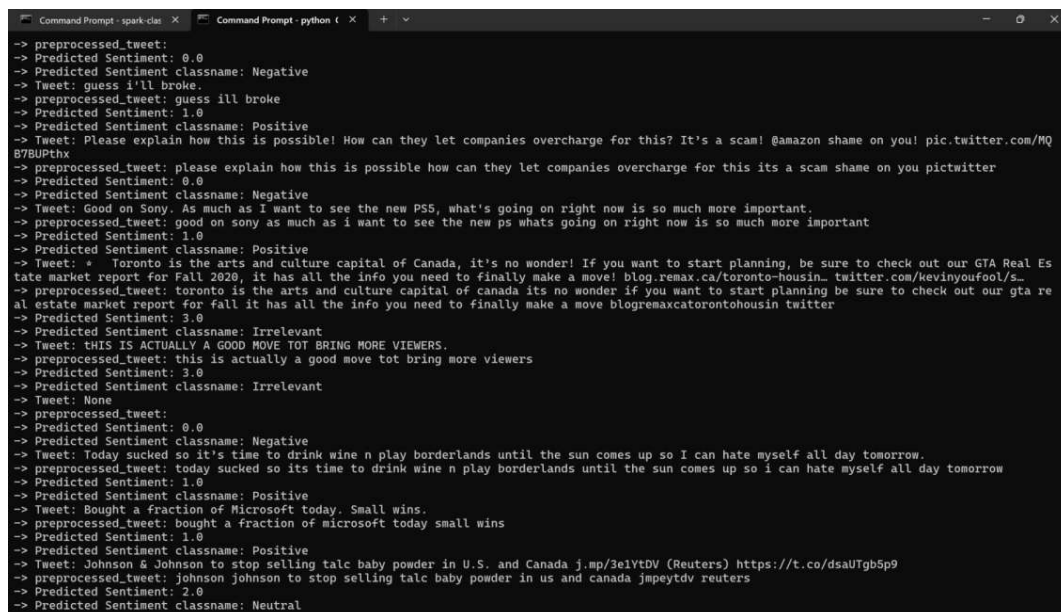
```

pipeline = PipelineModel.load('C:\Spark\model_spark\logistic_regression_model.pkl') # Tải mô
hình Logistic Regression đã huấn luyện từ file
# Định nghĩa hàm xử lý từng batch dữ liệu
class_index_mapping = {0: 'Negative', 1: 'Positive', 2: 'Neutral', 3: 'Irrelevant'} # Mã hóa chỉ số
thành nhãn cảm xúc
def process_batch(batch_df, batch_id): # Hàm này xử lý dữ liệu của từng batch trong dòng stream
    batch_df = batch_df.withColumn("text", clean_text_udf(col("tweet"))) # Áp dụng UDF để làm
sạch văn bản trong cột "tweet"
    processed_validation = pipeline.transform(batch_df) # Sử dụng mô hình đã huấn luyện để dự
đoán
    predictions = processed_validation.select("tweet", "text", "prediction").collect() # Lấy các kết
quả dự đoán từ DataFrame
    for row in predictions: # Duyệt qua từng dòng dự đoán
        tweet = row['tweet'] # Lấy tweet gốc
        preprocessed_tweet = row['text'] # Lấy tweet đã được làm sạch
        prediction = row['prediction'] # Lấy chỉ số dự đoán của mô hình
        prediction_classname = class_index_mapping[int(prediction)] # Lấy tên lớp dự đoán từ chỉ số

    # In ra kết quả dự đoán
    print("-> Tweet:", tweet)
    print("-> preprocessed_tweet:", preprocessed_tweet)
    print("-> Predicted Sentiment:", prediction)
    print("-> Predicted Sentiment classname:", prediction_classname)
# Khởi động streaming query
query = df.writeStream \
    .foreachBatch(process_batch) \
    .outputMode('append') \
    .start()
query.awaitTermination()
# Dừng SparkSession khi kết thúc
spark.stop()

```

Kết quả output trên máy master



```

-> preprocessed_tweet:
-> Predicted Sentiment: 0.0
-> Predicted Sentiment classname: Negative
-> Tweet: guess i'll broke.
-> preprocessed_tweet: guess ill broke
-> Predicted Sentiment: 1.0
-> Predicted Sentiment classname: Positive
-> Tweet: Please explain how this is possible! How can they let companies overcharge for this? It's a scam! @amazon shame on you! pic.twitter.com/MQ
gYUUPuhv
-> preprocessed_tweet: please explain how this is possible how can they let companies overcharge for this its a scam shame on you pictwitter
-> Predicted Sentiment: 0.0
-> Predicted Sentiment classname: Negative
-> Tweet: Good on Sony. As much as I want to see the new PS5, what's going on right now is so much more important.
-> preprocessed_tweet: good on sony as much as i want to see the new ps whats going on right now is so much more important
-> Predicted Sentiment: 1.0
-> Predicted Sentiment classname: Positive
-> Tweet: * Toronto is the arts and culture capital of Canada, it's no wonder! If you want to start planning, be sure to check out our GTA Real Es
tate market report for Fall 2020, it has all the info you need to finally make a move! blog.remax.ca/toronto-housin_ twitter.com/kevinyoufool/s_
-> preprocessed_tweet: toronto is the arts and culture capital of canada its no wonder if you want to start planning be sure to check out our gta re
al estate market report for fall it has all the info you need to finally make a move blogremaxcatorontohousin twitter
-> Predicted Sentiment: 3.0
-> Predicted Sentiment classname: Irrelevant
-> Tweet: THIS IS ACTUALLY A GOOD MOVE TOT BRING MORE VIEWERS.
-> preprocessed_tweet: this is actually a good move tot bring more viewers
-> Predicted Sentiment: 3.0
-> Predicted Sentiment classname: Irrelevant
-> Tweet: None
-> preprocessed_tweet:
-> Predicted Sentiment: 0.0
-> Predicted Sentiment classname: Negative
-> Tweet: Today sucked so it's time to drink wine n play borderlands until the sun comes up so I can hate myself all day tomorrow.
-> preprocessed_tweet: today sucked so its time to drink wine n play borderlands until the sun comes up so i can hate myself all day tomorrow
-> Predicted Sentiment: 1.0
-> Predicted Sentiment classname: Positive
-> Tweet: Bought a fraction of Microsoft today. Small wins.
-> preprocessed_tweet: bought a fraction of microsoft today small wins
-> Predicted Sentiment: 1.0
-> Predicted Sentiment classname: Positive
-> Tweet: Johnson & Johnson to stop selling talc baby powder in U.S. and Canada j.mp/3e1YtDV (Reuters) https://t.co/dsaUTgb5p9
-> preprocessed_tweet: johnson johnson to stop selling talc baby powder in us and canada jmpeytdv reuters
-> Predicted Sentiment: 2.0
-> Predicted Sentiment classname: Neutral

```

1 ví dụ trong output

("Bought a fraction of Microsoft today. Small wins.):

Tweet gốc: "Bought a fraction of Microsoft today. Small wins."

Tweet đã làm sạch: "bought a fraction of microsoft today small wins" (Văn bản đã được làm sạch tương tự).

Dự đoán cảm xúc: Mô hình phân loại cảm xúc này là Positive (Tích cực) với giá trị dự đoán là 1.0. Đây là một câu ngắn gọn và tích cực liên quan đến việc đạt được thành công nhỏ, do đó mô hình đã phân loại là tích cực.

TÀI LIỆU THAM KHẢO

1. Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, Ion Stoica “*Spark: Cluster Computing with Working Sets*”, 2010
[Zaharia.pdf \(usenix.org\)](#)
2. Grzegorz Piwowarek, “*Introduction to Apache Spark*”, Baeldung, 2024.
[Introduction to Apache Spark | Baeldung](#)
3. SharkByExamples, “*Apache Spark 3.5 Tutorial with Examples*”,
[Apache Spark Tutorial with Examples - Spark By {Examples}](#)
[\(sparkbyexamples.com\)](#)
4. Databricks, “*What is Spark SQL*”, [What is Spark SQL? \(databricks.com\)](#)
5. [ChatGPT](#)